

Curso de Godot







Sumário

1	Introdução	8
2	Instalação da Godot Engine	9
2.1	Baixando Godot com Voxel Tools	9
2.2	Instalação da Godot com Voxel Tools	9
3	Criando um Novo Projeto	11
4	Início da Lógica de Programação em GDScript	12
4.1	Criando um Script	13
4.2	Entendendo o Script Gerado	15
4.3	A Função <code>_ready()</code>	15
4.4	A Função <code>_process(delta)</code>	16
4.5	Criando uma Função	17
4.6	Chamando Funções de Outro Nó	18
5	Estruturas Condicionais em GDScript	20
5.1	O Básico da Estrutura <code>if</code>	20
5.2	Usando <code>else</code> para Criar Alternativas	20
5.3	Adicionando <code>elif</code> para Múltiplas Condições	20
5.4	Operadores de Comparação	21
5.5	Operadores Lógicos	21
5.6	Exercício Prático	21
5.7	Conclusão	22
6	Estruturas de Repetição em GDScript	23
6.1	O Loop <code>while</code>	23
6.1.1	Evitando Loops Infinitos	23
6.2	O Loop <code>for</code>	23
6.2.1	Iterando Sobre Intervalos Numéricos	23
6.2.2	Iterando Sobre Listas	24
6.3	Usando <code>break</code> e <code>continue</code>	24
6.3.1	Interrompendo um Loop com <code>break</code>	24
6.3.2	Pulando Iterações com <code>continue</code>	25
6.4	Loop Aninhado	25
6.5	Exercício Prático	25
7	Preparação do Ambiente de Desenvolvimento do Projeto	27
7.1	Estrutura de Pastas	27
7.2	Assets	28
7.3	Convenções	29
7.3.1	Convenções de Nomenclatura	29
7.3.2	Uso do Padrão ASCII	30
8	Criação do Terreno e do Personagem	31
8.1	Cena Principal	31
8.2	Criação do Terreno	32
8.2.1	Criando o Terreno	32
8.2.2	Adicionando Textura ao Terreno	33
8.3	Criação do Personagem	36
8.3.1	Importando o Modelo do Personagem	36
8.3.2	Criando a Lógica do Personagem	37
8.3.3	Adicionando Colisão ao Personagem	38



8.3.4	Adicionando uma Câmera ao Personagem	40
8.4	Adicionando o Personagem e o Terreno à Cena Principal	42
8.4.1	Instanciando as Cenas	42
8.4.2	Adicionando Iluminação Temporária	43
8.5	Controle do Personagem	44
8.5.1	Alterando os Controles	45
8.5.2	Adicionando Controle da Câmera com o Mouse	46
8.5.3	Criando o UI Manager	47
8.5.4	Ocultando as Meshes em Primeira Pessoa	48
9	Adicionando Mais Câmeras ao Personagem	50
9.1	Adicionando Câmeras de Segunda e Terceira Pessoa	50
9.2	Implementando a Lógica das Câmeras	52
10	Criando a Animação do Personagem	54
10.1	Adicionando os Nós de Animação	54
10.2	Criando a Animação (AnimationPlayer)	55
10.2.1	Criando as Faixas de Rotação	57
10.3	Criando as Transições de Animação (AnimationTree)	60
10.3.1	Criando as Transições entre os Estados	60
10.3.2	Controlando as Transições com Variáveis	61
10.3.3	Ajustando a Suavidade da Transição	61
10.4	Controlando as Animações Via Script	62
10.4.1	Controlando a Animação de Movimentação	62
10.4.2	Controlando a Animação da Cabeça	63
11	Criando os Menus	65
11.1	Criando a Tela Inicial	65
11.1.1	Adicionando o Background	65
11.1.2	Adicionando o Título	66
11.1.3	Adicionando os Botões	67
11.1.4	Adicionando o Footer de Informações	68
11.1.5	Acrescentando um Pouco de Estilo	69
11.1.6	Criando o Script de Transição	71
11.2	Criando o Menu de Pausa	73
11.2.1	Criando a Cena	73
11.2.2	Criando o Script de Pause	75
12	Criação do Inventário do Personagem	78
12.1	Criação da Cena do Inventário	78
12.2	Criação da Cena do Slot	79
12.3	Criação do Script do Inventário	80
12.3.1	Criação da Classe dos Itens	80
12.3.2	Criação do InventoryManager	81
12.3.3	Criação do Script do Inventário	86
12.4	Criação do Script do Slot	88
12.4.1	Implementando Drag and Drop	89
12.5	Criando um Item	91
12.6	Ajustando o Layout do Slot	92
12.7	Ajustando Layout do Inventário	95
12.7.1	Ajustando o Layout	95
12.7.2	Criando Tema	98
12.7.3	Testando o Inventário	98
12.8	Integrando o Inventário ao Jogo	100



13 Sistema de Dia e Noite	102
13.1 Introdução ao Sistema	102
13.2 Elementos	102
13.2.1 Sol e Lua	102
13.2.2 Céu	104
13.2.3 Tempo	105
13.3 Configuração do Script	105
13.4 Funcionamento do Código	107
13.4.1 Processamento do Tempo	107
13.4.2 Rotação do Sol e da Lua	108
13.4.3 Iluminação	109
13.4.4 Configurando no Godot	109
13.4.5 Configurando no Godot	109
14 Mecânicas de Cultivo - Planta	111
14.1 Configurando as Entradas de Controle	111
14.2 Cena do Milho	112
14.2.1 O que é um StaticBody3D?	112
14.3 Adicionando os Componentes da Planta	113
14.4 Ajustando as Colisões	114
14.4.1 Ajuste do Colisor do Small	115
14.4.2 Ajuste do Colisor do Medium	116
14.4.3 Ajuste do Colisor do Large	117
14.4.4 Scripts de Plantas	118
14.5 Criando o Script do Milho	118
14.5.1 Definindo as Variáveis da Planta	119
14.5.2 Funções Básicas da Planta	119
14.5.3 Atualizando a Interface do Jogador	120
14.5.4 Implementando o Crescimento da Planta	120
14.5.5 Código Completo do Crop.gd	121
15 Mecânicas de Cultivo - Plantio	124
15.1 Preparando o Terreno	124
15.1.1 Aplicando Cor ao Solo	125
15.1.2 Ajustando o Formato do Terreno	128
15.1.3 Configuração de Colisão	128
15.2 Player Interacter	129
15.2.1 Implementando a Função de Raycast	131
15.2.2 Adicionando Plantation à Crop	133
15.2.3 Implementando a Função de Preparar o Solo	133
15.2.4 Implementando a Função de Plantar	134
15.2.5 Regar e Fertilizar	136
15.2.6 Função de Colher	137
15.2.7 Função process	138
16 Criando HUD	139
16.1 Criando a Estrutura da HUD	139
16.2 Configurando a HUD	140
16.3 Criando o Script da HUD	141
16.4 Conectando a HUD ao Interacter	142



17	Animando Ferramentas	143
17.1	Criando a Estrutura da Animação	143
17.2	Criando a Animação da Ferramenta	144
17.3	Criando Animações Adicionais	146
17.3.1	Animação idle	146
17.3.2	Animação interagir	146
17.4	Configuração do AnimationTree	147
17.4.1	Configuração do AnimationTree no Inspetor	147
17.4.2	Adicionando Estados de Animação	148
17.4.3	Criando Transições Entre os Estados	148
17.4.4	Adicionando Condições às Transições	149
18	Hotbar	151
18.1	Estruturando a Hotbar	151
18.2	Criando um Resource para a Hotbar	155
18.2.1	Criando o Resource de Estilização	155
18.3	Implementando o Script da Hotbar	157
19	Finalizando Mecânicas de Cultivo	160
19.1	Sincronizando o AnimationTree do Hand com o Player	160
19.2	Ativando a Animação de Interação	160
19.3	Sincronizando Item da Hotbar com Animação	161
19.3.1	Criando a Cena da Enxada	161
19.3.2	Criando o Recurso da Enxada	162
19.4	Sincronizando a Hotbar com o Sistema de Ferramentas	163
19.4.1	Modificando o Script do HUD	163
19.4.2	Criando o Script para o Nó Hand	163
19.4.3	Atualizando o Item na Mão do Jogador	164
19.5	Finalizando o Interacter	165
19.5.1	Atualizando a Função _process	165
19.5.2	Explicação das Modificações	167
19.5.3	Conclusão	167
20	Implementando a Loja	168
20.1	Visão Geral da Loja	168
20.2	Explicando o Código	168
20.2.1	Script shop.gd	168
20.2.2	Script shop_items.gd	169
20.2.3	Script shop_manager.gd	169
20.3	Configuração no Editor	170
21	Criando a Galinha	171
21.1	Criando a Cena da Galinha	171
21.2	Criando o Script da Galinha	171
21.2.1	Definição do script e variáveis	171
21.2.2	Inicialização da galinha	171
21.2.3	Contagem regressiva para botar ovos	172
21.2.4	Movimento da galinha	172
21.2.5	Escolha aleatória de direção	172
21.2.6	Interação do jogador com a galinha	173
21.2.7	Exibição do tempo para o próximo ovo	173
21.3	Configuração no Editor	174
21.3.1	Criando a Cena da Galinha	174
21.3.2	Importando o Modelo da Galinha	174
21.3.3	Adicionando o Script na Cena	174



21.3.4	Configurando o CollisionShape3D	174
21.3.5	Configurando a Camada de Colisão	175



1 Introdução

Esta apostila foi desenvolvida como material de apoio para o **Curso de Godot** ministrado pelo **PET de Computação e Design** da **Universidade Estadual de Londrina (UEL)**. O curso tem como objetivo introduzir os participantes ao desenvolvimento de jogos utilizando a **Godot Engine**, uma ferramenta poderosa e de código aberto amplamente utilizada para criar jogos 2D e 3D.

Ao longo deste curso, os participantes aprenderão conceitos fundamentais do desenvolvimento de jogos, incluindo:

- Princípios básicos da **programação** com a linguagem **GScript**;
- Estrutura e organização de um projeto na **Godot Engine**;
- Fundamentos de **modelagem 3D** e uso de **assets** no jogo;
- Conceitos essenciais de **game design** e mecânicas de jogabilidade;
- Desenvolvimento de sistemas interativos e inteligência artificial básica.

Para consolidar o aprendizado, ao longo do curso **desenvolveremos juntos um jogo no estilo fazenda**, onde serão aplicados diversos conceitos adquiridos nas aulas. Esse projeto proporcionará uma experiência prática e permitirá que os participantes construam uma base sólida para o desenvolvimento de futuros jogos.

A apostila acompanha o curso passo a passo, fornecendo suporte teórico e prático para os exercícios e desafios propostos. Ao final, os alunos terão adquirido um conhecimento abrangente sobre o desenvolvimento de jogos, podendo aplicar esses conceitos em projetos próprios.

Nota: Não é necessário ter experiência prévia com a **Godot Engine** ou programação. O curso foi estruturado para atender desde iniciantes até aqueles que já possuem algum conhecimento na área.



2 Instalação da Godot Engine

Para iniciar o desenvolvimento do jogo, é necessário configurar corretamente o ambiente de desenvolvimento. Neste curso, utilizaremos a **Godot Engine 4.0** com **Voxel Tools**, um conjunto de ferramentas que permite a geração procedural de terrenos em tempo real, ajustando o mundo conforme a necessidade.

2.1 Baixando Godot com Voxel Tools

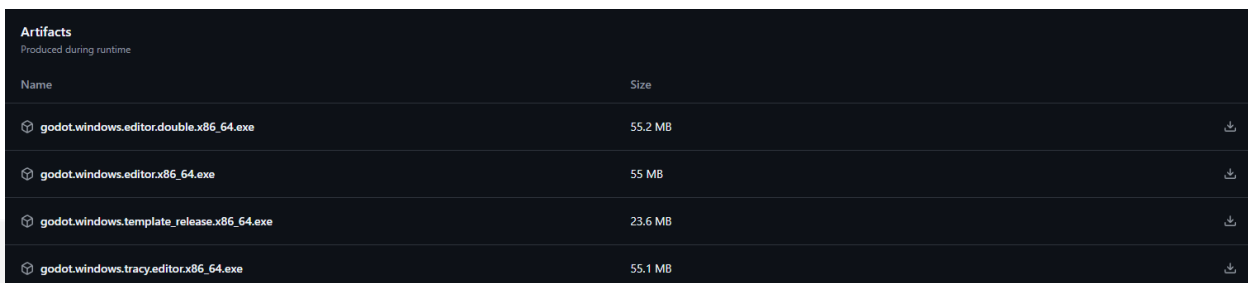
O **Voxel Tools** é um módulo adicional da Godot, desenvolvido por **Zylann**, que permite a manipulação de terrenos voxelizados. O código-fonte e os arquivos de instalação podem ser encontrados no seguinte repositório:

- [Repositório Oficial do Voxel Tools](#)
- [Escolha do Sistema Operacional\(Windows, MacOS e Linux\)](#)

Para baixar a versão correta, siga os passos abaixo:

1. Acesse o link do [download](#).
2. Escolha o Sistema Operacional em que será instalado.
3. Clique na **versão mais recente** disponível (A primeira).
4. Role até o final da página e baixe o arquivo `godot.windows.editor.double.x86-64.exe`.

A Figura 1 ilustra onde encontrar os arquivos para download.



Artifacts		
Produced during runtime		
Name	Size	
 <code>godot.windows.editor.double.x86_64.exe</code>	55.2 MB	
 <code>godot.windows.editor.x86_64.exe</code>	55 MB	
 <code>godot.windows.template_release.x86_64.exe</code>	23.6 MB	
 <code>godot.windows.tracy.editor.x86_64.exe</code>	55.1 MB	

Figura 1: Download do Voxel Tools no GitHub

Nota: Certifique-se de baixar a versão compatível com a **Godot Engine 4.0**, pois versões anteriores ou posteriores podem apresentar incompatibilidades.

Na sua pasta de downloads terá uma pasta .zip com o nome "godot.windows.editor.double.x86-64.exe", Basta unzipar a pasta e executar o programa.

O windows pode acusar como ameaça (imagem do windows defender), Basta clicar em "Mais informações" seguido de "Executar mesmo assim"

2.2 Instalação da Godot com Voxel Tools

Após baixar o arquivo de instalação, siga os passos abaixo para configurar corretamente a **Godot Engine 4.0** com **Voxel Tools** no Windows:

1. Localize o arquivo `godot.windows.editor.double.x86-64.exe.zip` na sua pasta de **Downloads**.



2. Extraia o conteúdo do arquivo **.zip** para um diretório de sua preferência.
3. Dentro da pasta extraída, execute o arquivo **godot.windows.editor.double.x86-64.exe** para iniciar o editor.

Durante a primeira execução, no caso de Windows, o **Windows Defender** pode exibir um alerta de segurança semelhante ao mostrado na Figura 2.

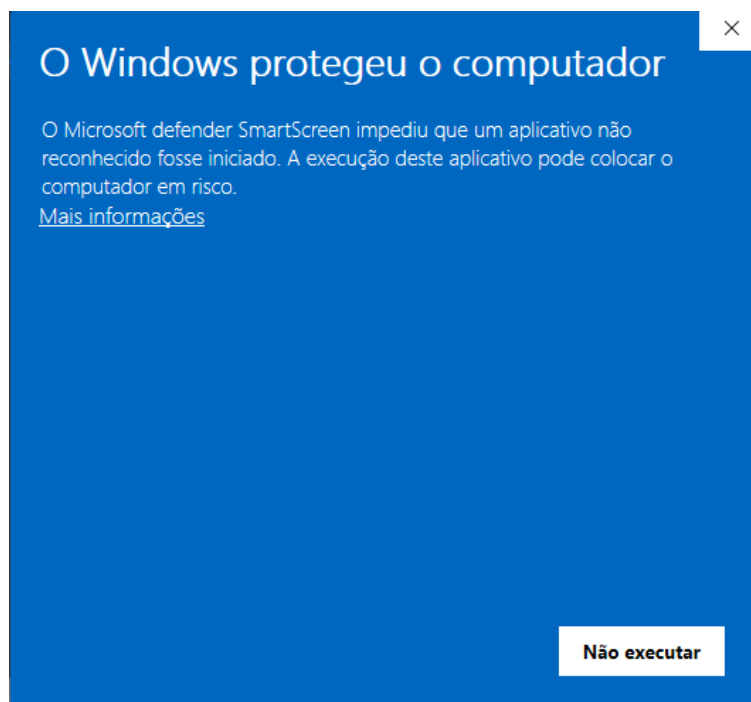


Figura 2: Aviso do Windows Defender ao executar a Godot

Nota: Se esse aviso aparecer, clique em **Mais Informações** e, em seguida, em **Executar mesmo assim** para permitir a execução do programa.



3 Criando um Novo Projeto

Após abrir a **Godot Engine**, você será recebido pela tela inicial do programa, conforme mostrado na Figura 3.

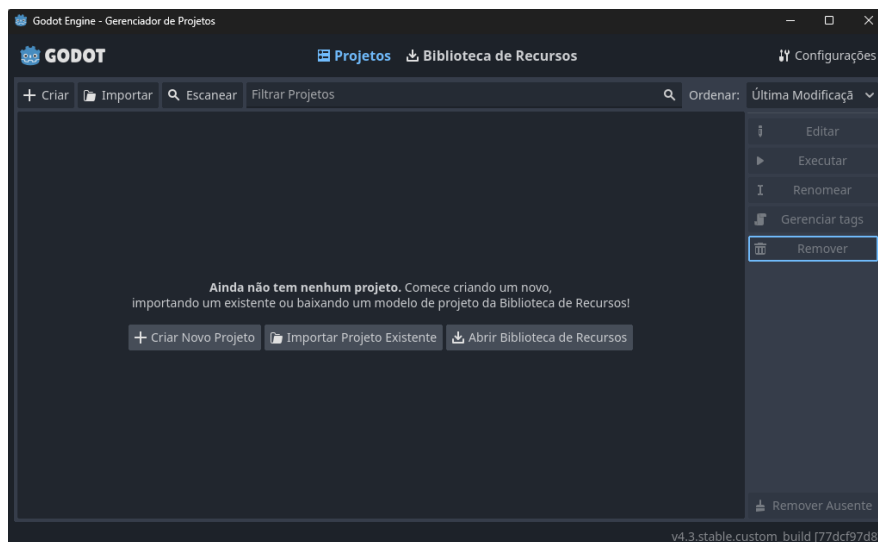


Figura 3: Tela inicial da Godot Engine

Para criar um novo projeto, siga os passos abaixo:

1. Clique no botão **Criar Novo Projeto**.
2. A seguinte janela será exibida (Figura 4).
3. Escolha um nome para o projeto e defina o diretório onde ele será salvo.
4. Opcionalmente, é possível selecionar o **Renderizador** que será utilizado. Para este curso, deixe a opção padrão.
5. Defina um nome como **projeto1** e clique em **Criar e Editar**.

Agora seu projeto foi criado, e você está dentro do ambiente de desenvolvimento da **Godot**. Nos próximos capítulos, aprenderemos a utilizar as ferramentas que a engine oferece e iniciaremos a construção do nosso primeiro jogo.

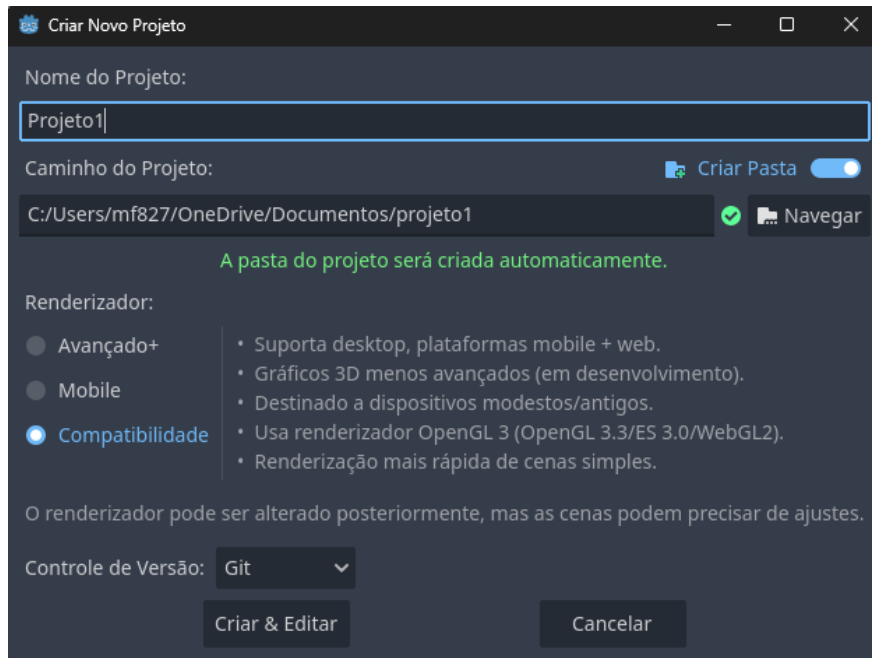


Figura 4: Janela de criação de um novo projeto

4 Início da Lógica de Programação em GDScript

Agora apresentaremos noções iniciais de programação e como programar dentro da **Godot Engine**. Antes de começarmos a escrever código, precisamos criar uma cena básica. No canto superior esquerdo da interface, é possível ver opções para criar um **Nó Raiz**, ou seja, o tipo da cena que será criada.

Na **Godot**, os jogos são baseados em uma estrutura hierárquica de **nós** (*nodes*), onde cada cena é composta por diferentes nós que interagem entre si. Cada nó pode representar diferentes elementos do jogo, como personagens, objetos interativos ou interfaces gráficas.

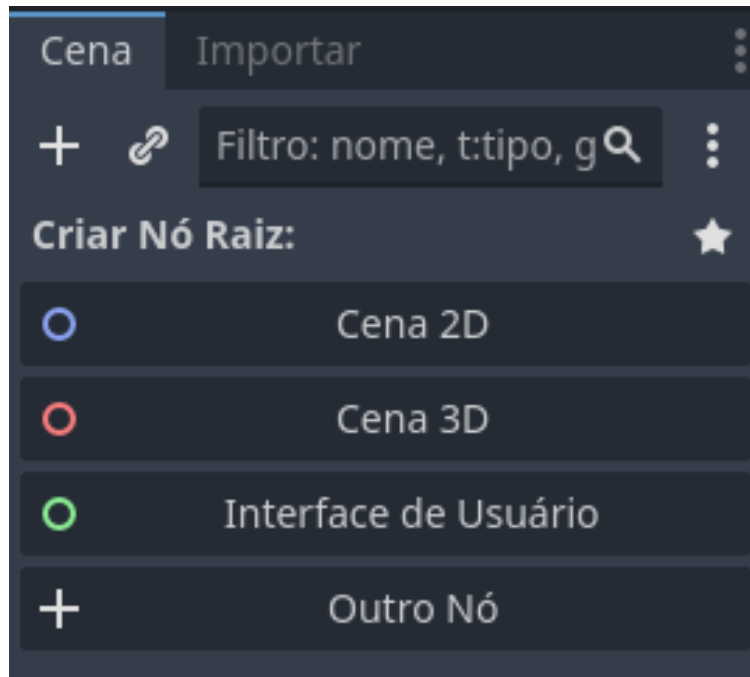


Figura 5: Opções para a criação do Nó Raiz

Clique na opção **Outro Nó** para visualizar a lista completa de nós disponíveis (Figura 6).

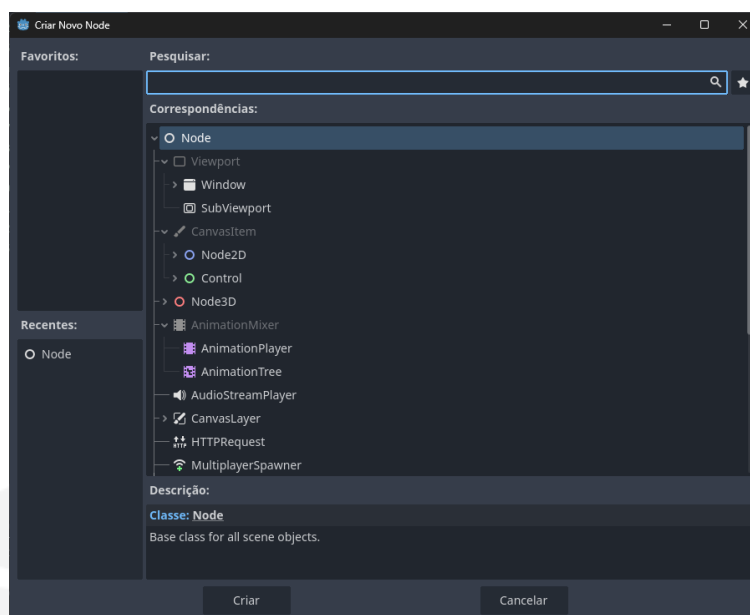


Figura 6: Janela para seleção de Nó

Após selecionar o nó desejado, clique em **Criar** para gerar uma cena com um **Node** padrão. Agora podemos começar a programação.

4.1 Criando um Script

Para criar um script vinculado a um nó, basta selecionar o nó desejado e clicar no ícone de **Script** (Figura 7).

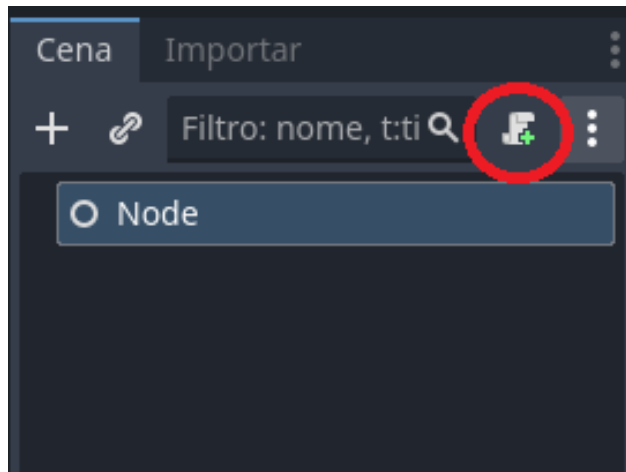


Figura 7: Ícone para criação de um script

Após clicar para criar o script, será exibida a janela da Figura 8.

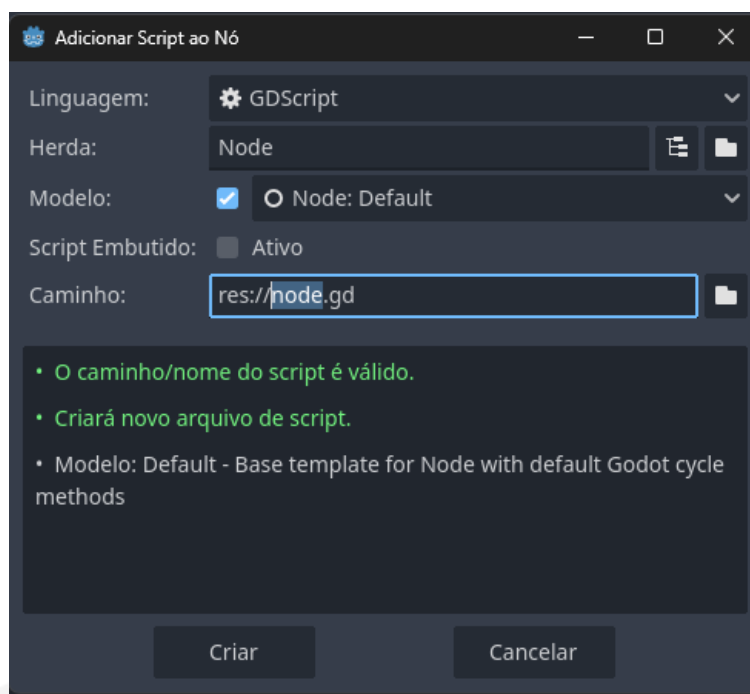


Figura 8: Janela para criação do script

Basta pressionar **Criar** (é possível renomear o script, se desejar). Assim que o script for aberto, ele terá o seguinte código gerado automaticamente:



```
extends Node

# Chamado quando o nó entra na árvore de cena pela primeira vez.
func _ready() -> void:
    pass # Substitua pelo corpo da função.

# Chamado a cada frame. 'delta' é o tempo decorrido desde o frame anterior.
func _process(delta: float) -> void:
    pass
```

4.2 Entendendo o Script Gerado

O script começa com a linha:

```
extends Node
```

Isso significa que esse script pertence a um nó da classe **Node**. Isso é muito útil, pois a **Godot** já oferece diversas funções prontas para diferentes classes de nós. Se, por exemplo, a classe **Node** tivesse a função **respirar()**, todos os nós da classe **Node** poderiam utilizá-la sem precisar reescrevê-la.

4.3 A Função _ready()

A função **_ready()** é executada assim que o nó é criado. Todos os nós possuem essa função, e ela é útil para inicializar variáveis, carregar recursos ou configurar elementos da cena logo no início do jogo.

Por exemplo, ao adicionar a linha:

```
func _ready() -> void:
    print("Foi criado")
```

E executar o jogo pressionando o botão de **Play** (Figura 9), será exibida a seguinte janela:

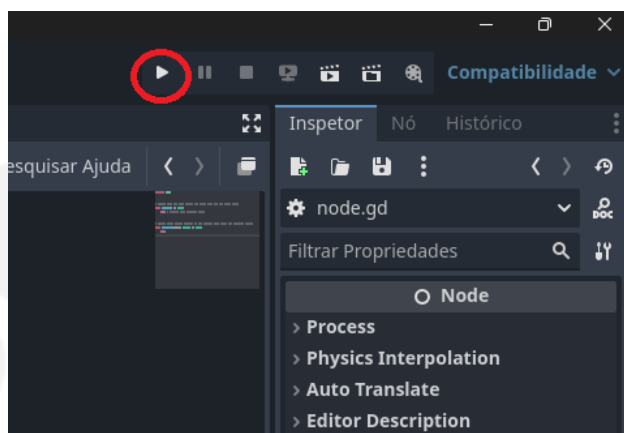


Figura 9: Botão para rodar o jogo

Nessa janela, selecione a opção **Selecionar Atual** (Figura 10).

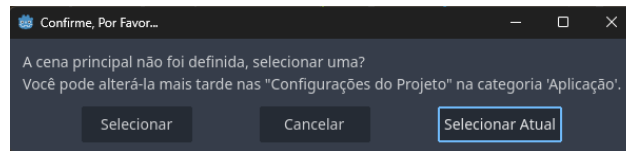


Figura 10: Janela para selecionar a cena principal

Isso executará o jogo, que ainda está vazio (por isso a tela aparecerá cinza). No entanto, podemos observar que no terminal foi escrita a mensagem:

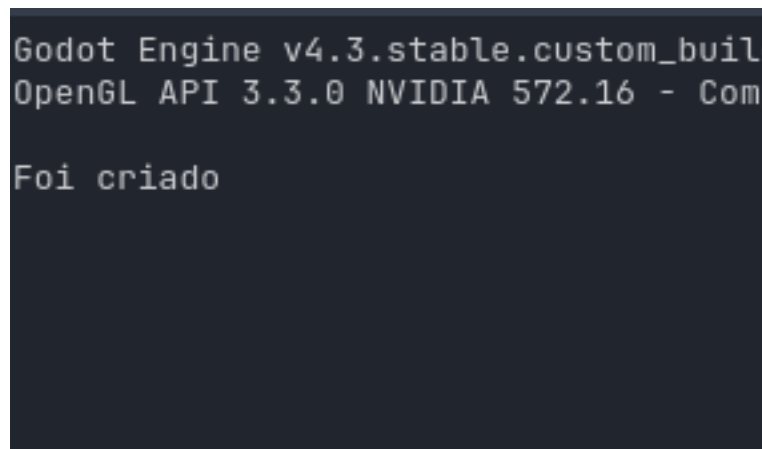


Figura 11: Saída do terminal - Foi criado

Isso confirma que o nó raiz **Node** foi criado corretamente na cena.

4.4 A Função `_process(delta)`

Além da função `_ready()`, outro método importante é `_process()`:

```
func _process(delta: float) -> void:  
    pass
```

A diferença entre `_ready()` e `_process()` é que `_ready()` executa apenas uma vez, no momento em que o nó é criado, enquanto `_process()` é executada constantemente, a cada quadro (*frame*) do jogo.

Podemos testar isso adicionando a seguinte linha dentro da função `_process()`:

```
func _process(delta: float) -> void:  
    print("Estou VIVO")
```

Agora, ao executar o jogo novamente, podemos ver no terminal que ele fica constantemente imprimindo "Estou VIVO", demonstrando que essa função roda continuamente enquanto o nó existir.



```
Estou VIVO  
Estou VIVO  
Estou VIVO  
Estou VIVO  
Estou VIVO  
Estou VIVO  
Estou VIVO  
Estou VIVO  
Estou VIVO  
Estou VIVO  
Estou VIVO  
Estou VIVO
```

Figura 12: Saída do terminal - Estou VIVO

Essas duas funções são apenas algumas das várias disponíveis na **Godot** para programarmos a lógica do jogo.

4.5 Criando uma Função

Em GDScript, funções são blocos de código reutilizáveis que podem ser chamados a qualquer momento no jogo. Para criar uma função, basta adicioná-la no script com a palavra-chave **func**, conforme o exemplo abaixo:

```
func OlaMundo():  
    print("Ola Mundo!")
```

Agora, para chamar essa função automaticamente quando o jogo inicia, devemos adicioná-la dentro da função **_ready()**:

```
func _ready() -> void:  
    print("Foi criado")  
    OlaMundo()  
pass # Replace with function body.
```

Com essa alteração, ao executar o jogo novamente, podemos ver que a nova função está funcionando corretamente e imprimindo a mensagem no terminal.



```
Foi criado  
Ola Mundo!  
Estou VIVO  
Estou VIVO  
Estou VIVO  
Estou VIVO
```

Figura 13: Saída do terminal - Ola Mundo

4.6 Chamando Funções de Outro Nó

Em um jogo, é comum que diferentes nós precisem se comunicar entre si, chamando funções uns dos outros. Para isso, é necessário ter mais de um Nó na cena. Vamos adicionar um novo Nó à cena e testar essa funcionalidade.

Para adicionar um Nó Filho:

1. Selecione o Nó Raiz.
2. Pressione o atalho **Ctrl + A** para abrir a janela de criação de nós (Figura 14).
3. No campo de busca, digite **Node**, selecione a opção **Node** e clique em **Criar**.

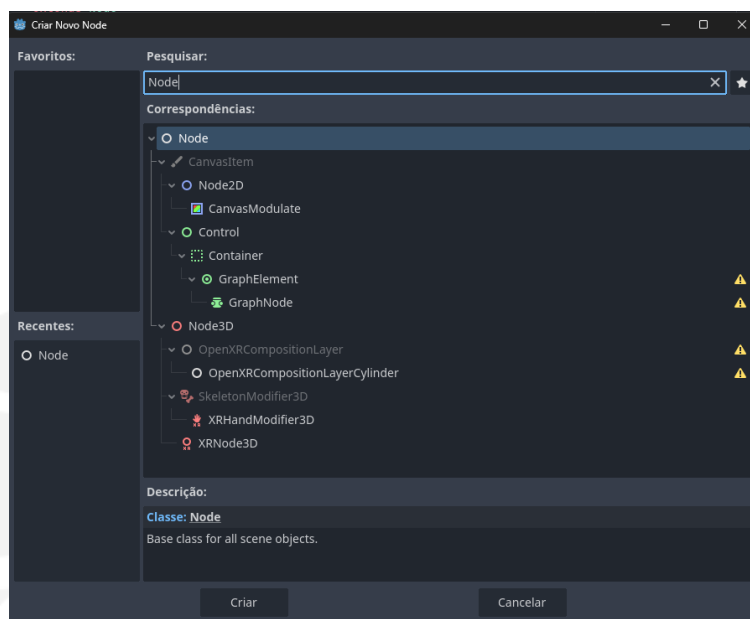


Figura 14: Janela para criar Nó Filho



Agora temos dois nós na cena: um **Nó Pai** e um **Nó Filho**. O próximo passo é editar a função `_ready()` do Nó Filho para chamar uma função do Nó Pai.

Nota: Antes disso, apague os prints das funções do Nó Pai, pois eles podem interferir no teste.

Agora, no script do Nó Filho, adicione a seguinte função:

```
# Chamado quando o nó entra na árvore de cena pela primeira vez.  
func _ready() -> void:  
    print("Eu sou o Filho")  
    var pai = get_parent()  
    pai.OlaMundo()  
pass # Replace with function body.
```

A função `get_parent()` retorna uma referência ao Nó Pai, permitindo que o Nó Filho acesse e execute funções do seu pai. Neste caso, ele chama a função `OlaMundo()` do Nó Pai, e a saída será:

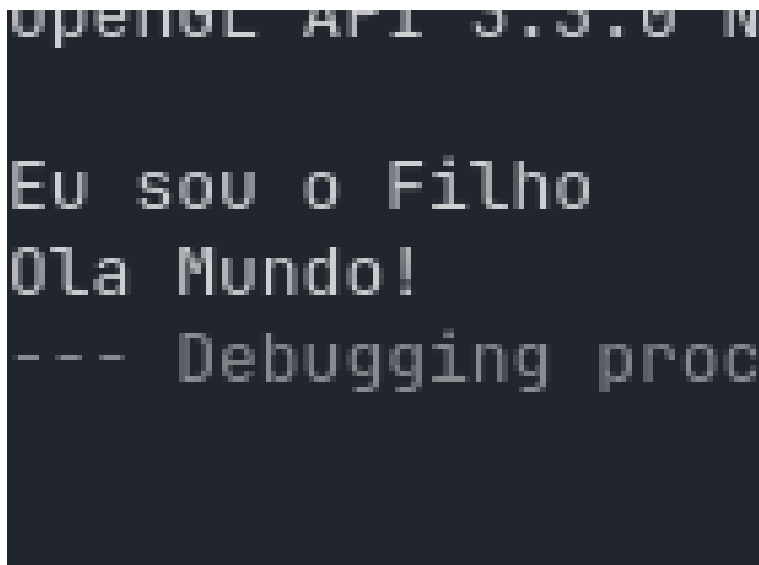


Figura 15: Saída do terminal - Mensagem do Filho

Agora, podemos ver que o Nó Filho foi criado, identificou seu Nó Pai e chamou a função `OlaMundo()` do Pai, demonstrando a comunicação entre os nós.

Com esse conceito, podemos estruturar melhor a lógica do jogo, separando funcionalidades entre diferentes nós e garantindo um código mais organizado e reutilizável. Mais funções e códigos serão abordados durante este curso, mas recomendamos fortemente uma leitura pela documentação oficial da Godot Engine.

[Documentação Oficial Godot](#)



5 Estruturas Condicionais em GDScript

As **estruturas condicionais** permitem que o código funcione de forma diferente com base em diversas situações. Em **GDScript**, utilizamos as instruções **if**, **elif** e **else** para definir essas condições.

5.1 O Básico da Estrutura if

A palavra-chave **if** verifica se uma condição é verdadeira. Se for, o bloco de código dentro dela será executado.

Sintaxe básica:

```
if condição:
    # Código a ser executado se a condição for verdadeira
```

Exemplo prático:

```
var vida = 100

if vida > 0:
    print("O personagem ainda está vivo!")
```

Se a variável **vida** for maior que zero, a mensagem será exibida no console.

5.2 Usando else para Criar Alternativas

O **else** permite definir um bloco de código que será executado caso a condição do **if** seja falsa.

```
var vida = 0

if vida > 0:
    print("O personagem ainda está vivo!")
else:
    print("Game Over!")
```

Como **vida** é igual a 0, a mensagem "Game Over!" será exibida.

5.3 Adicionando elif para Múltiplas Condições

O **elif** (abreviação de *else if*) permite verificar múltiplas condições antes de chegar ao **else**.

```
var energia = 50

if energia > 80:
    print("O jogador está cheio de energia!")
elif energia > 40:
    print("O jogador está com energia moderada.")
else:
    print("O jogador está cansado.")
```



5.4 Operadores de Comparação

Ao usar condicionais, podemos comparar valores com operadores:

Operador	Significado	Exemplo (a = 5, b = 10)
<code>==</code>	Igual a	<code>a == b</code> (falso)
<code>!=</code>	Diferente de	<code>a != b</code> (verdadeiro)
<code>></code>	Maior que	<code>b > a</code> (verdadeiro)
<code><</code>	Menor que	<code>a < b</code> (verdadeiro)
<code>>=</code>	Maior ou igual a	<code>a >= 5</code> (verdadeiro)
<code><=</code>	Menor ou igual a	<code>b <= 5</code> (falso)

Tabela 1: Operadores de comparação em GDScript

5.5 Operadores Lógicos

Podemos combinar múltiplas condições com os operadores lógicos **and**, **or** e **not**.

Operador	Significado	Exemplo (x = 10, y = 20)
and	Ambas condições devem ser verdadeiras	<code>x > 5 and y < 30</code> (verdadeiro)
or	Pelo menos uma condição deve ser verdadeira	<code>x > 15 or y < 30</code> (verdadeiro)
not	Inverte o valor lógico	<code>not (x > y)</code> (verdadeiro)

Tabela 2: Operadores lógicos em GDScript

Exemplo prático:

```
var tem_chave = true
var porta_trancada = false

if tem_chave and not porta_trancada:
    print("Você abriu a porta!")
else:
    print("A porta está trancada.")
```

5.6 Exercício Prático

Tente escrever um código que:

1. Verifique a idade do jogador.
2. Se for menor de 10 anos, exiba "Modo infantil ativado!".
3. Se estiver entre 10 e 17 anos, exiba "Modo juvenil ativado!".
4. Se for 18 ou mais, exiba "Modo adulto ativado!".



```
var idade = 15 # Mude o valor para testar diferentes saídas

if idade < 10:
    print("Modo infantil ativado!")
elif idade < 18:
    print("Modo juvenil ativado!")
else:
    print("Modo adulto ativado!")
```

5.7 Conclusão

As estruturas condicionais são essenciais para criar lógica interativa em jogos. Saber utilizá-las corretamente permite que seu jogo reaja a diferentes eventos, tornando a experiência mais dinâmica e envolvente.



6 Estruturas de Repetição em GDScript

As **estruturas de repetição**, ou *loops*, permitem executar um bloco de código várias vezes, tornando o programa mais eficiente. Em **GDScript**, utilizamos **while** e **for** para criar repetições.

6.1 O Loop while

O loop **while** executa um bloco de código enquanto uma condição for verdadeira.

Sintaxe básica:

```
while condição:  
    # Código a ser repetido
```

Exemplo prático:

```
var contador = 0  
  
while contador < 5:  
    print("Contagem: ", contador)  
    contador += 1
```

Explicação: - O código dentro do **while** será repetido enquanto **contador** for menor que 5. - A cada repetição, **contador** aumenta em 1. - O loop para quando **contador** chega a 5.

6.1.1 Evitando Loops Infinitos

Se a condição do **while** nunca for falsa, o loop continuará para sempre, travando o jogo.

```
var numero = 10  
  
while numero > 0:  
    print("Número:", numero)  
    # Sem decremento, o loop será infinito!
```

Para evitar isso, é necessário garantir que a variável de controle seja atualizada corretamente dentro do loop.

6.2 O Loop for

O loop **for** é usado para iterar sobre sequências, como listas e intervalos numéricos.

6.2.1 Iterando Sobre Intervalos Numéricos

Podemos usar a função **range()** para gerar uma sequência de números.

```
for i in range(5):  
    print(i)
```

Saída esperada:



```
0
1
2
3
4
```

Explicação: - `range(5)` gera os números de 0 a 4. - O loop executa o bloco de código uma vez para cada número.

Também podemos definir um valor inicial e um incremento:

```
for i in range(2, 10, 2):
    print(i)
```

Saída esperada:

```
2
4
6
8
```

Explicação: - `range(2, 10, 2)` gera os números de 2 a 8, incrementando de 2 em 2.

6.2.2 Iterando Sobre Listas

O `for` pode percorrer listas diretamente.

```
var itens = ["Espada", "Escudo", "Poção"]

for item in itens:
    print("Item no inventário: ", item)
```

Saída esperada:

```
Item no inventário: Espada
Item no inventário: Escudo
Item no inventário: Poção
```

6.3 Usando break e continue

6.3.1 Interrompendo um Loop com break

O comando `break` encerra um loop imediatamente.

```
for i in range(10):
    if i == 5:
        print("Interrompendo o loop!")
        break
    print(i)
```

Saída esperada:

```
0
1
2
```



```
3
4
Interrompendo o loop!
```

6.3.2 Pulando iterações com continue

O comando **continue** pula a iteração atual e vai para a próxima.

```
for i in range(5):
    if i == 2:
        continue
    print(i)
```

Saída esperada:

```
0
1
3
4
```

Explicação: - Quando **i** for 2, o **continue** faz com que a linha **print(i)** seja ignorada.

6.4 Loop Aninhado

Podemos colocar um loop dentro de outro.

```
for i in range(3):
    for j in range(2):
        print("i:", i, "j:", j)
```

Saída esperada:

```
i: 0 j: 0
i: 0 j: 1
i: 1 j: 0
i: 1 j: 1
i: 2 j: 0
i: 2 j: 1
```

Isso pode ser útil para manipular matrizes ou grids.

6.5 Exercício Prático

Crie um código que:

1. Utilize um loop **for** para exibir os números de 1 a 10.
2. Se o número for múltiplo de 3, exiba **"Fizz"**.
3. Se for múltiplo de 5, exiba **"Buzz"**.
4. Se for múltiplo de ambos, exiba **"FizzBuzz"**.



```
for i in range(1, 11):  
    if i % 3 == 0 and i % 5 == 0:  
        print("FizzBuzz")  
    elif i % 3 == 0:  
        print("Fizz")  
    elif i % 5 == 0:  
        print("Buzz")  
    else:  
        print(i)
```

Saída esperada:

1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz



7 Preparação do Ambiente de Desenvolvimento do Projeto

Antes de iniciar o desenvolvimento do jogo, é necessário configurar corretamente o ambiente de desenvolvimento. Nesta seção, apresentamos um conjunto de orientações a serem seguidas.

7.1 Estrutura de Pastas

No diretório principal do jogo, serão criadas as seguintes pastas: **assets**, **scripts**, **scenes** e **resources**. A estrutura de diretórios ficará conforme ilustrado na Figura 16.

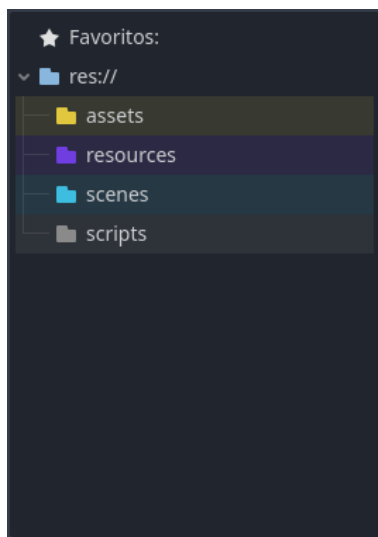


Figura 16: Estrutura de pastas no Godot

Nota: Para adicionar cores às pastas no **Godot**, clique com o botão direito na pasta desejada e selecione *Definir Cor da Pasta...* > [Escolha a cor].

Propósito de cada pasta

- **assets:** Armazena imagens, áudios, modelos 3D, ícones, etc.
- **scripts:** Contém os scripts do jogo, com a extensão **.gd**.
- **scenes:** Armazena as cenas do jogo, que possuem a extensão **.tscn**.
- **resources:** Contém recursos do jogo, geralmente arquivos **.tres** ou **.res**, utilizados internamente pelo **Godot**.



7.2 Assets

Após a criação das pastas, será necessário baixar os assets disponíveis no seguinte repositório: [Download dos Assets](#).

Para isso, faça o download do repositório **Curso-Godot** no formato **.zip**, extraia-o e copie a pasta **assets** para dentro do diretório **assets** do seu projeto.

A imagem a seguir ilustra como baixar o repositório no formato **.zip**:

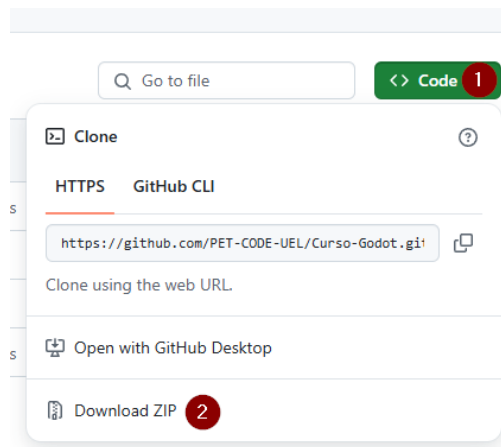


Figura 17: Download do repositório em formato ZIP



7.3 Convenções

Para manter a padronização do código e facilitar a colaboração, este curso seguirá um conjunto de convenções para escrita e nomenclatura. Adicionalmente, apresentamos uma breve explicação sobre os estilos de nomenclatura que serão utilizados.

7.3.1 Convenções de Nomenclatura

Estilos de Nomenclatura:

- **PascalCase:** Em nomes compostos, cada palavra inicia com letra maiúscula e não há separadores. Exemplo: **PlayerController**.
- **snake_case:** As palavras são separadas por sublinhados e todas as letras são minúsculas. Exemplo: **player_controller**.
- **camelCase:** Similar ao **PascalCase**, porém a primeira letra é minúscula. Exemplo: **playerController**.
- **SCREAMING_SNAKE_CASE:** Como o **snake_case**, mas todas as letras são maiúsculas. Exemplo: **PLAYER_CONTROLLER**.

No Projeto:

- **Nodes:** PascalCase
- **Arquivos de Cena:** snake_case
- **Arquivos de Script:** snake_case

Para Scripts:

- **Classes:** PascalCase
- **Variáveis/Parâmetros:** snake_case
- **Funções:** snake_case
- **Constantes:** SCREAMING_SNAKE_CASE
- **Enums:** PascalCase
- **Elementos do Enum:** SCREAMING_SNAKE_CASE



7.3.2 Uso do Padrão ASCII

Na programação, caracteres especiais como **â**, **ã** e **ç** podem causar problemas ao serem usados em nomes de variáveis, funções, etc. Para evitar complicações, utilizaremos exclusivamente caracteres ASCII para a nomenclatura no código. Considere o seguinte exemplo:

```
# Código problemático: contém caracteres não ASCII
var maçã = 1

# Código correto: utiliza apenas caracteres ASCII
var maca = 1
```

Embora o exemplo acima evite problemas técnicos, a palavra **maca** está ortograficamente incorreta, além de que em determinados contextos pode ser confundida com uma cama ou um leito. Uma alternativa mais elegante é utilizar nomes em inglês, que atendem ao padrão ASCII e estão corretos do ponto de vista ortográfico:

```
# Código problemático: contém caracteres especiais não ASCII
var maçã = 1

# Código incorreto: utiliza ASCII, mas resulta em uma palavra ambígua e incorreta
# ortograficamente
var maca = 1

# Código ideal: utiliza ASCII e a palavra está ortograficamente correta
var apple = 1
```

Nota: Usar nomes de variáveis em inglês torna o código mais universal, evita ambiguidades e melhora a compatibilidade com ferramentas e bibliotecas externas.

No entanto, essa limitação se aplica apenas a nomes de variáveis, funções e identificadores do código. Caracteres especiais podem ser usados livremente em strings e comentários, pois nesses casos não há impacto técnico:

```
# Uso de caracteres especiais em strings e comentários
var message = "Você encontrou uma maçã!"
print("Ação realizada com êxito.")

# Comentário explicativo
# Esta função calcula a área de um círculo usando pi.
func calculate_area(radius):
    return PI * radius * radius
```



8 Criação do Terreno e do Personagem

8.1 Cena Principal

Inicialmente, criaremos uma cena chamada **main_scene.tscn**, que será a cena principal do jogo. Ela conterá toda a parte *in-game* e será carregada quando o jogador clicar em **Novo Jogo**.

O nó pai dessa cena será um nó genérico, pois conterá tanto elementos de interface quanto objetos 3D. A estrutura inicial da cena será composta pelos seguintes nós:

- **Main (Node)** - Nó pai da cena.
- **Game (Node3D)** - Responsável pelos elementos do jogo.

A estrutura de nós será semelhante à mostrada na Figura 18.

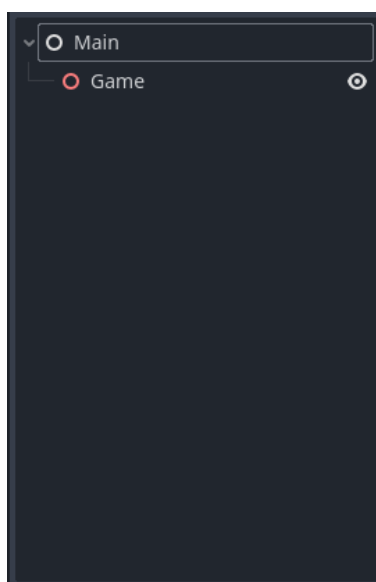


Figura 18: Estrutura inicial de **main_scene.tscn**



8.2 Criação do Terreno

Para modularizar o projeto e facilitar o desenvolvimento em equipe, cada parte do jogo será dividida em cenas separadas. Assim, criaremos uma nova cena chamada **terrain.tscn**, que será responsável pela lógica e pelos elementos do terreno.

O nó pai dessa cena será um nó especial da Godot Voxel (criado por Zylann), permitindo a geração de terrenos procedurais. Portanto, criaremos um nó **VoxelLodTerrain** e o nomearemos como **Terrain**.

8.2.1 Criando o Terreno

No menu de propriedades do nó **VoxelLodTerrain**, siga os passos abaixo:

1. Vá até **VoxelNode** > **Generator**.
2. Clique no campo vazio ao lado de **Generator**.
3. Escolha a opção **Novo VoxelGeneratorFlat**.

O resultado será semelhante ao mostrado na Figura 19.

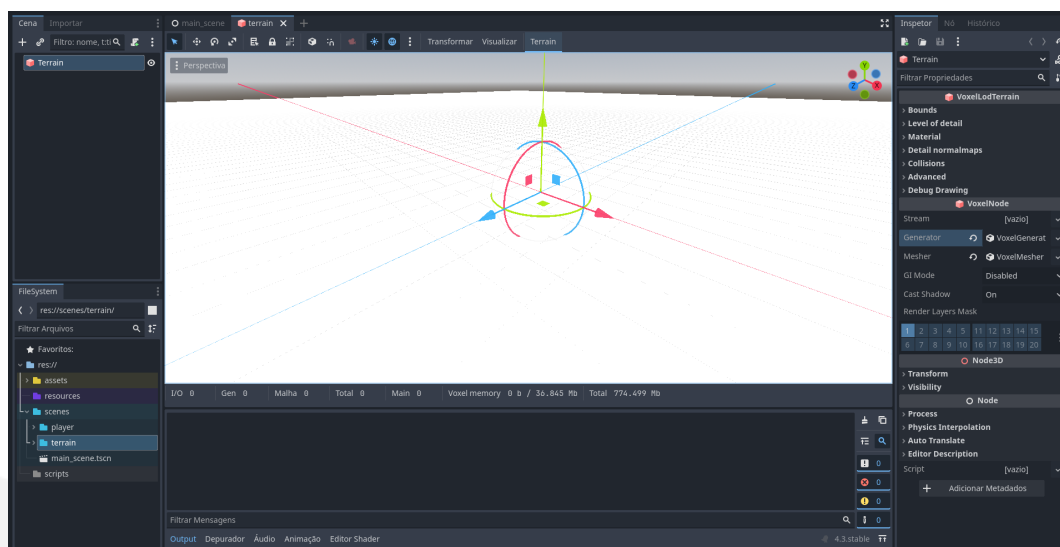


Figura 19: Configurando o **VoxelGeneratorFlat** no **VoxelLodTerrain**

Observação

Note que o terreno gerado inicialmente está branco. Para corrigir isso, adicionaremos uma **textura** ao terreno.



8.2.2 Adicionando Textura ao Terreno

Para adicionar uma textura ao terreno, siga os passos abaixo:

1. Vá até **VoxelLodTerrain** > **Materials** > **Material**.
 2. Clique no campo vazio e selecione **Novo StandardMaterial3D**.
- O resultado será semelhante ao mostrado na Figura 20.

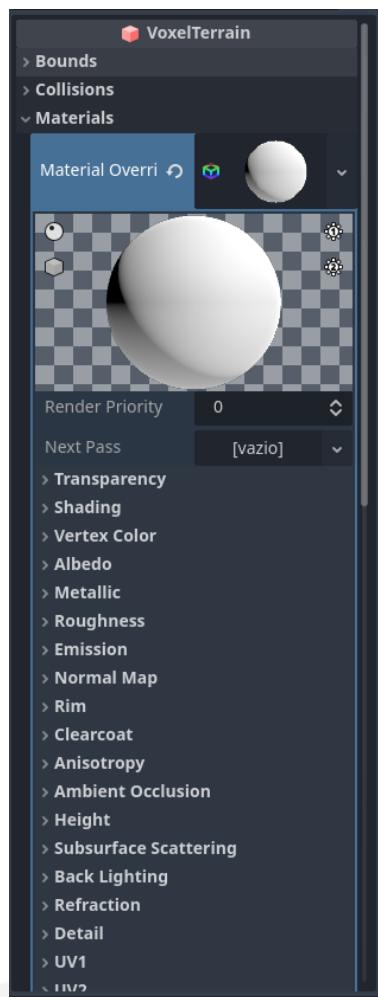


Figura 20: Criando um **StandardMaterial3D** no **VoxelLodTerrain**



Agora adicionaremos a textura de grama ao material:

1. Vá até **Albedo > Albedo Texture**.
 2. Clique no campo vazio e escolha **Carregar...**
 3. Selecione o arquivo **grama.png** em **assets/textures**.
- O resultado será semelhante ao mostrado na Figura 21.

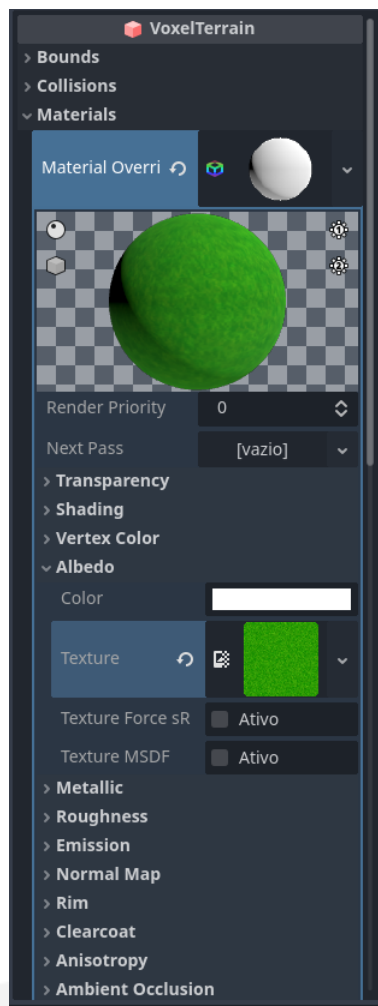


Figura 21: Adicionando a textura de grama ao terreno



Porém, nesse estágio, o terreno ainda pode parecer apenas pintado de verde, sem detalhes realistas. Para melhorar a aplicação da textura, siga os passos abaixo:

1. No menu **Albedo**, habilite a opção **Texture Force sRGB**.
2. Em **UV1**, ative a opção **Triplanar**.

A opção **Triplanar** aplica a textura de três direções diferentes (cima, frente e lados), evitando que ela fique esticada ou deformada em superfícies inclinadas, criando uma transição mais natural entre as áreas do terreno. Agora nosso terreno tem uma aparência mais realista! Entretanto, ainda não podemos testá-lo porque não temos um personagem no jogo. Portanto, vamos criá-lo a seguir.



8.3 Criação do Personagem

8.3.1 Importando o Modelo do Personagem

Para criar o personagem, vamos instanciar o modelo 3D e configurar sua animação. Siga os passos abaixo:

1. Crie uma nova cena vazia.
2. Instancie o modelo do personagem localizado em **assets/models/player**.
3. Selecione **player.glb**.

O resultado será semelhante ao mostrado na Figura 22.

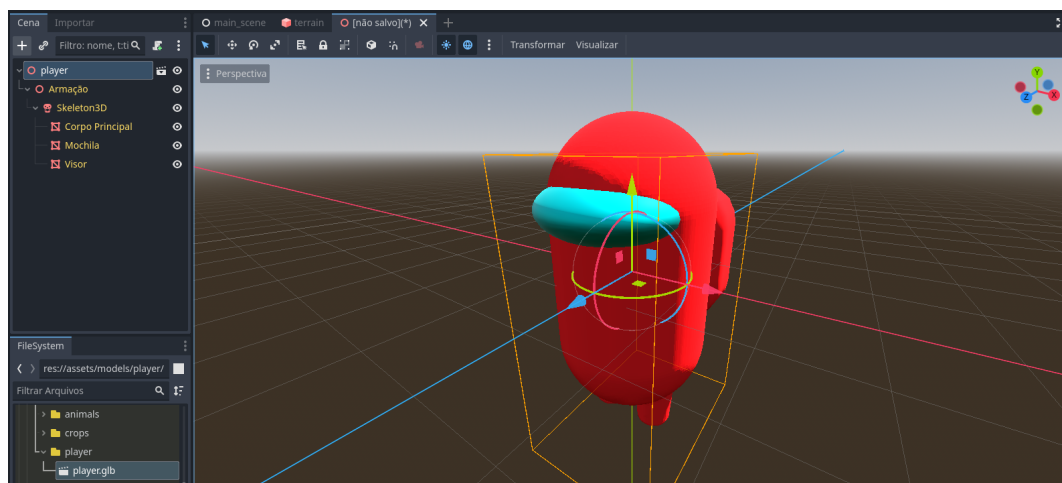


Figura 22: Importando o modelo 3D do personagem

Renomeie o nó pai de **player** para **PlayerModel** e salve a cena como **player_model.tscn**.



8.3.2 Criando a Lógica do Personagem

Agora criaremos a cena que conterá a lógica do personagem. Siga os passos abaixo:

1. Crie uma nova cena e nomeie como **player.tscn**.
2. Defina o nó pai como **CharacterBody3D** e nomeie-o **Player**.
3. Adicione um nó filho **CollisionShape3D** e nomeie-o **CollisionShape**.
4. Como filho de **CollisionShape**, instancie a cena **player_model.tscn**.

A estrutura de nós resultante será semelhante à mostrada na Figura 23.

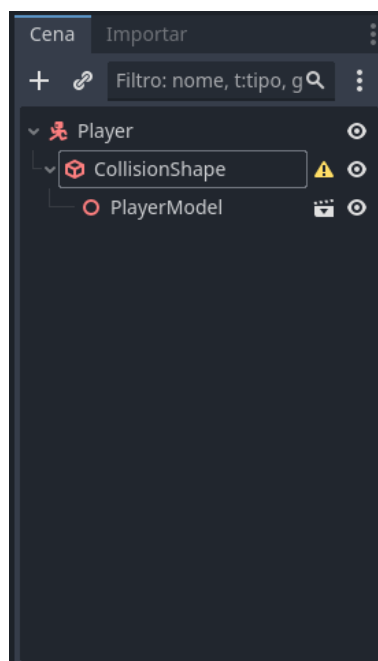


Figura 23: Estrutura de nós de **player.tscn**

Note que o Godot exibe um aviso (**Warning**) no **CollisionShape**, pois ainda não definimos uma forma de colisão.



8.3.3 Adicionando Colisão ao Personagem

Para adicionar uma forma de colisão ao personagem, selecione o nó **CollisionShape** (filho de **Player**) e, no menu de propriedades, siga os passos:

1. Vá até **Collision Shape > Shape**.
2. Clique no campo vazio e selecione **Novo BoxShape3D**.

O resultado será semelhante ao mostrado na Figura 24.

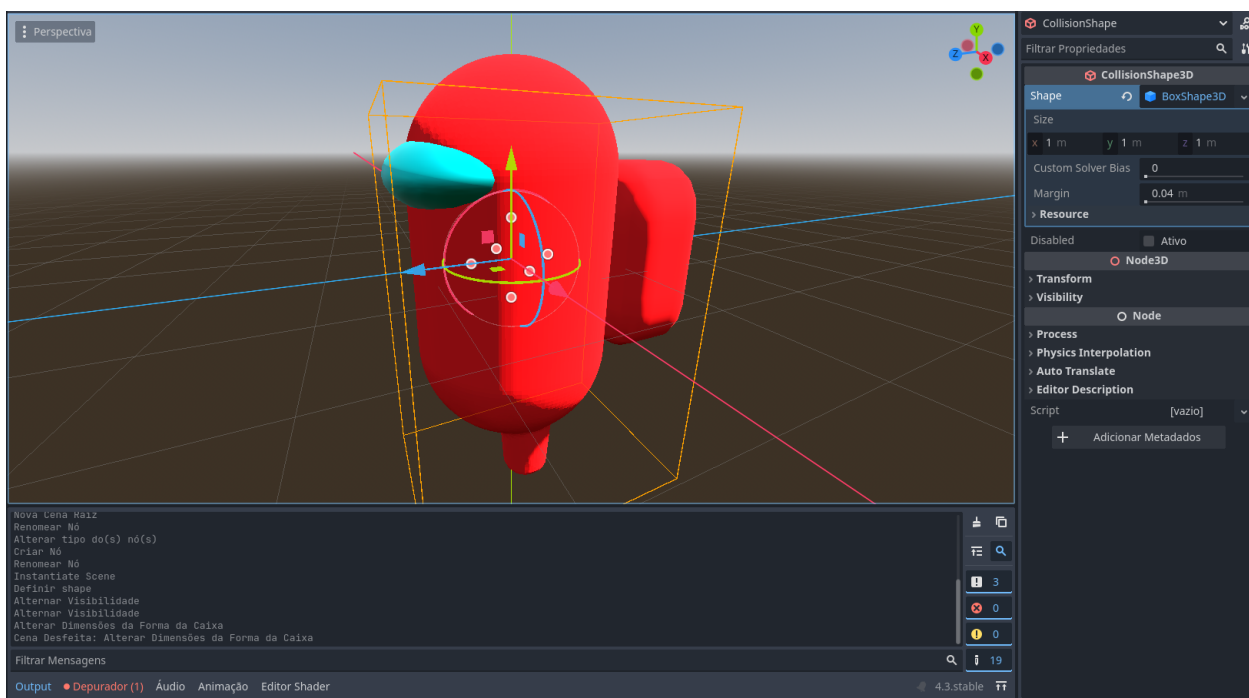


Figura 24: Adicionando colisão ao personagem



Agora ajuste o tamanho da colisão (representado pelo cubo com linhas azuis) para que ele encaixe corretamente no modelo do personagem. Para facilitar esse processo, você pode seguir os passos abaixo:

1. Mova temporariamente **PlayerModel** para ser irmão de **CollisionShape**.
2. Ajuste o tamanho da colisão para cobrir o personagem corretamente.
3. Retorne **PlayerModel** como filho de **CollisionShape**.

Após esses ajustes, a colisão deve se parecer com a Figura 25.

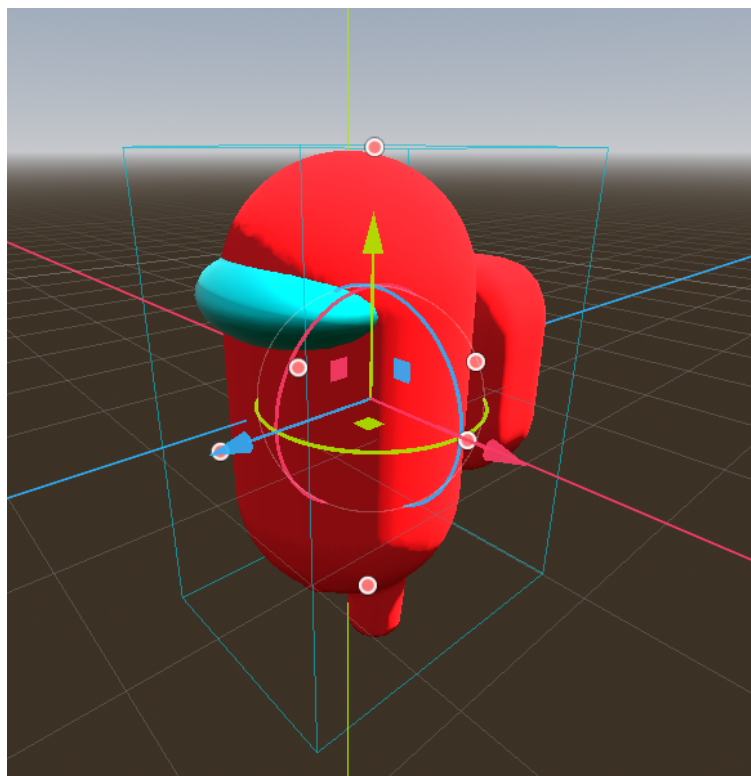


Figura 25: Colisão ajustada no personagem



8.3.4 Adicionando uma Câmera ao Personagem

Agora que o personagem já tem forma e colisão, queremos que o jogador possa enxergar o mundo a partir da perspectiva dele. Para isso, adicionaremos uma câmera.

Criando a Estrutura da Câmera

1. Adicione um nó **Node3D** como filho de **Player** e nomeie-o **CameraPivot**. Esse nó facilitará a rotação da câmera e, futuramente, permitirá alternar entre visão em primeira e terceira pessoa.
2. Como filho de **CameraPivot**, adicione um nó **Camera3D** e renomeie-o para **FirstPerson**.

A estrutura da cena deve se parecer com a Figura 26.

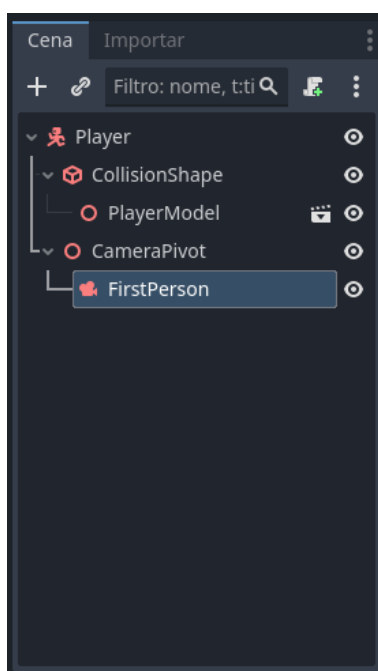


Figura 26: Estrutura de nós de `player.tscn`



Ajustando a Orientação da Câmera

Por padrão, a câmera estará virada na direção oposta ao personagem (a base da pirâmide da câmera indica sua direção). Para corrigir isso, será necessário girar o personagem 180° no eixo Y.

Aviso

Não gire o PlayerModel! Isso desconfiguraria a colisão. Em vez disso, gire o nó **CollisionShape**, pois ele já afeta automaticamente o modelo do personagem.

Para girar o **CollisionShape**, siga os passos:

1. Selecione o nó **CollisionShape**.
2. No painel de propriedades, vá até **Node3D > Transform > Rotation**.
3. Defina a rotação no eixo Y como **180°**.

Posicionando a Câmera

Agora que a direção da câmera está correta, devemos posicioná-la corretamente para que fique na altura dos olhos do jogador. Para isso, basta mover o nó **CameraPivot** até a posição desejada.

Após esse ajuste, a câmera deve se parecer com a Figura 27.

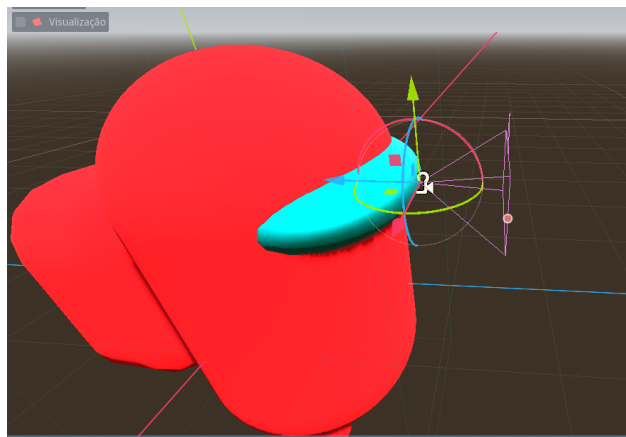


Figura 27: Posicionamento da câmera

Ativando a Câmera

Por padrão, a Godot não usa automaticamente a nova câmera adicionada. Para garantir que a visão do jogador será exibida corretamente, siga os passos:

1. Selecione o nó **FirstPerson**.
2. No painel de propriedades, vá até **Camera3D > Current**.
3. Marque essa opção como **ativa**.

Com isso, ao iniciar a cena, a Godot usará essa câmera como a principal.



8.4 Adicionando o Personagem e o Terreno à Cena Principal

Agora que concluímos a criação do terreno e do personagem, vamos adicioná-los à cena principal.

8.4.1 Instanciando as Cenas

No arquivo `main_scene.tscn`, siga os seguintes passos:

1. Instancie a cena `terrain.tscn` como filha do nó `Game`.
2. Instancie a cena `player.tscn` também como filha de `Game`.
3. Posicione o jogador logo acima do terreno para evitar que ele caia imediatamente ao iniciar o jogo.

Após esses passos, a estrutura da cena principal deve se parecer com a Figura 28.

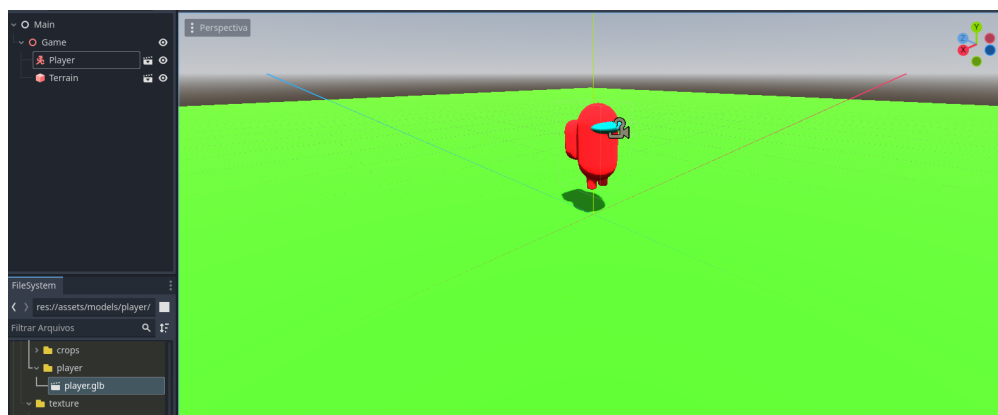


Figura 28: Estrutura de `main_scene.tscn`



8.4.2 Adicionando Iluminação Temporária

Se rodarmos a cena principal neste momento, veremos que está tudo escuro. Isso ocorre porque os objetos do jogo dependem da luz refletida sobre eles, e ainda não adicionamos nenhuma fonte de iluminação à cena.

Mais adiante, implementaremos um sistema de iluminação e ambientação mais elaborado. No entanto, para fins de teste, podemos adicionar uma luz básica temporária:

1. No menu superior do Godot, clique no ícone de três pontos localizado ao lado de um **sol** e um **globo**.
2. Selecione *Adicionar Sol à Cena*.
3. Selecione *Adicionar Ambiente à Cena*.

Após esse procedimento, a cena passará a ser exibida no jogo da mesma forma como é vista no editor. Veja a Figura 29 para referência.

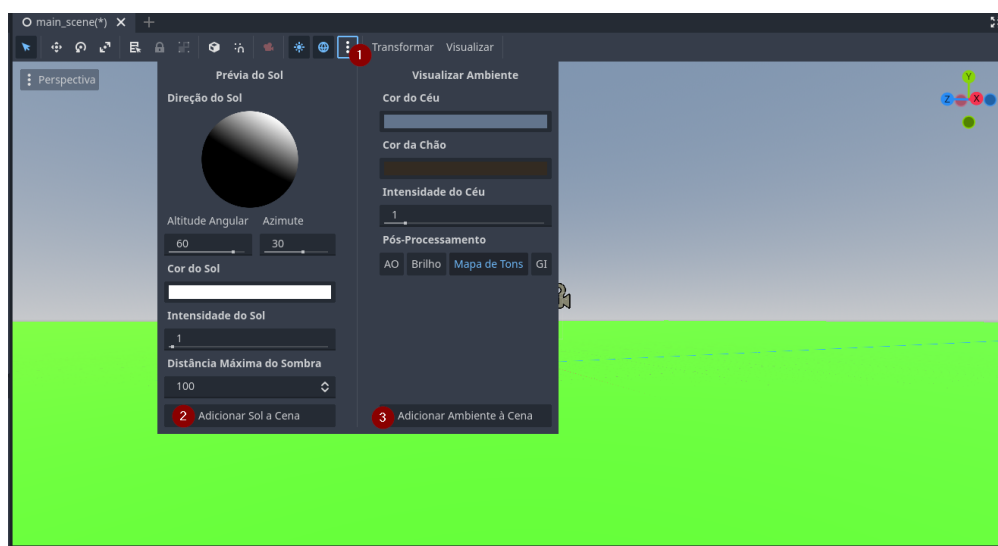


Figura 29: Adicionando Luz e Ambiente na Cena Principal (Para Testes)

Ao iniciar a cena principal, veremos que o jogo está rodando, mas ainda não há controle sobre o personagem.



8.5 Controle do Personagem

Voltando à cena do personagem em `player.tscn`, vamos adicionar um script ao nó pai **Player**. Utilizaremos o modelo de script padrão do Godot para **CharacterBody3D**, que contém o seguinte código:

```
extends CharacterBody3D

const SPEED = 5.0
const JUMP_VELOCITY = 4.5

func _physics_process(delta: float) -> void:
    # Add the gravity.
    if not is_on_floor():
        velocity += get_gravity() * delta

    # Handle jump.
    if Input.is_action_just_pressed("ui_accept") and is_on_floor():
        velocity.y = JUMP_VELOCITY

    # Get the input direction and handle the movement/deceleration.
    # As good practice, you should replace UI actions with custom gameplay actions.
    var input_dir := Input.get_vector("ui_left", "ui_right", "ui_up", "ui_down")
    var direction := (transform.basis * Vector3(input_dir.x, 0, input_dir.y)).normalized()
    if direction:
        velocity.x = direction.x * SPEED
        velocity.z = direction.z * SPEED
    else:
        velocity.x = move_toward(velocity.x, 0, SPEED)
        velocity.z = move_toward(velocity.z, 0, SPEED)

    move_and_slide()
```

Explicação do Código

- **SPEED** representa a velocidade do personagem.
- **JUMP_VELOCITY** define a força do pulo.
- O método **_physics_process(delta)** é um método especial do Godot chamado a cada frame de física.

Diferente do método **_process()**, que é chamado em cada frame com uma taxa variável, **_physics_process()** é chamado em uma taxa fixa (geralmente 60 vezes por segundo), garantindo consistência na física do jogo.



Detalhamento da Lógica do Código

1. Primeiro, verifica se o personagem **não está no chão**. Caso não esteja, aplica uma força vertical para simular a gravidade.
2. Em seguida, verifica se o jogador pressionou a tecla correspondente ao **"ui_accept"** e se o personagem está no chão. Se ambas as condições forem verdadeiras, um impulso é aplicado para realizar o pulo.
3. Depois, o código captura a entrada do usuário nas direções **"ui_left"**, **"ui_right"**, **"ui_up"** e **"ui_down"**.
 - Se houver entrada, a velocidade do personagem é ajustada conforme a direção pressionada.
 - Se nenhuma tecla for pressionada, a velocidade do personagem diminui progressivamente até zerar.
4. Por fim, o método **move_and_slide()** é chamado para movimentar o personagem de acordo com os cálculos anteriores.

Nota: O uso do método **move_and_slide()** simplifica o gerenciamento de colisões e deslizamento do personagem em superfícies inclinadas, tornando a movimentação mais natural.

8.5.1 Alterando os Controles

Ao rodarmos a cena principal, veremos que o personagem se movimenta utilizando as **setas direcionais** e pula com a **barra de espaço**. Se quisermos adicionar suporte às teclas **WASD**, precisaremos modificar os controles.

Para isso, siga os passos abaixo:

1. No menu superior do Godot, vá em **Projeto > Project Settings > Mapa de Entrada**.
2. Habilite a opção **Mostrar Ações Integradas**.
3. Adicione as teclas extras para os seguintes comandos:
 - **ui_left** → A
 - **ui_right** → D
 - **ui_up** → W
 - **ui_down** → S

Agora nosso personagem pode se mover tanto pelas setas direcionais quanto pelo esquema **WASD**.



8.5.2 Adicionando Controle da Câmera com o Mouse

Atualmente, o personagem se move apenas pelo teclado, e a câmera permanece fixa. Nosso objetivo é permitir que o jogador controle a rotação da câmera com o mouse.

Para isso, precisamos modificar o script do personagem. Primeiramente, adicione o seguinte trecho no início do código:

```
@export var mouse_sensitivity := Vector2(0.01, 0.01)
var rotation_horizontal := 0.0
var rotation_vertical := 0.0
```

Esse código define:

- **mouse_sensitivity**: Sensibilidade do mouse nos eixos X e Y.
- **rotation_horizontal**: Armazena a rotação horizontal do personagem.
- **rotation_vertical**: Armazena a rotação vertical da câmera.

Agora, criaremos uma função especial chamada **_input(event)**, que é chamada sempre que o jogador interage com o teclado ou mouse. Essa função detectará o movimento do mouse e atualizará as variáveis de rotação. Além disso, limitaremos a rotação vertical para evitar que a câmera gire 360°.

```
func _input(event):
    if event is InputEventMouseMotion and Input.get_mouse_mode() == Input.MOUSE_MODE_CAPTURED:
        rotation_horizontal -= event.relative.x * mouse_sensitivity.x
        rotation_vertical -= event.relative.y * mouse_sensitivity.y
        rotation_vertical = clamp(rotation_vertical, deg_to_rad(-90), deg_to_rad(90))
```

Agora precisamos acessar o pivô da câmera, então adicionamos o seguinte trecho no início do código:

```
@onready var camera_pivot: Node3D = $CameraPivot
```

Por fim, adicionamos a função **_process(delta)**, que atualizará a rotação da câmera:

```
func _process(_delta):
    rotation.y = rotation_horizontal
    camera_pivot.rotation.x = rotation_vertical
```

Nota: O uso do **rotation.y** faz com que rode o personagem em si não só a câmera, enquanto que a rotação do **camera_pivot** faz com que rode apenas a "cabeça" do personagem.

Agora falta apenas capturar o mouse para que ele controle a câmera corretamente. Para facilitar futuras implementações, centralizaremos todo o controle da UI em um script único.



8.5.3 Criando o UI Manager

Vamos criar um script chamado **UIManager.gd**, que será um **Singleton** — ou seja, poderá ser acessado de qualquer lugar do jogo.

Configurando o Singleton:

1. Crie o script **UIManager.gd**, fazendo com que ele herde de **Node**.
2. No Godot, vá em **Projeto > Project Settings > Globals > Autoload**.
3. Clique no ícone de pasta e selecione o script criado.
4. Nomeie o nó como **UIManager** e clique em **Adicionar**.
5. Certifique-se de que ele está habilitado. A configuração deve ficar semelhante à Figura 30.

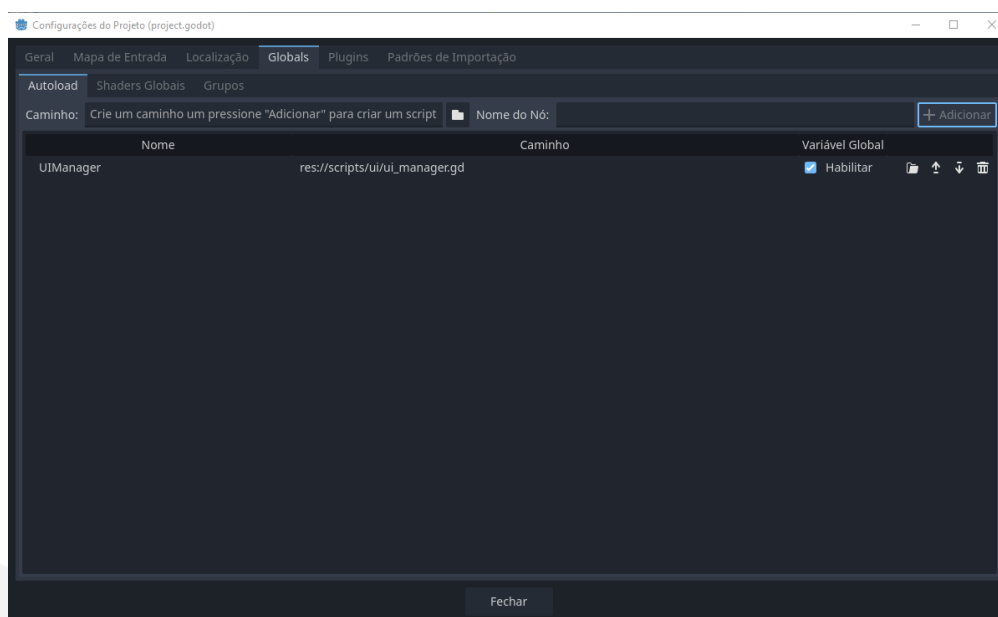


Figura 30: Adicionando um script Singleton

Agora, no script **UIManager.gd**, adicione o seguinte código:

```
var ui: UI

func initialize(ui_node: CanvasLayer):
    ui = ui_node
    Input.set_mouse_mode(Input.MOUSE_MODE_CAPTURED)
```



Agora, vá até `main_scene.tscn` e adicione um novo nó do tipo **CanvasLayer** como filho de **Main**. Nomeie esse nó como **UI** e adicione um script a ele com o seguinte conteúdo:

```
class_name UI
extends CanvasLayer

func _ready() -> void:
    UIManager.initialize(self)
```

Nota: O uso de `UIManager.initialize(self)` em `_ready()` garante que o script **UIManager** só será executado após o nó **UI** ser carregado, evitando erros de referência.

A palavra-chave **self** é utilizada para passar uma referência do nó **UI** para o **UIManager**. Isso será útil posteriormente para gerenciar mudanças na interface e liberar o cursor do mouse quando necessário.

Agora, ao rodar a cena principal, veremos que conseguimos controlar a câmera com o mouse. No entanto, se olharmos para cima ou para baixo, perceberemos que o personagem acaba "vendo por dentro" de si mesmo. Esse problema será resolvido na próxima seção.

8.5.4 Ocultando as *Meshes* em Primeira Pessoa

Ao utilizar a visão em primeira pessoa, o jogador pode acabar visualizando o interior do próprio modelo do personagem, o que não é desejado. Para resolver esse problema, ocultaremos as *meshes* do personagem quando ele estiver nesse modo de visão. No entanto, manteremos a projeção da sombra, para que ainda pareça existir um corpo físico no jogo.

Criar uma Referência ao **Skeleton3D**

Para ocultar as *meshes*, primeiro precisamos acessar a estrutura do esqueleto do personagem. Para isso, edite o script `player.gd` e adicione a seguinte variável no topo do arquivo:

```
@onready var skeleton: Skeleton3D = $"CollisionShape/PlayerModel/Armação/Skeleton3D"
```

Essa variável armazenará uma referência ao **Skeleton3D**, permitindo que iteremos sobre suas *meshes* e alteremos suas propriedades.



Criar a Função de Alternância de Visibilidade

Agora, adicionaremos uma função para ativar ou desativar a visibilidade das *meshes* do personagem, dependendo se a câmera está no modo primeira pessoa:

```
func toggle_visibility(is_first_person: bool):  
    for child in skeleton.get_children():  
        if child is MeshInstance3D:  
            if is_first_person:  
                child.cast_shadow = GeometryInstance3D.SHADOW_CASTING_SETTING_SHADOWS_ONLY  
            else:  
                child.cast_shadow = GeometryInstance3D.SHADOW_CASTING_SETTING_ON
```

Essa função recebe um valor booleano (**is_first_person**) indicando se o jogo está no modo de câmera em primeira pessoa. Ela então percorre todos os filhos do **skeleton**, verificando se cada um deles é uma instância de **MeshInstance3D**. Caso seja, a função modifica a propriedade **cast_shadow**, que controla como a *mesh* projeta sombras:

- Se estiver em primeira pessoa (**is_first_person == true**), a *mesh* será oculta, mas a sombra ainda será renderizada (**SHADOW_CASTING_SETTING_SHADOWS_ONLY**).
- Se **não** estiver em primeira pessoa (**is_first_person == false**), a sombra será projetada normalmente (**SHADOW_CASTING_SETTING_ON**).

Criar a Função de Atualização da Câmera

Agora, criaremos uma nova função chamada **update_camera_mode()**, que, por enquanto, apenas chamará **toggle_visibility()**. Embora pareça redundante neste momento, ela será útil posteriormente para atualizar a visibilidade do personagem ao alternar entre modos de câmera.

```
func update_camera_mode():  
    toggle_visibility(true)
```

Chamar a Função no **_ready()**

Por fim, precisamos garantir que a função seja executada assim que o personagem for carregado na cena. Para isso, chamaremos **update_camera_mode()** dentro do método **_ready()** do personagem:

```
func _ready():  
    update_camera_mode()
```

Resultado Final

Ao executar a cena principal, o problema será resolvido: O corpo do personagem não será visível em primeira pessoa, mas sua sombra ainda será projetada corretamente no ambiente.

Nota: A função **update_camera_mode()** será expandida posteriormente para permitir a alternância entre primeira, segunda e terceira pessoa.



9 Adicionando Mais Câmeras ao Personagem

9.1 Adicionando Câmeras de Segunda e Terceira Pessoa

Agora que o personagem foi integrado ao jogo, vamos adicionar mais câmeras para aprimorar a experiência do jogador, incluindo câmeras de segunda e terceira pessoa. Para isso, abra o arquivo **player.tscn** e adicione duas novas câmeras como filhas de **CameraPivot**: uma para a segunda pessoa e outra para a terceira, nomeando-as como **SecondPerson** e **ThirdPerson**, respectivamente. A estrutura dos nós ficará da seguinte forma:

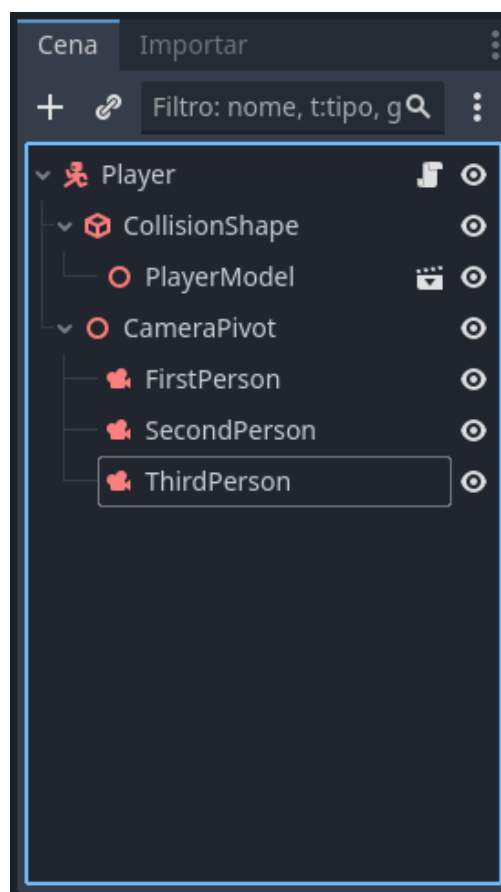


Figura 31: Adicionando Câmeras de Segunda e Terceira Pessoa ao Personagem



Agora, posicione as câmeras conforme o descrito abaixo:

- A câmera de **segunda pessoa** deve ter uma visão frontal do personagem, como se ele estivesse se vendo no espelho.
- A câmera de **terceira pessoa** deve ter uma visão das costas do personagem.

Você pode testar as visões das câmeras selecionando a câmera desejada e clicando em **Visualização** no canto superior esquerdo da tela ou pressionando **Ctrl+P**.

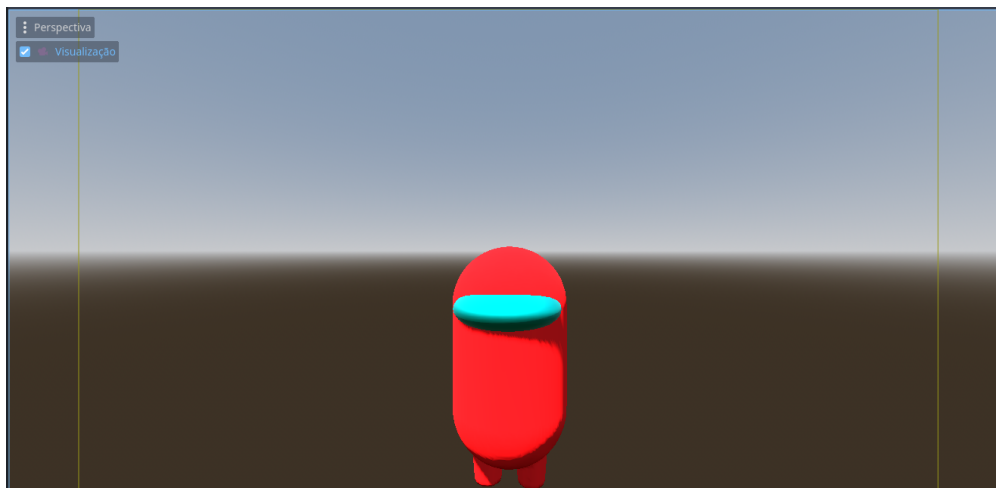


Figura 32: Exemplo de Visão da Câmera de Segunda Pessoa no Editor

A disposição das câmeras no editor ficará semelhante à imagem abaixo:

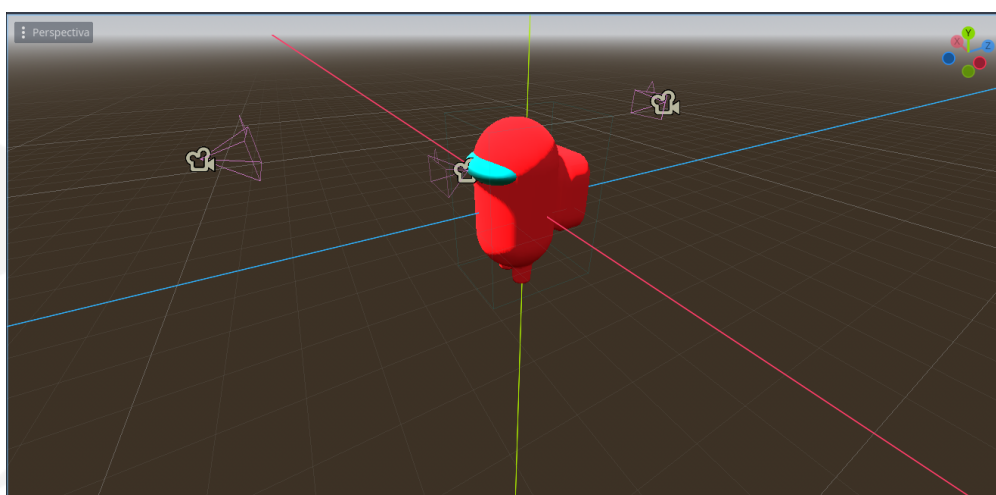


Figura 33: Disposição das Câmeras



9.2 Implementando a Lógica das Câmeras

Agora, vamos atualizar o script para gerenciar as novas câmeras e permitir a troca entre elas usando a entrada do teclado. Primeiro, criaremos um *enum* para representar cada modo de câmera e uma variável para armazenar qual câmera está ativa no momento. Inicializaremos com a câmera de primeira pessoa:

```
enum CameraModes {  
    FIRST_PERSON,  
    SECOND_PERSON,  
    THIRD_PERSON,  
}  
  
var current_camera_mode: CameraModes = CameraModes.FIRST_PERSON
```

Em seguida, adicionaremos algumas variáveis **@onready** para referenciar as câmeras do personagem. O código com todas as variáveis até agora será:

```
# Movimentação  
const SPEED = 5.0  
const JUMP_VELOCITY = 4.5  
  
# Movimentação da Câmera  
@export var mouse_sensitivity := Vector2(0.01, 0.01)  
var rotation_horizontal := 0.0  
var rotation_vertical := 0.0  
  
# Câmeras  
enum CameraModes {  
    FIRST_PERSON,  
    SECOND_PERSON,  
    THIRD_PERSON,  
}  
  
var current_camera_mode: CameraModes = CameraModes.FIRST_PERSON  
  
@onready var camera_pivot: Node3D = $CameraPivot  
@onready var first_person: Camera3D = $CameraPivot/FirstPerson  
@onready var second_person: Camera3D = $CameraPivot/SecondPerson  
@onready var third_person: Camera3D = $CameraPivot/ThirdPerson  
  
@onready var skeleton: Skeleton3D = $"CollisionShape/PlayerModel/Armação/Skeleton3D"
```

Agora, vamos criar uma função para alternar entre as câmeras. Quando chamada, essa função atualizará a variável **current_camera_mode** para o próximo modo de câmera. Quando atingir a última câmera, ela retornará para a primeira. Em seguida, chamaremos a função **update_camera_mode()** para fazer a troca de câmera.

```
func cycle_camera_mode():  
    var camera_modes := CameraModes.values()  
    current_camera_mode = camera_modes[(int(current_camera_mode) + 1) % camera_modes.size()]  
    update_camera_mode()
```



Agora, precisamos atualizar a função `update_camera_mode` para realizar a troca de câmeras. A propriedade `current` de cada `Camera3D` define se ela será a câmera ativa que o jogador verá. Portanto, vamos atualizar essa propriedade conforme o valor de `current_camera_mode`:

```
func update_camera_mode():
    first_person.current = (current_camera_mode == CameraModes.FIRST_PERSON)
    second_person.current = (current_camera_mode == CameraModes.SECOND_PERSON)
    third_person.current = (current_camera_mode == CameraModes.THIRD_PERSON)
    toggle_visibility(current_camera_mode == CameraModes.FIRST_PERSON)
```

Agora, para permitir que o jogador troque de câmera usando uma tecla, vá em **Project > Project Settings > Mapa de Entrada**, adicione uma nova ação chamada `"camera_switch"` e atribua uma tecla a essa ação (como, por exemplo, **F5**). Em seguida, atualize o método `_input()` para capturar a tecla pressionada e alternar entre as câmeras:

```
func _input(event):
    if event is InputEventMouseMotion and Input.get_mouse_mode() == Input.MOUSE_MODE_CAPTURED:
        :
        rotation_horizontal -= event.relative.x * mouse_sensitivity.x
        rotation_vertical -= event.relative.y * mouse_sensitivity.y
        rotation_vertical = clamp(rotation_vertical, deg_to_rad(-90), deg_to_rad(90))

    if event.is_action_pressed("camera_switch"):
        cycle_camera_mode()
```

Observação

Diferentemente da implementação da câmera de primeira pessoa, não é necessário implementar a rotação para as câmeras de segunda e terceira pessoa, pois o controle de rotação é aplicado ao **pivô** (`CameraPivot`). Essa abordagem simplifica a lógica e é uma das vantagens de utilizar um pivô para o controle das câmeras.

Ao executar a cena principal, você perceberá que o jogador pode alternar entre as diferentes câmeras pressionando a tecla configurada. Entretanto, note que, em terceira pessoa, o personagem pode parecer "flutuar" durante a movimentação. Esse efeito ocorre porque a animação de caminhada ainda não foi implementada, tema que será abordado no próximo capítulo.



10 Criando a Animação do Personagem

10.1 Adicionando os Nós de Animação

Para criar a animação do nosso personagem, edite a cena `player_model.tscn`. Adicione um nó **AnimationPlayer** como filho de **Armação** e insira também um nó do tipo **AnimationTree** como filho de **Armação**. A estrutura da cena deverá se assemelhar à apresentada na Figura 34.

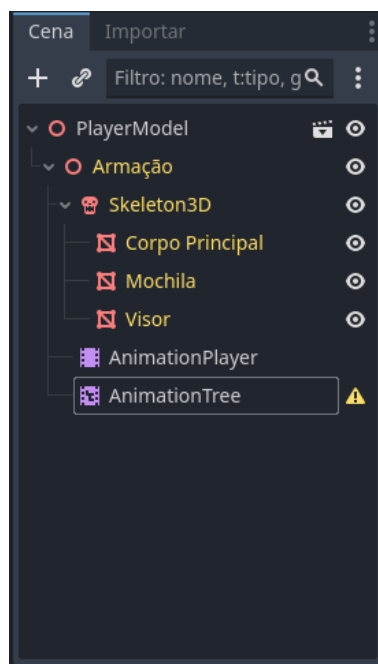


Figura 34: Estrutura de `player_model.tscn`



10.2 Criando a Animação (AnimationPlayer)

Para configurar a animação do personagem, siga os passos abaixo:

1. Selecione o nó **AnimationPlayer** e, em seguida, vá em **Animação > Nova...** Dê o nome *Idle* para a animação padrão (quando o personagem estiver parado).
2. Crie uma nova animação chamada **Walk**.

Com a animação **Walk** selecionada, mantenha o painel de animação aberto e, em seguida, selecione o nó **Skeleton3D**. Você deverá visualizar algo semelhante à imagem na Figura 35.

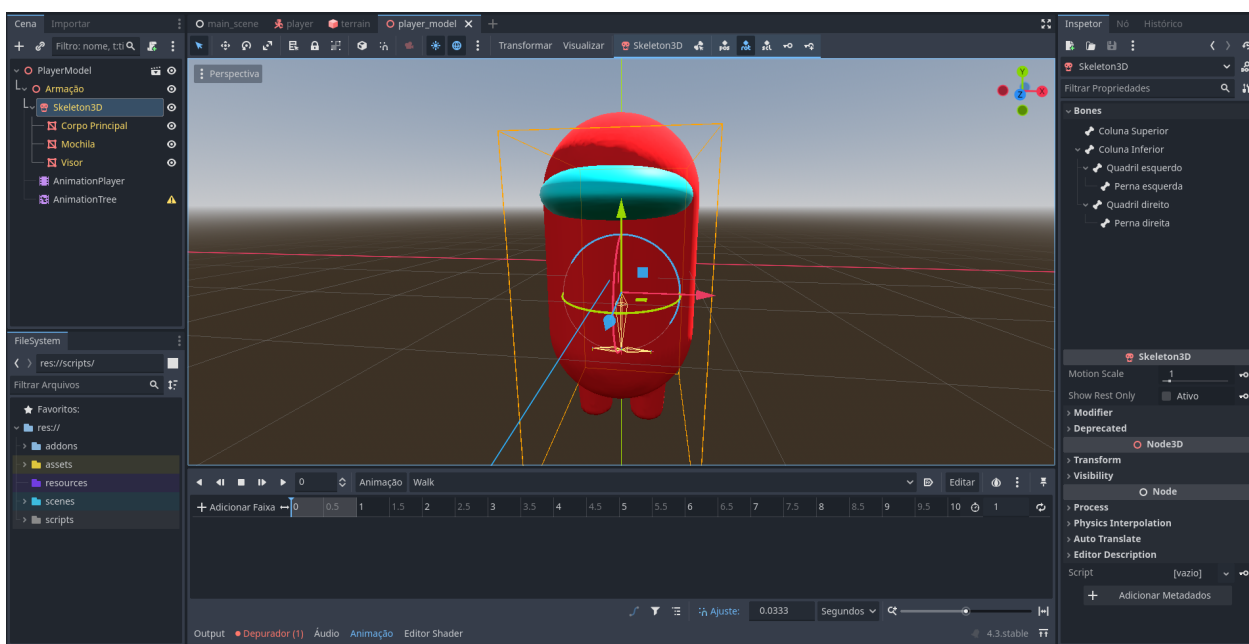


Figura 35: Estrutura do **Skeleton3D**

Observação: No painel lateral de propriedades, na seção **Bones** temos os ossos, e no menu do topo encima temos o **Skeleton3D** com vários ícones, dentre os quais há um ícone de chave. Vamos utilizar desses recursos para criar as faixas de animação.



Para começar, selecione o ícone de chave no menu superior de um osso e escolha um **bone** no painel de propriedades ou diretamente no personagem. Por exemplo, selecione a **Perna Direita** conforme ilustrado na Figura 36.

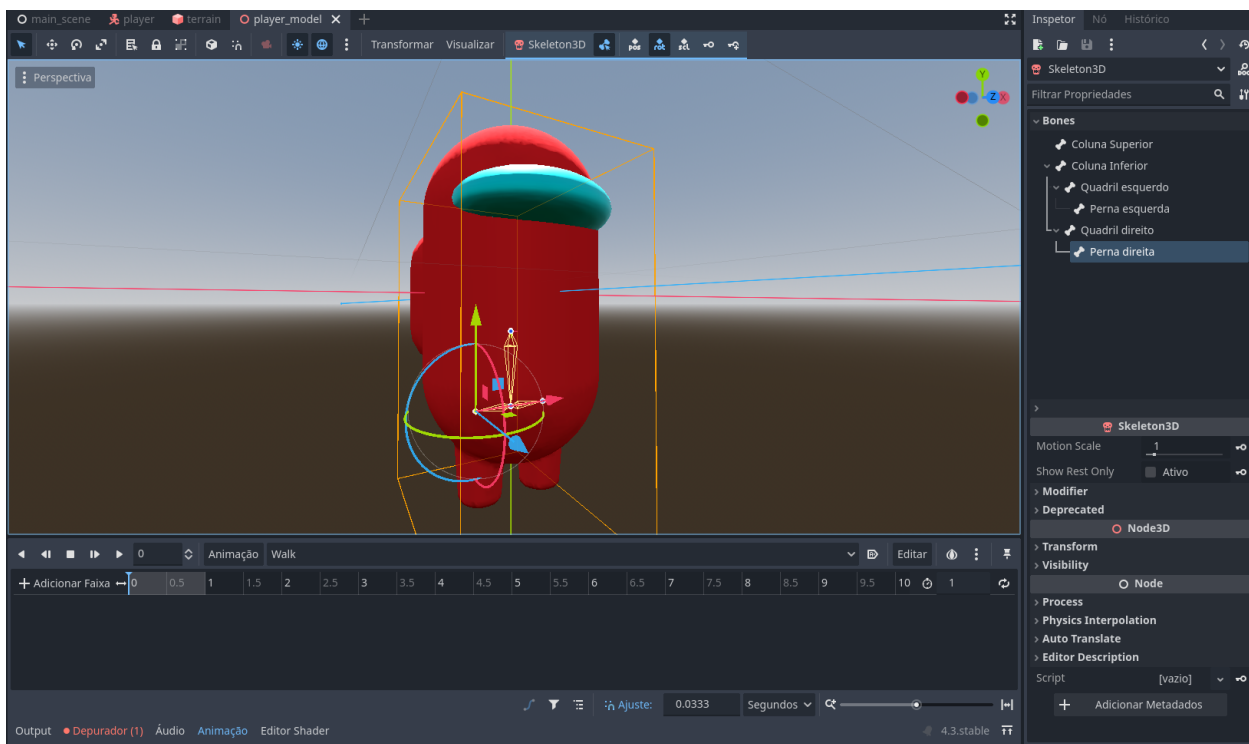


Figura 36: Editor do Osso



10.2.1 Criando as Faixas de Rotação

Antes de iniciar a rotação das pernas, precisamos criar as faixas de rotação e marcar a posição inicial da **Perna Direita** e da **Perna Esquerda**. **Não rotacione as pernas ainda!** Apenas registre suas posições atuais.

Para isso, siga os passos abaixo:

1. Selecione a **Perna Direita**.
2. No menu lateral de propriedades, localize a seção de **rotação**.
3. Clique no ícone de uma **chave**, que representa a criação de um quadro-chave (*keyframe*).
4. Caso apareça um menu perguntando se deseja criar a faixa de animação, clique em **Criar**.
5. Repita o mesmo processo para a **Perna Esquerda**.

Após esses passos, o editor de animação apresentará o seguinte resultado:

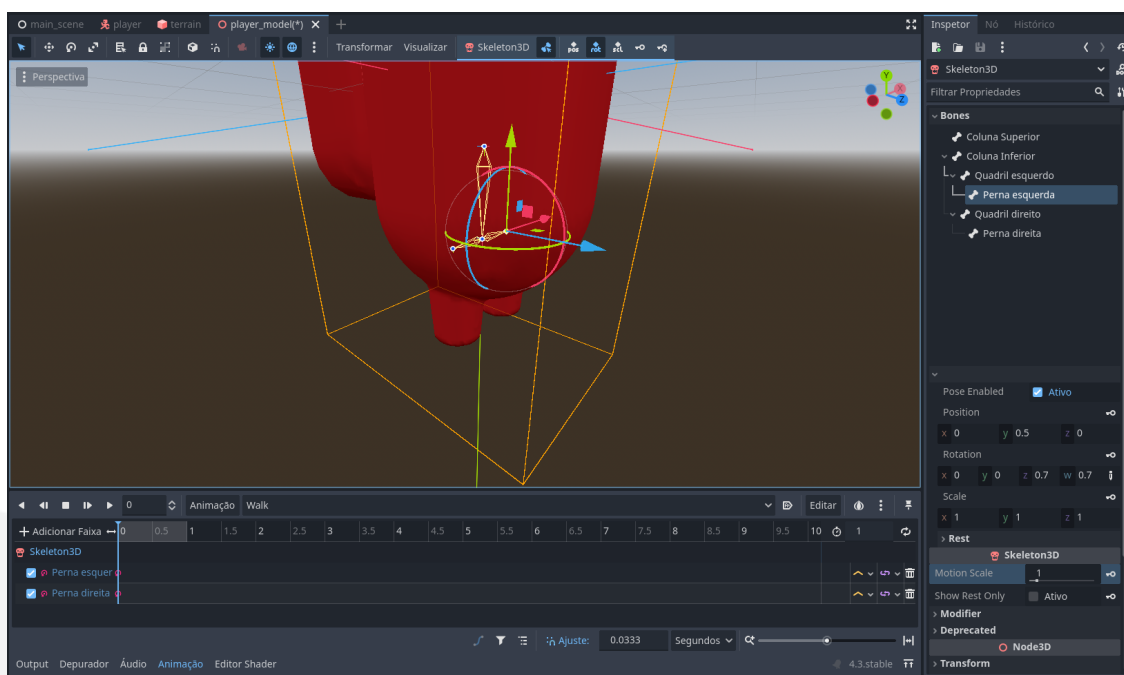


Figura 37: Adicionando Chaves de Animação

Em seguida, ajuste a escala da animação: edite o valor **1** (próximo ao ícone de cronômetro) para **0.5** e altere o tempo de ajuste para **0.25 segundos**, conforme ilustrado na Figura 38.

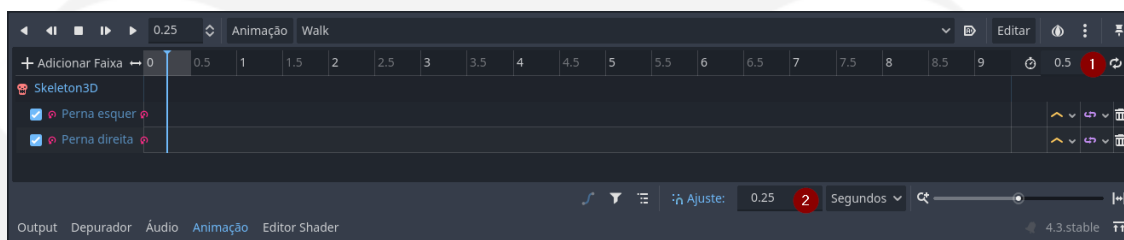


Figura 38: Alterando Escala da Animação



Com o cursor da animação (linha azul) no início, rotacione a **Perna Direita** em **X** (eixo vermelho) para frente e a **Perna Esquerda** em **X** para trás. Clique no ícone da chave para registrar a posição de rotação em ambas as faixas. O resultado deverá ser similar ao exibido na Figura 39.

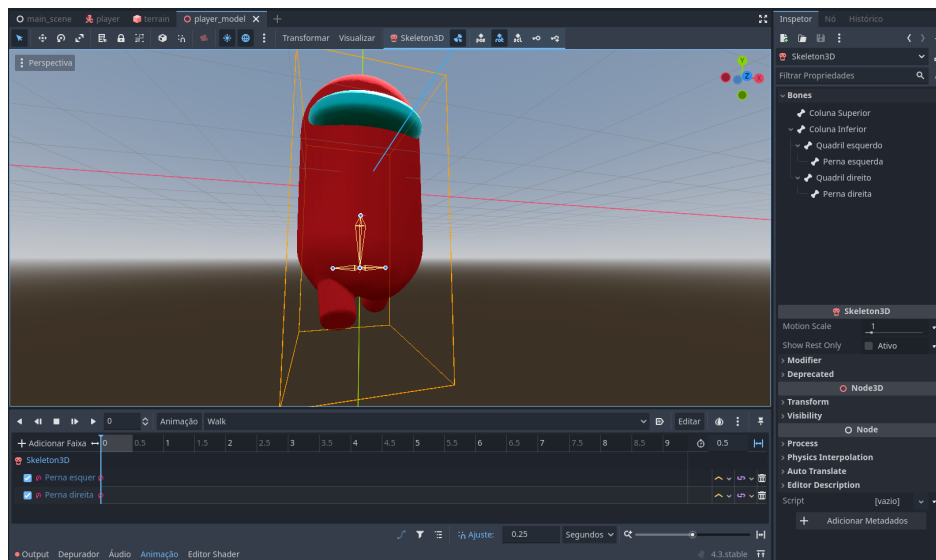


Figura 39: Movimentando as Pernas

Agora, mova o cursor para o fim da animação (0.5 segundos) e rotacione a **Perna Direita** em **X** para trás e a **Perna Esquerda** para frente. Não se esqueça de clicar novamente na chave para adicionar os novos valores. Veja o resultado na Figura 40.

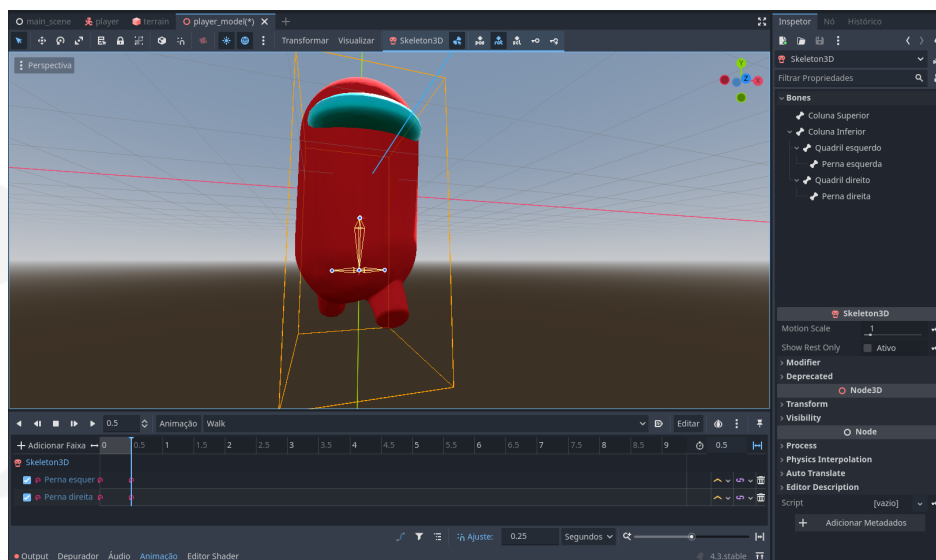


Figura 40: Movimentando as Pernas (Frame 2)



Por fim, configure a animação para entrar em **loop**. No painel de animação, próximo ao cronômetro, ative a opção de loop, conforme mostrado na Figura 41.

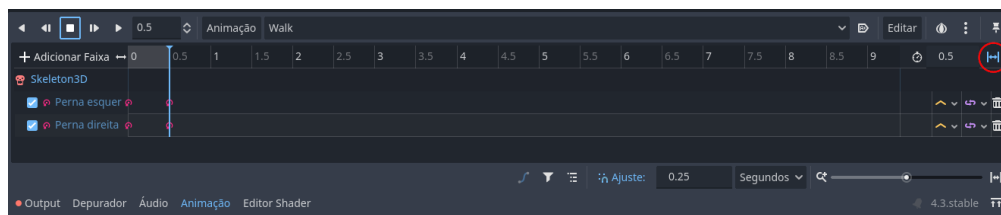


Figura 41: Configurando o Loop da Animação

Ao pressionar o botão de play, a animação das pernas deve ser executada corretamente.



10.3 Criando as Transições de Animação (AnimationTree)

Agora vamos configurar o nó **AnimationTree**, que será responsável por gerenciar as transições entre as animações. Ele funcionará como uma **máquina de estados**, controlando qual animação deve ser executada em determinado momento.

Para configurá-lo, siga os passos abaixo:

1. Selecione o nó **AnimationTree**.
2. No menu de propriedades, vá até **Tree Root** e selecione **Novo AnimationNodeStateMachine**.
3. Em **Anim Player**, selecione o **AnimationPlayer** que criamos anteriormente.
4. No editor de **AnimationTree** (na parte inferior da tela), clique com o botão direito e selecione **Adicionar Animação**.
5. Adicione tanto a animação de **Idle** quanto a animação de **Walk**.

O resultado será semelhante ao da Figura 42.

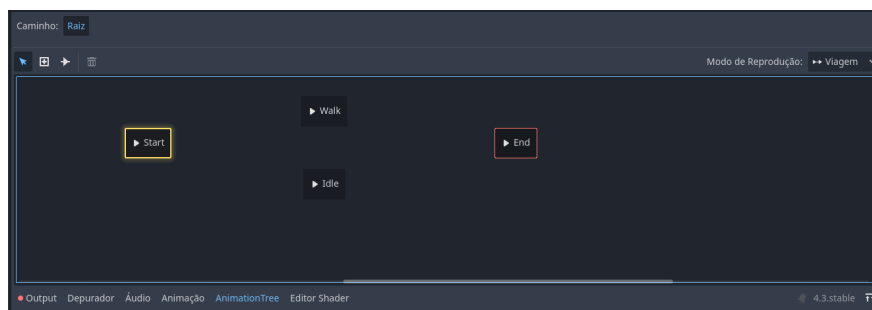


Figura 42: Máquina de Estados no AnimationTree

10.3.1 Criando as Transições entre os Estados

Agora vamos criar as conexões entre os estados de animação:

1. Selecione a ferramenta de transição (ícone de uma seta com uma linha no meio).
2. Clique no estado **Start** e arraste até a animação **Idle**, criando uma transição.
3. Repita o processo para conectar **Idle** a **Walk** e vice-versa.

Após essas conexões, sua máquina de estados ficará semelhante à Figura 43.

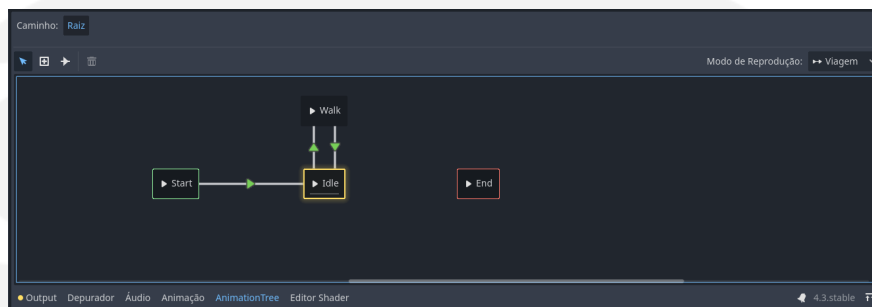


Figura 43: Configuração Inicial das Transições



10.3.2 Controlando as Transições com Variáveis

Se testarmos a animação agora, veremos que ela alterna continuamente entre **Idle** e **Walk**, pois não há uma condição para a transição. Para corrigir isso, vamos adicionar variáveis de controle:

1. Selecione a transição de **Idle** para **Walk**.
2. No menu de propriedades, vá até a aba **Advance**.
3. Na opção **Condition**, digite **walking**.
4. Agora, selecione a transição de **Walk** para **Idle**.
5. Na mesma opção **Condition**, digite **idle**.

Isso garante que a troca entre as animações só aconteça quando a variável correspondente for ativada.

10.3.3 Ajustando a Suavidade da Transição

Para tornar as transições mais naturais, siga os passos abaixo para ambas as transições:

1. No menu de propriedades da transição, desative a opção **Reset**.
2. Em **Switch**, mude o **Switch Mode** para **Sync**.

Isso evita cortes bruscos entre as animações e melhora a fluidez do movimento. No final, a máquina de estados ficará como mostrado na Figura 44.



Figura 44: Máquina de Estados Finalizada



10.4 Controlando as Animações Via Script

10.4.1 Controlando a Animação de Movimentação

Voltando para a cena do personagem em `player.tscn`, vamos editar o script `player.gd` para adicionar o **AnimationTree** como uma variável `@onready`. Além disso, vamos implementar a movimentação da cabeça para quando o personagem olhar para cima ou para baixo, o que requer uma referência ao osso da cabeça e ao **Skeleton3D**.

O código atualizado, contendo todas as variáveis até o momento, ficará assim:

```
# Movimentação
const SPEED = 5.0
const JUMP_VELOCITY = 4.5

# Movimentação da Câmera
@export var mouse_sensitivity := Vector2(0.01, 0.01)
var rotation_horizontal := 0.0
var rotation_vertical := 0.0

# Câmeras
enum CameraModes {
    FIRST_PERSON,
    SECOND_PERSON,
    THIRD_PERSON,
}

var current_camera_mode: CameraModes = CameraModes.FIRST_PERSON

@onready var camera_pivot: Node3D = $CameraPivot
@onready var first_person: Camera3D = $CameraPivot/FirstPerson
@onready var second_person: Camera3D = $CameraPivot/SecondPerson
@onready var third_person: Camera3D = $CameraPivot/ThirdPerson

# Animação
@onready var animation_tree: AnimationTree = $"CollisionShape/PlayerModel/Armação/
    AnimationTree"
@onready var skeleton: Skeleton3D = $"CollisionShape/PlayerModel/Armação/Skeleton3D"
@onready var head_bone := skeleton.find_bone("Coluna Superior")
```

Agora, vamos criar uma função para controlar as condições das transições definidas no **AnimationTree**. Essa função verificará se o personagem está andando ou parado, ajustando as condições **walking** e **idle** corretamente:

```
func set_walk(value: bool):
    animation_tree["parameters/conditions/walking"] = value
    animation_tree["parameters/conditions/idle"] = not value
```



Em seguida, atualizaremos a função `_physics_process()` para chamar `set_walk()` de acordo com a movimentação do personagem:

```
func _physics_process(delta: float) -> void:
    # Add the gravity.
    if not is_on_floor():
        velocity += get_gravity() * delta

    # Handle jump.
    if Input.is_action_just_pressed("ui_accept") and is_on_floor():
        velocity.y = JUMP_VELOCITY

    # Get the input direction and handle the movement/deceleration.
    # As good practice, you should replace UI actions with custom gameplay actions.
    var input_dir := Input.get_vector("ui_left", "ui_right", "ui_up", "ui_down")
    var direction := (transform.basis * Vector3(input_dir.x, 0, input_dir.y)).normalized()
    if direction:
        set_walk(true)
        velocity.x = direction.x * SPEED
        velocity.z = direction.z * SPEED
    else:
        set_walk(false)
        velocity.x = move_toward(velocity.x, 0, SPEED)
        velocity.z = move_toward(velocity.z, 0, SPEED)

    move_and_slide()
```

Ao rodar a cena principal, você verá que a animação de caminhada já está sendo executada corretamente ao mover o personagem.

10.4.2 Controlando a Animação da Cabeça

Para fazer com que a cabeça do personagem gire conforme o movimento do mouse, atualize a função `_process()`:

```
func _process(_delta):
    rotation.y = rotation_horizontal
    camera_pivot.rotation.x = rotation_vertical
    var current_pose := skeleton.get_bone_pose(head_bone)
    current_pose.basis = Basis.from_euler(Vector3(-rotation_vertical, 0, 0))
    skeleton.set_bone_pose(head_bone, current_pose)
```



Ao rodar a cena, a cabeça do personagem deverá girar conforme o movimento do mouse. Se o corpo apresentar um aspecto estranho, isso pode ser devido à otimização de LOD (Level of Detail) aplicada pelo Godot nas *meshes* do personagem, devido a distância que a câmera está do personagem. Para corrigir, vamos na cena `player_model.tscn`, selecione o nó **Corpo Principal**, vá até as propriedades do **GeometryInstance3D** e ajuste a propriedade **Extra Cull Margin** para um valor adequado que seja próximo da distância que a câmera está do personagem (por exemplo, 10m), conforme ilustrado na Figura 45.

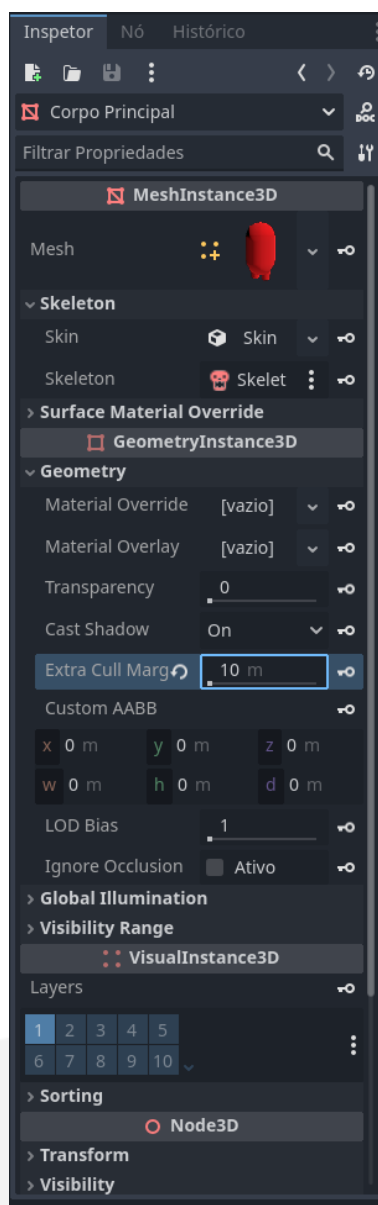


Figura 45: Alterando Extra Cull Margin

Agora, ao rodar a cena principal, os detalhes do corpo do personagem serão exibidos corretamente.



11 Criando os Menus

11.1 Criando a Tela Inicial

11.1.1 Adicionando o Background

Vamos agora criar a tela inicial. Para isso, crie uma nova cena chamada `main_menu.tscn` e escolha como nó pai um nó do tipo **Control**, nomeando-o como **MainMenu**.

Em seguida, adicione um nó filho do tipo **TextureRect** e renomeie-o como **Background**. No menu de propriedades do **TextureRect**, defina a textura como o arquivo `assets/background/background.png`. A tela ficará conforme ilustrado na Figura 46.

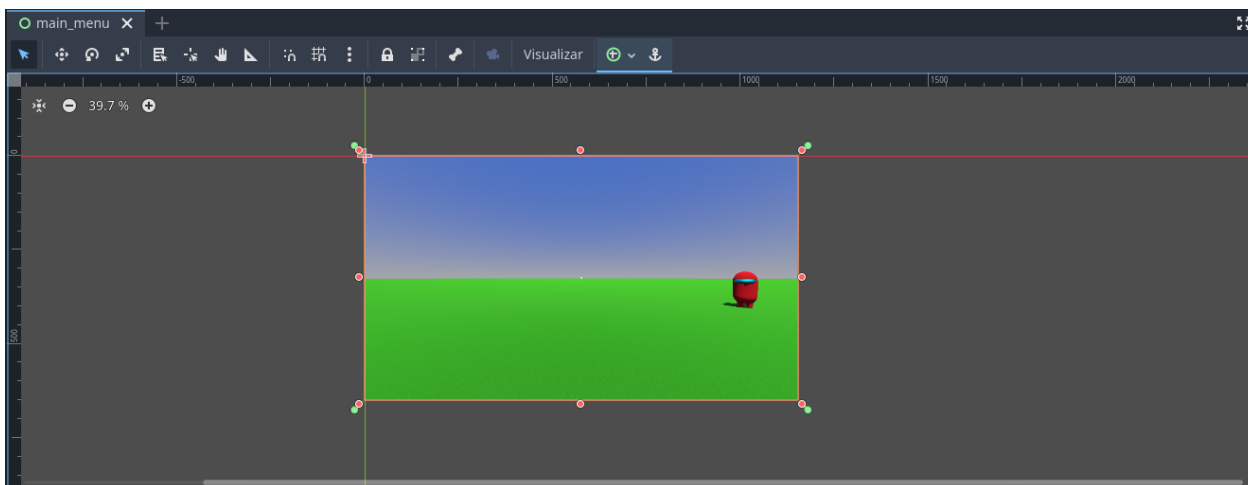


Figura 46: Adicionando o Background

Dica

Caso a imagem fique desalinhada ou não ocupe corretamente a tela, pode ser que as âncoras não estejam configuradas corretamente. Para corrigir isso, vá ao menu de propriedades do **Control**, tanto do **TextureRect** quanto do **MainMenu**, e clique em **Layout > Anchors Preset**, definindo para **Retângulo Completo**.

Observação

Em nós do tipo **Control** e seus derivados, as âncoras do nó pai limitam as âncoras dos filhos. Portanto, para que um nó filho ocupe toda a tela, o nó pai deve permitir essa configuração.



11.1.2 Adicionando o Título

Para adicionar o título do jogo:

1. Adicione um **Label** como filho de **MainMenu** e renomeie-o para **Title**.
2. Defina o texto na propriedade **Text**, por exemplo, *Jogo da Fazenda*.
3. Configure o alinhamento horizontal para **Center**.
4. Para aumentar o tamanho da fonte, adicione um novo **LabelSettings**, vá em **Font > Size** e ajuste o tamanho desejado.
5. Defina as âncoras para **largura superior**, centralizando o **Label** horizontalmente.
6. Ajuste sua posição para a altura desejada.

A tela ficará conforme ilustrado na Figura 47.

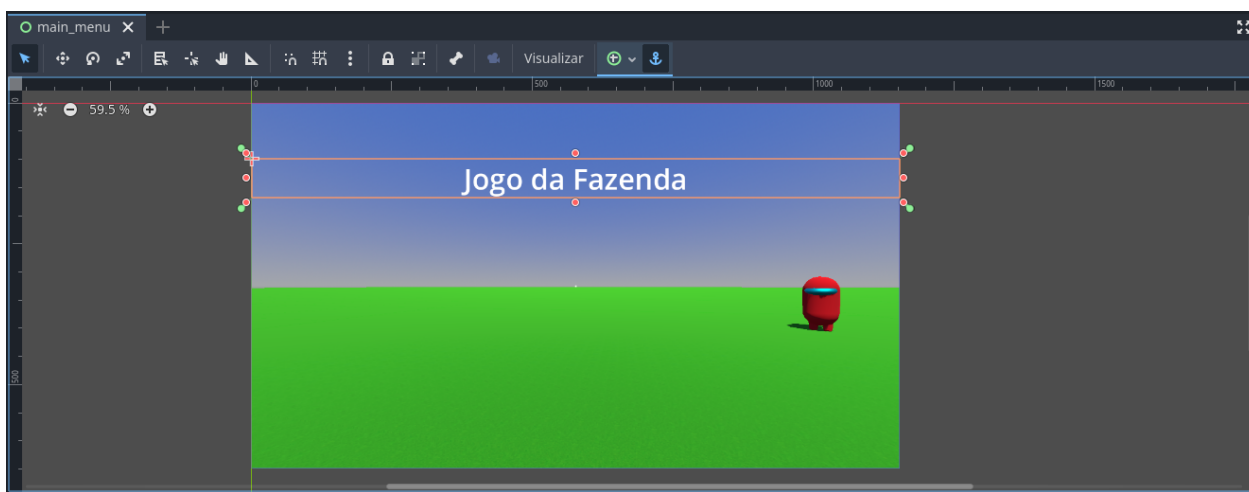


Figura 47: Adicionando o Título



11.1.3 Adicionando os Botões

Para adicionar os botões:

1. Adicione um **VBoxContainer** como filho de **MainMenu** e renomeie-o como **ButtonMenu**.
2. Adicione dois nós do tipo **Button** como filhos de **ButtonMenu**, nomeando-os como **NewGame** e **Exit**.
3. Configure os textos dos botões como *Novo Jogo* e *Sair*, respectivamente.
4. Posicione os botões para que fiquem centralizados na cena.

A tela ficará conforme ilustrado na Figura 48.

Dica

Para posicionar melhor os botões no centro da cena, você pode ativar o encaixe de grade clicando no ícone correspondente no menu superior ou pressionando **Shift+G**. Além disso, você pode ativar o encaixe inteligente pressionando **Shift+S**.

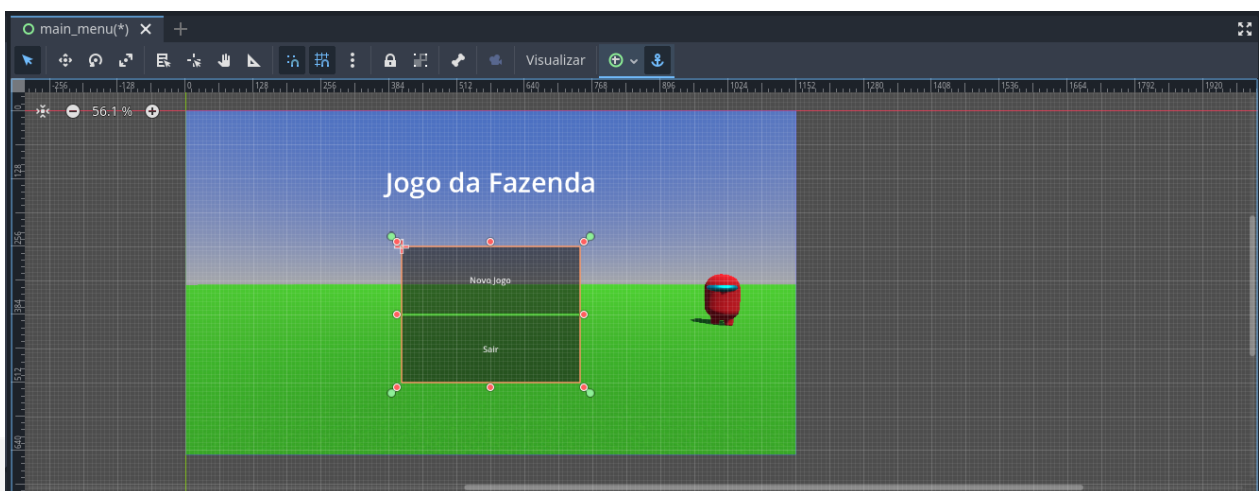


Figura 48: Adicionando os Botões

Para ajustar o espaçamento entre os botões, vá até o menu de propriedades do **ButtonMenu** (**VBoxContainer**) e clique em:

- **Theme Overrides > Constants > Separation.**
- Habilite a opção e defina o espaçamento desejado.



11.1.4 Adicionando o Footer de Informações

Para adicionar o footer de informações:

1. Adicione um **MarginContainer** como filho de **MainMenu**.
2. Como filho de **MarginContainer**, adicione um **HBoxContainer** e renomeie-o como **Footer**.
3. Dentro de **Footer**, adicione dois nós do tipo **Label**, nomeando-os como **Version** e **Author**.
4. No **MarginContainer**, defina as âncoras como **largura inferior**, posicionando corretamente o footer.
5. Para distribuir o texto, selecione **Version**, configure seu **Container Sizing Horizontal** para **Preencher** e habilite a opção **Expandir**.
6. Para evitar que os textos fiquem colados na parte inferior da tela, vá ao menu de propriedades do **MarginContainer** e acesse:
 - **Theme Overrides > Constants > Margin.**
 - Defina as margens desejadas.

A tela ficará conforme ilustrado na Figura 49.

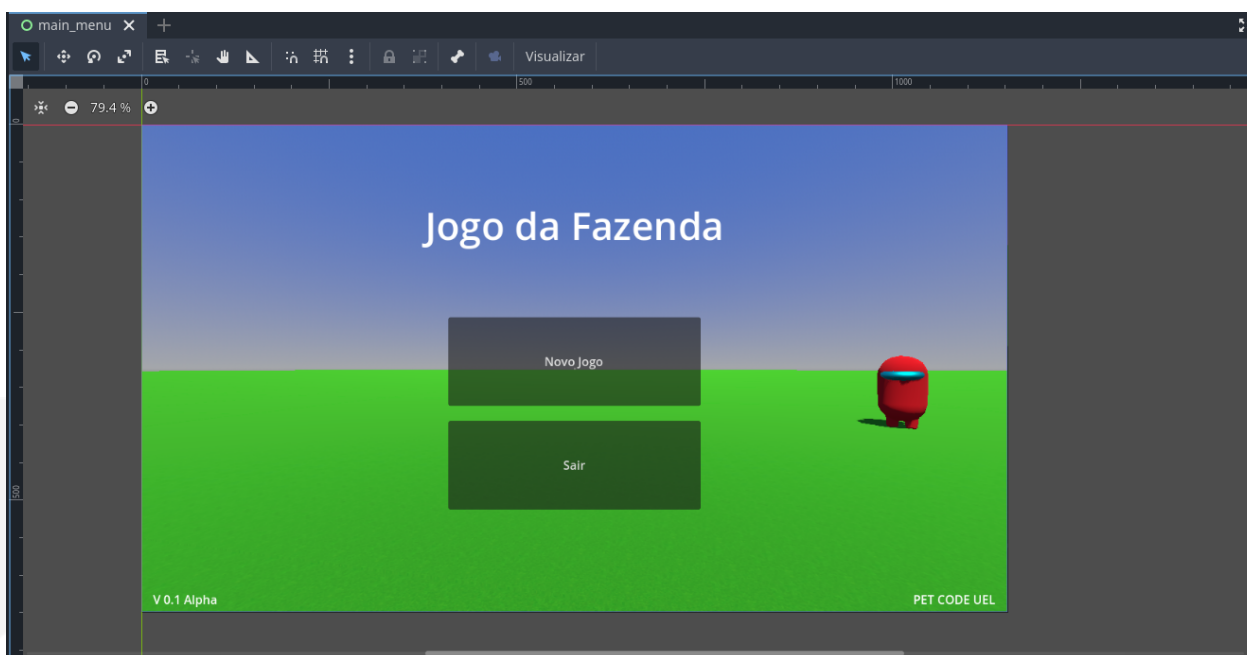


Figura 49: Adicionando o Footer de Informações



11.1.5 Acrescentando um Pouco de Estilo

Em **MainMenu**, na barra lateral de propriedades vá em **Control > Theme > Novo Theme**, aparecerá um menu assim:

Dica

Adicionando estilos em um nó pai, ele propaga os estilos para todos os seus filhos, evitando que você tenha que repetir o mesmo estilo em todos os filhos.

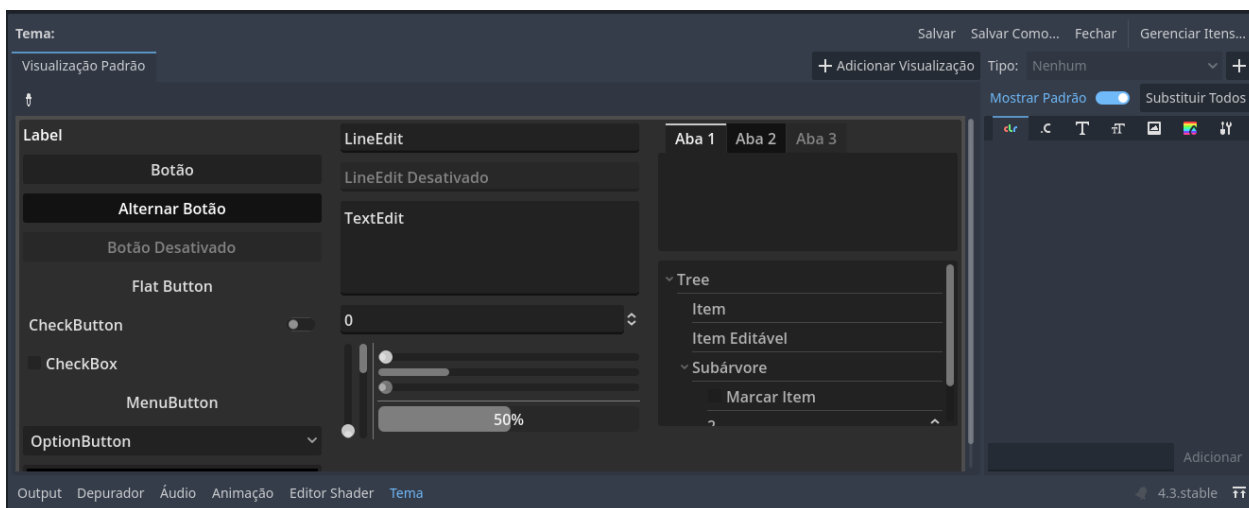


Figura 50: Menu de Temas

Vamos modificar o botão padrão para adicionar bordas arredondadas. Para importar os estilos do botão padrão do próprio Godot, vá em **Gerenciar Itens... > Importar Itens > Tema Padrão**, selecione o **Button** e importe com dados. Vamos na aba de **StyleBoxes** (ícone em arco-íris com uma gota), e para cada **StyleBox** já existente em **Button**, edite a propriedade **Corner Radius** para **20**. A tela ficará como na imagem mostrada na Figura 51.

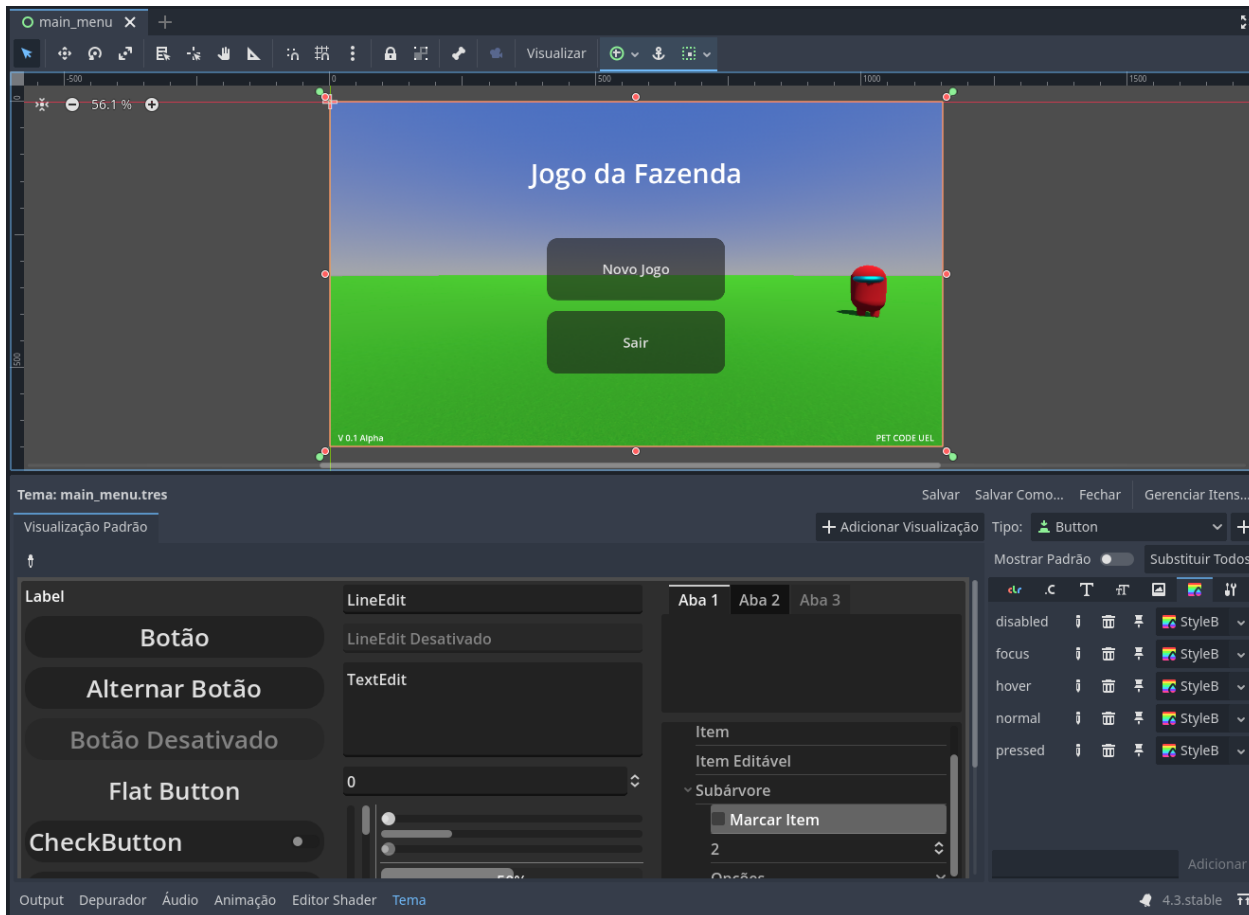


Figura 51: Adicionando um Pouco de Estilo

Agora estilize como desejar. O sistema permite personalizar quase todos os tipos de nós de interface. Caso não queira que um estilo global seja propagado para um filho em específico, adicione um tema somente para aquele filho. Isso sobrescreverá o estilo do pai, permitindo especializar o filho. Para reutilizar esse estilo em outras cenas, é uma boa prática salvá-lo externamente. Para fazer isso, clique em **Salvar Como...** e nomeie o arquivo como **main_menu.tres**.



11.1.6 Criando o Script de Transição

Para criar um script de transição, crie um novo script associado ao **MainMenu** e nomeie-o como **main_menu.gd**. Agora, selecione um dos botões e, no menu de propriedades no topo, acesse o menu **Nó**, que contém o submenu de **Sinais**. Esses sinais serão lançados quando o botão for pressionado, e podemos usá-los para vincular a transição.

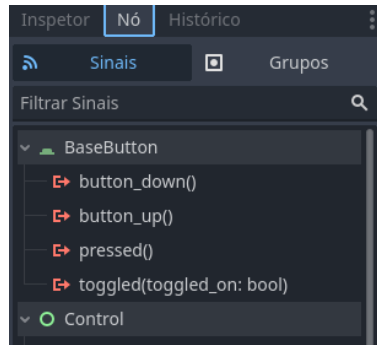


Figura 52: Menu de Sinais

Para fazer isso no botão **NewGame**, selecione o sinal **pressed()** e clique em **Conectar** ou dê dois cliques. Selecione o nó **MainMenu** e clique em **Conectar**. O script será gerado com a seguinte função:

```
extends Control

func _on_new_game_pressed() -> void:
    pass # Replace with function body.
```

Faça o mesmo para o **Exit**, e o script ficará assim:

```
extends Control

func _on_new_game_pressed() -> void:
    pass # Replace with function body.

func _on_exit_pressed() -> void:
    pass # Replace with function body.
```

Agora, adicione a lógica para que, ao clicar em **NewGame**, o jogo seja iniciado, referenciando **main_scene.tscn**. Para isso, modifique a função **_on_new_game_pressed()**:

```
func _on_new_game_pressed() -> void:
    get_tree().change_scene_to_file("res://scenes/main_scene.tscn")
```

Agora, para adicionar a funcionalidade de sair do jogo, modifique a função **_on_exit_pressed()**:

```
func _on_exit_pressed() -> void:
    get_tree().quit()
```

Ao testar a cena, os botões funcionarão corretamente. Agora, precisamos definir essa cena como a principal do projeto para que, ao iniciar o jogo, ele comece por ela. Para isso, vá em **Project > Project**



Settings... > Geral > Aplicação > Executar e defina a cena principal como `main_menu.tscn`. Assim, ao clicar no botão de **Play** do Godot, ele executará sempre essa cena, e quando o jogo for exportado, ele iniciará a partir dela também.



11.2 Criando o Menu de Pausa

11.2.1 Criando a Cena

Para criar o menu de pausa, siga os passos abaixo:

1. Crie uma nova cena vazia e nomeie-a como **pause_menu.tscn**.
2. Defina o nó pai como **Control** e renomeie-o para **PauseMenu**.
3. Para escurecer a tela e dar a impressão de que o jogo está pausado, adicione um **ColorRect** como filho de **PauseMenu** e renomeie-o para **Overlay**.
4. Altere sua cor para um preto transparente, por exemplo, **#00000060**.
5. Como filho de **PauseMenu**, adicione um **CenterContainer** e renomeie-o para **Center**.
6. Dentro de **Center**, adicione um **Panel** e, em suas propriedades, defina um **Custom Minimum Size** de **600** para o eixo x e **200** para o eixo y.

A estrutura dos nós ficará conforme ilustrado na Figura 53.

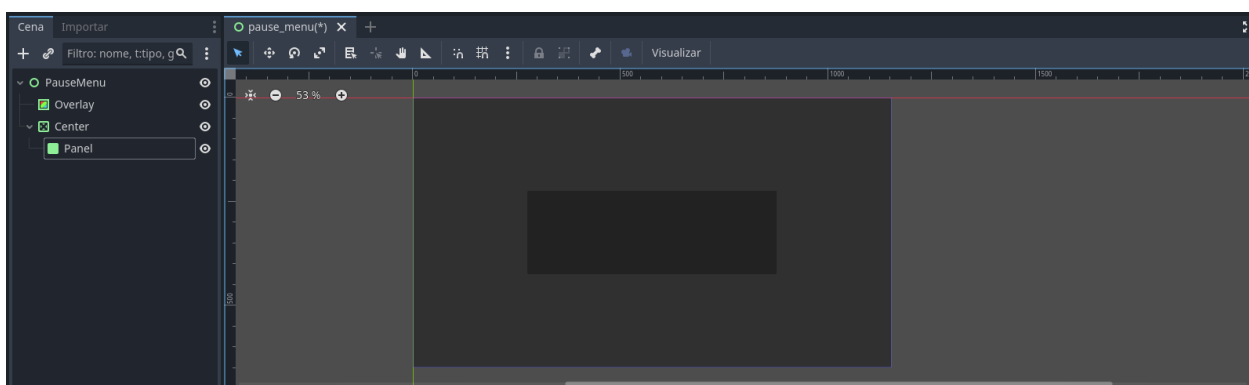


Figura 53: Adicionando o Panel



Agora, siga os passos abaixo para estruturar os botões do menu de pausa:

1. Como filho de **Panel**, adicione um **MarginContainer** e renomeie-o para **Margin**.
2. Dentro de **Margin**, adicione um **VBoxContainer** e renomeie-o para **VBox**.
3. Adicione três nós do tipo **Button** como filhos de **VBox** e nomeie-os como **Resume**, **MenuExit** e **DesktopExit**.

A tela ficará conforme ilustrado na Figura 54.

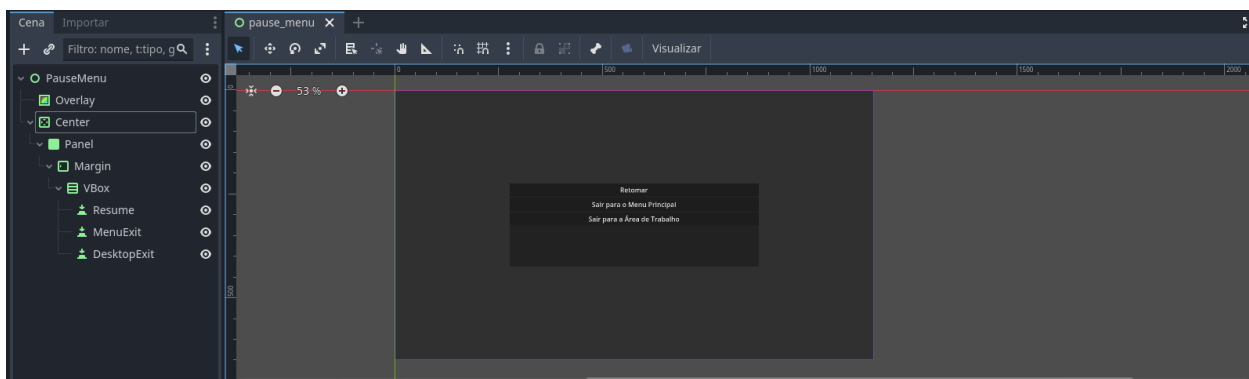


Figura 54: Menu de Pausa

Agora, precisamos ajustar o layout para que os botões fiquem alinhados ao centro do **Panel**. Para isso:

- Ajuste as margens do **Margin**, definindo, por exemplo, **40** em todas as direções.
- Ajuste o espaçamento no **VBox** para tornar o layout mais harmonioso.

O layout final ficou conforme ilustrado na Figura 55.

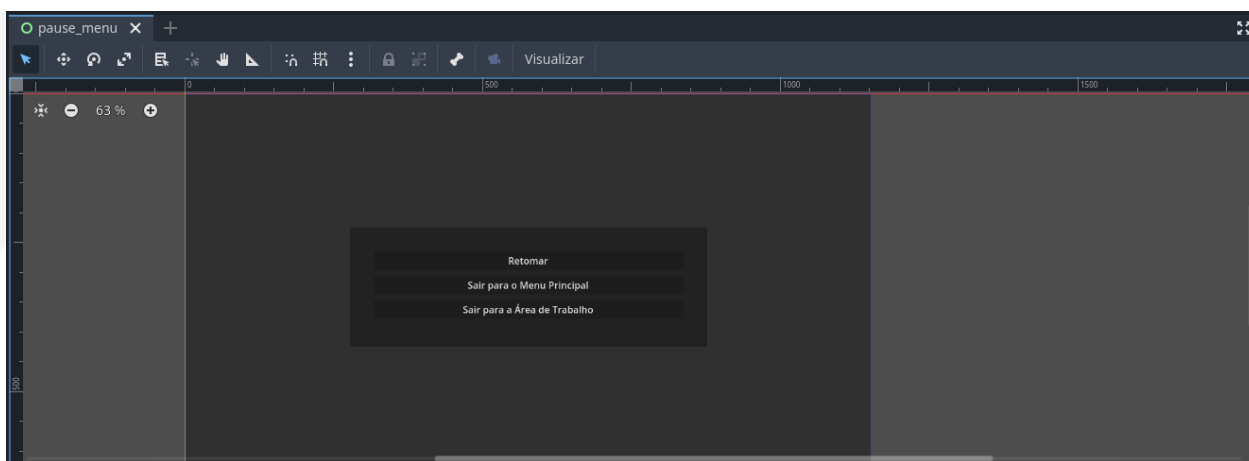


Figura 55: Ajuste do Layout da Tela de Pausa



11.2.2 Criando o Script de Pause

Crie um novo script associado a **PauseMenu** e nomeie-o como **pause_menu.gd**. Em seguida, selecione cada um dos botões e conecte o sinal **pressed()**, conforme demonstrado anteriormente. O script ficará da seguinte forma:

```
extends Control

func _on_resume_pressed() -> void:
    pass # Replace with function body.

func _on_menu_exit_pressed() -> void:
    pass # Replace with function body.

func _on_desktop_exit_pressed() -> void:
    pass # Replace with function body.
```

Quando o jogador pausar o jogo, o *mouse* deverá aparecer. O controle do *mouse*, que configuramos anteriormente, está centralizado na **UIManager**. Assim, para mantermos a consistência na centralização das ações de interface, precisaremos implementar alguns métodos nela.

Primeiramente, instancie o menu de pausa na **main_scene.tscn** como filho do nó **UI** e oculte-o clicando no ícone do olho. No script **ui.gd**, crie uma variável **@onready** para referenciar o menu de pausa:

```
@onready var pause_menu: Control = $PauseMenu
```

Observação

Note que, no código de **ui.gd**, passamos a referência do próprio nó **UI** para o **UIManager**. Dessa forma, não é necessário referenciar o **PauseMenu** separadamente, pois, acessando o nó pai, temos acesso a todos os seus filhos, inclusive às suas variáveis e funções.

Agora, podemos acessar o menu de pausa a partir da **UIManager**. No script **ui_manager.gd**, adicione a variável correspondente. O código, com todas as variáveis definidas até o momento, ficará assim:

```
extends Node

# Referência para o nó UI que contém todos os elementos de interface
var ui: UI

var is_pause_open: bool = false # O jogo, por padrão, inicia sem estar pausado
```

Em seguida, crie uma função que alterne o estado de pausa, exibindo ou ocultando o menu de pausa. Lembre-se de que, ao pausar o jogo, o *mouse* (que estava capturado para movimentar a câmera do jogador) deverá ser liberado. Para isso, crie a seguinte função:



```
func toggle_pause() -> void:
    is_pause_open = !is_pause_open
    ui.pause_menu.visible = is_pause_open
    get_tree().paused = is_pause_open
    if is_pause_open:
        Input.set_mouse_mode(Input.MOUSE_MODE_VISIBLE)
    else:
        Input.set_mouse_mode(Input.MOUSE_MODE_CAPTURED)
```

Contudo, essa função não pode ser utilizada se o jogador optar por sair para o menu principal, pois nesse caso precisamos despausar o jogo e manter o *mouse* visível — ação que **toggle_pause()** não realiza. Para resolver essa situação, crie uma nova função que force o despausamento e libere o *mouse*:

```
func force_unpause() -> void:
    is_pause_open = false
    ui.pause_menu.visible = false
    get_tree().paused = false
    Input.set_mouse_mode(Input.MOUSE_MODE_VISIBLE)
```

Agora, retorne ao script **pause_menu.gd** e atualize as funções para chamar os métodos criados. O código ficará assim:

```
extends Control

func _on_resume_pressed() -> void:
    UIManager.toggle_pause()

func _on_menu_exit_pressed() -> void:
    UIManager.force_unpause()
    get_tree().change_scene_to_file("res://scenes/ui/menus/main_menu.tscn")

func _on_desktop_exit_pressed() -> void:
    get_tree().quit()
```

Ainda não adicionamos um botão específico para pausar o jogo. Observe que o Godot já possui uma associação de tecla para a ação **ui_cancel** (normalmente vinculada à tecla **ESC**). Caso queira que outro botão também pause o jogo, proceda como demonstrado na Seção 9. Realizaremos o rastreamento do *input* no nó **UI** presente na **main_scene.tscn**. Para isso, adicione o seguinte código ao script **ui.gd**:

```
func _input(event):
    if event.is_action_pressed("ui_cancel"):
        UIManager.toggle_pause()
```

Embora todos os scripts estejam prontos, ao executar a cena o menu de pausa aparecerá, mas os botões poderão não funcionar — inclusive o botão de sair. Isso ocorre porque a chamada **get_tree().paused** está pausando inclusive o menu de pausa. Para corrigir isso, siga os passos abaixo:

1. Abra a cena **main_scene.tscn** e selecione o nó **UI**. Em suas propriedades, vá até **Node** → **Process** e altere o *Mode* para **Always**.



2. No nó **Game**, em **Node** → **Process**, modifique o *Mode* para **Pausable**.

Dessa forma, somente o jogo será pausado, enquanto a interface (**UI**) permanecerá ativa. Para otimizar o desempenho, ajuste o *Mode* do nó **PauseMenu** para **When Paused**, evitando que a cena seja executada em segundo plano desnecessariamente.

Observação: Mesmo que um nó esteja oculto, ele continua executando suas rotinas.



12 Criação do Inventário do Personagem

12.1 Criação da Cena do Inventário

Crie uma nova cena chamada **inventory.tscn**, que conterá o inventário do personagem. Defina como nó pai um nó do tipo **Control** e renomeie-o como **Inventory**.

1. Adicione um nó filho do tipo **ColorRect** e renomeie-o como **Overlay**. Defina sua cor como preto transparente (**#00000060**).
2. Como filho de **Inventory**, adicione um **MarginContainer** e renomeie-o como **Margin**. Configure suas margens para **60** unidades em todas as direções.
3. Como filho de **Margin**, adicione um **Panel**.
4. Dentro do **Panel**, adicione um **MarginContainer** e renomeie-o como **Margin**. Configure suas margens para **30** unidades em todas as direções.
5. Como filho de **Margin**, adicione um **VBoxContainer** e renomeie-o como **VBox**.
6. Dentro de **VBox**, adicione dois nós do tipo **GridContainer**, nomeando-os como **InventoryGrid** e **HotbarGrid**.

A estrutura da cena ficará conforme ilustrado na Figura 56.

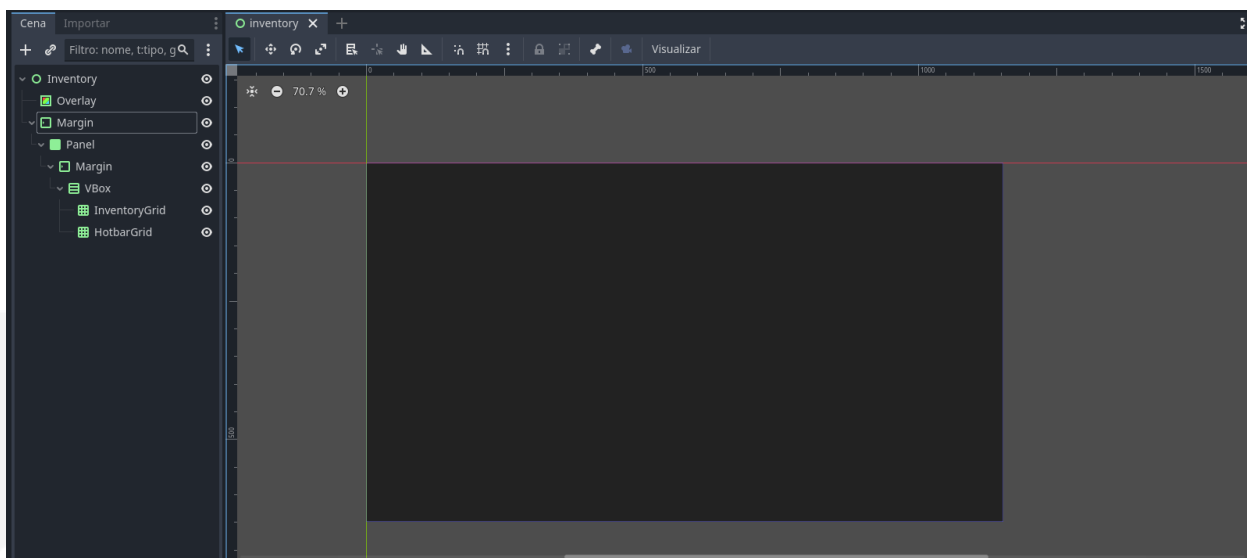


Figura 56: Estrutura da Cena do Inventário



12.2 Criação da Cena do Slot

Agora, vamos criar a cena que representará cada slot do inventário. Crie uma nova cena chamada `item_slot.tscn`, que conterá o slot do inventário. Defina como nó pai um nó do tipo **Button** e renomeie-o como **ItemSlot**.

1. Como filho de **ItemSlot**, adicione um **MarginContainer** e renomeie-o como **Margin**.
2. Dentro de **Margin**, adicione um **TextureRect** e renomeie-o como **ItemIcon**.
3. Ainda dentro de **Margin**, adicione um **Label** e renomeie-o como **Index**.
4. Como filho de **ItemSlot**, adicione um **Label** e renomeie-o como **ItemQuantity**.
5. No nó **ItemSlot**, vá para **Control** > **Transform** e defina o tamanho para **74px x 74px**.
6. Configure a predefinição de margem para **5px** em todas as direções.
7. Configure a predefinição de âncora para **Centro** e ajuste para a proporção desejada.

A estrutura da cena ficará conforme ilustrado na Figura 57.

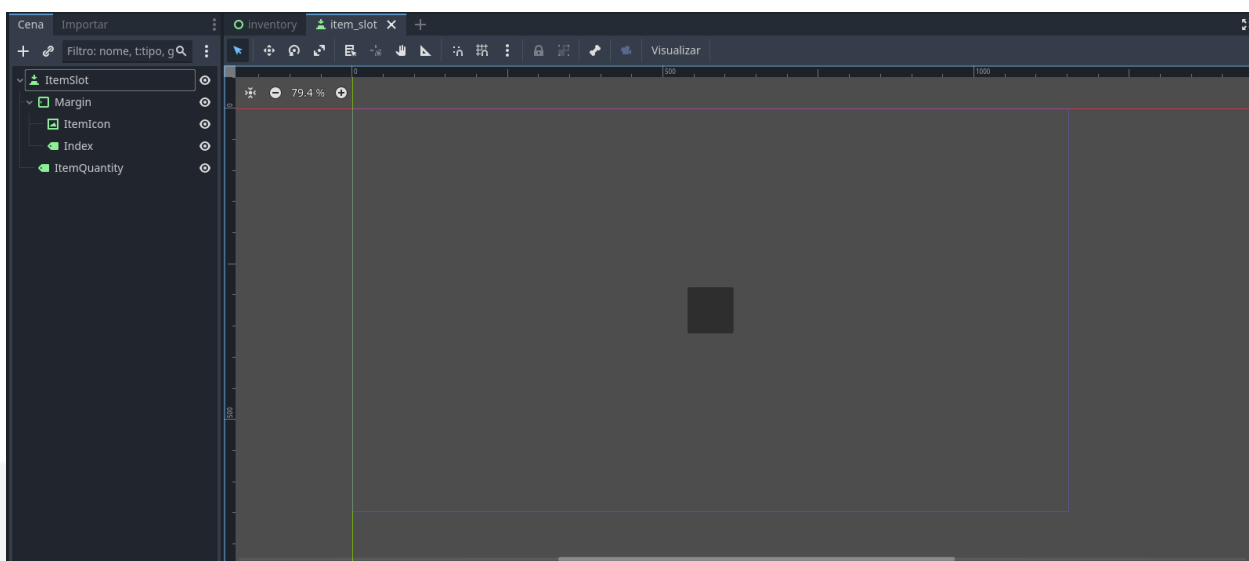


Figura 57: Estrutura da Cena do Slot

Nota: Usamos **74px x 74px**, pois queremos que os ícones dos itens tenham **64px x 64px**, e já que adicionamos margens de 5 em cada lado, então perdemos 10px na altura e 10px na largura.



12.3 Criação do Script do Inventário

12.3.1 Criação da Classe dos Itens

Para representar os itens vamos criar uma classe chamada **Item**, ela herdará de **Resource**, e terá os seguintes atributos:

- **name**: Nome do item.
- **texture**: Textura do item.
- **max_stack**: Quantidade máxima de itens que podem ser colocados no inventário.
- **quantity**: Quantidade de itens no inventário.
- **price**: Preço do item.
- **can_sell**: Indica se o item pode ser vendido.

Além disso vamos criar um método adicional para criar uma cópia do item, pois dependendo de como manipulamos o Godot pode passar uma referência ao item ao invés de uma cópia do item. Então crie um script chamado **item.gd** e adicione o seguinte código:

```
class_name Item
extends Resource

@export var name: String = ""
@export var texture: Texture2D = null
@export var max_stack: int = 64
@export var quantity: int = 1
@export var price: int = 0
@export var can_sell: bool = true

func clone() -> Item:
    return duplicate() as Item
```



12.3.2 Criação do InventoryManager

Para gerenciar o inventário de forma eficaz, adotaremos uma abordagem similar à utilizada no **UIManager**. Assim, criaremos um script denominado **inventory_manager.gd**, que funcionará como um **Singleton**, garantindo sua disponibilidade **global** no projeto.

Configurações Iniciais:

- **Criação do Script:** Crie o arquivo **inventory_manager.gd** herdando de **Node**.
- **AutoLoad:** Configure-o como **AutoLoad** em **Project Settings**.
- **Dimensões do Inventário:** O inventário conterá **36 slots** organizados em **4 linhas** e **9 colunas** (sendo uma das linhas destinada à **hotbar**).
- **Signal de Atualização:** Adicione um **signal** que será emitido sempre que o inventário for modificado, permitindo a atualização da interface (UI) posteriormente.

Utilizaremos uma **estrutura matricial** para armazenar os itens do inventário. Insira o seguinte código em **inventory_manager.gd**:

```
signal inventory_updated # Emite quando o inventário é alterado

const ROWS := 4
const COLUMNS := 9
const INVENTORY_SIZE := (ROWS - 1) * COLUMNS
const HOTBAR_SIZE := 1 * COLUMNS

var inventory_items = []
```



Problema

Questão: Qual o tipo adequado para `inventory_items`?

Intuição Inicial: Utilizar um Array de Arrays, por exemplo:

```
var inventory_items: Array[Array[Item]] = []
```

Problema: O Godot não permite definir a tipagem de um contêiner de forma que seus elementos sejam outros contêineres tipados.

Alternativa: Utilizar um **Array** simples, onde cada elemento é um **Array** representando uma linha do inventário:

```
var inventory_items: Array = []

func _init():
    for i in range(ROWS):
        var row: Array[Item] = []
        for j in range(COLUMNS):
            row.append(null)
        inventory_items.append(row)
```

Problema: A IDE não reconhece `inventory_items` como um Array de Arrays, limitando a detecção de erros e a funcionalidade de autocompletar.

Solução

Solução Proposta: Criar uma classe para representar uma **linha do inventário**.

Para isso, crie um script chamado `utils.gd`, adicione-o como **AutoLoad** e posicione-o no topo da ordem de carregamento (pois será utilizado em outros AutoLoads). Em `utils.gd`, insira:

```
class ItemRow:
    var slots: Array[Item] = []

    func _init(size: int):
        for i in range(size):
            slots.append(null)
```

Agora, em `inventory_manager.gd`, utilize a classe `ItemRow` para representar cada linha:

```
var inventory_items: Array[Utils.ItemRow] = []
```



Ainda em **utils.gd**, adicione duas funções auxiliares para facilitar o acesso aos índices da matriz:

```
func get_row(slot_index: int, columns: int) -> int:
    var row = floor(slot_index / float(columns))
    return row

func get_col(slot_index: int, columns: int) -> int:
    var col = slot_index % columns
    return col
```

De volta ao **inventory_manager.gd**, inicialize as linhas do inventário utilizando o método especial **_init()**, o qual é executado na criação do objeto, antes do método padrão **_ready()**:

```
func _init():
    for i in range(ROWS):
        inventory_items.append(Utils.ItemRow.new(COLUMNS))
```

Getters e Setters: Em seguida, crie funções para acessar os slots do inventário, tanto de forma global (para o inventário completo, incluindo a hotbar) quanto especificamente para a hotbar:

```
func get_inventory_item(slot_index: int) -> Item:
    var row = Utils.get_row(slot_index, COLUMNS)
    var col = Utils.get_col(slot_index, COLUMNS)
    if row >= 0 and row < ROWS and col >= 0 and col < COLUMNS:
        return inventory_items[row].slots[col]
    return null

func set_inventory_item(slot_index: int, item: Item):
    var row = Utils.get_row(slot_index, COLUMNS)
    var col = Utils.get_col(slot_index, COLUMNS)
    if row >= 0 and row < ROWS and col >= 0 and col < COLUMNS:
        inventory_items[row].slots[col] = item
        inventory_updated.emit()

func get_hotbar_item(index: int) -> Item:
    if index >= 0 and index < HOTBAR_SIZE:
        return inventory_items[ROWS - 1].slots[index]
    return null

func set_hotbar_item(index: int, item: Item):
    if index >= 0 and index < HOTBAR_SIZE:
        inventory_items[ROWS - 1].slots[index] = item
        inventory_updated.emit()
```



Troca de Itens: Crie uma função para realizar a troca de itens, que recebe dois índices, efetua o **swap** dos itens.

```
func swap_items(src_index : int, dst_index : int):  
    var src_item = get_inventory_item(src_index)  
    var dst_item = get_inventory_item(dst_index)  
    if src_item != null or dst_item != null:  
        set_inventory_item(src_index, dst_item)  
        set_inventory_item(dst_index, src_item)
```

Adição de Itens: Implemente uma função que adiciona um item ao inventário. A função receberá um objeto da classe **Item** e seguirá os seguintes passos:

1. Procura um item idêntico no inventário para empilhá-lo, caso o item seja empilhável e não tenha atingido o máximo.
2. Se não houver espaço para empilhar, procura o primeiro slot vazio e posiciona o novo item.
3. Retorna um valor booleano indicando se a adição foi bem-sucedida.

```
func add_item_to_inventory(new_item: Item) -> bool:  
    # Procura no inventário um item igual para empilhar  
    for row in inventory_items:  
        for slot in row.slots:  
            if slot and slot.name == new_item.name and slot.quantity < slot.max_stack:  
                var add_amount = min(new_item.quantity, slot.max_stack - slot.quantity)  
                slot.quantity += add_amount  
                new_item.quantity -= add_amount  
                if new_item.quantity <= 0:  
                    inventory_updated.emit()  
                return true  
  
    # Se não foi possível empilhar, coloca em um slot vazio  
    for row in inventory_items:  
        for i in range(row.slots.size()):  
            if row.slots[i] == null:  
                row.slots[i] = new_item.clone()  
                inventory_updated.emit()  
                return true  
  
    return false # Inventário cheio
```



Remoção de Itens: Por fim, implemente uma função que remove uma quantidade arbitrária de itens do inventário. A função recebe:

- **item_name (String):** nome do item a ser removido.
- **quantity (int):** quantidade a ser removida.

Ela verifica se a quantidade total disponível é suficiente e, se sim, remove os itens, retornando um valor booleano indicando o sucesso da operação.

```
func remove_item_from_inventory(item_name: String, quantity: int) -> bool:
    var needed = quantity
    var slots = []

    # Primeiro, coletamos os slots que possuem o item
    for row in inventory_items:
        for col in range(row.slots.size()):
            var item = row.slots[col]
            if item and item.name == item_name:
                slots.append([row, col, item])
                needed -= item.quantity
                if needed <= 0:
                    break
        if needed <= 0:
            break

    # Se não há itens suficientes, retorna falso
    if needed > 0:
        return false

    # Agora removemos a quantidade necessária
    for slot in slots:
        var row: Utils.ItemRow = slot[0]
        var col: int = slot[1]
        var item : Item = slot[2]

        if item.quantity > quantity:
            item.quantity -= quantity
            inventory_updated.emit()
            return true

        quantity -= item.quantity
        row.slots[col] = null

    inventory_updated.emit()
    return true
```



12.3.3 Criação do Script do Inventário

Agora voltando a nossa cena **inventory.tscn**, adicione um script chamado **inventory.gd** associe-o com o nó pai **Inventory** e acrescente as seguintes variáveis:

```
extends Control

@export var slot_scene: PackedScene
@onready var hotbar_grid: GridContainer = $Margin/Panel/Margin/VBox/HotbarGrid
@onready var inventory_grid: GridContainer = $Margin/Panel/Margin/VBox/InventoryGrid
```

Como temos uma variável **@export**, clique no nó pai **Inventory** e em propriedades selecione Slot Scene, e carregue a cena **item_slot.tscn**. Agora vamos criar uma função auxiliar para excluir todos os filhos de um nó. Para isso, crie a seguinte função:

```
func clear_children(node: Node):
    for child in node.get_children():
        child.queue_free()
```



Agora vamos criar duas funções uma que será utilizada para inicializar a UI do inventário e outra que irá atualizar a UI do inventário. Para isso, adicione as seguintes funções:

```
# Inicializa a interface com os slots
func initialize_ui():
    clear_children(hotbar_grid)
    clear_children(inventory_grid)

    # Cria os slots do inventário
    for i in range(InventoryManager.INVENTORY_SIZE):
        var slot = slot_scene.instantiate()
        slot.slot_index = i
        slot.add_to_group("inventory_slots")
        inventory_grid.add_child(slot)

    # Cria os slots da hotbar_grid
    for i in range(InventoryManager.HOTBAR_SIZE):
        var slot = slot_scene.instantiate()
        slot.slot_index = i + InventoryManager.INVENTORY_SIZE
        slot.add_to_group("inventory_slots")
        hotbar_grid.add_child(slot)

    # Atualiza os dados dos slots após a inicialização
    refresh_ui()

# Atualiza todos os slots na UI
func refresh_ui():
    for i in range(InventoryManager.INVENTORY_SIZE):
        var slot = inventory_grid.get_child(i)
        var item = InventoryManager.get_inventory_item(i)
        slot.set_item(item)

    for i in range(InventoryManager.HOTBAR_SIZE):
        var slot = hotbar_grid.get_child(i)
        var item = InventoryManager.get_hotbar_item(i)
        slot.set_item(item)
```

Agora vamos chamar a função **initialize_ui()** na função **_ready()** da cena **inventory.tscn**. E também vamos linkar o evento **inventory_updated** ao **refresh_ui()**, para que toda vez que o inventário seja atualizado, a UI seja atualizada.

```
func _ready():
    initialize_ui()
    InventoryManager.inventory_updated.connect(refresh_ui)
```



12.4 Criação do Script do Slot

Na cena `item_slot.tscn`, associe um script denominado `item_slot.gd` ao nó pai `ItemSlot`. Para facilitar a integração com a IDE em outros scripts, declare o `class_name` e defina as seguintes variáveis:

```
class_name ItemSlot
extends Button

var item: Item = null
var dragged_item: Item = null
var slot_index: int = -1
@onready var item_icon : TextureRect = $Margin/ItemIcon
@onready var quantity_label : Label = $ItemQuantity
@onready var index_label : Label = $Margin/Index
```

Função para Limpar o Slot:

Crie uma função que *resete* os dados do slot, assegurando que ele retorne a um estado vazio:

```
func clear_item():
    item = null
    item_icon.texture = null
    quantity_label.text = ""
    quantity_label.visible = false
```

Função para Definir o Slot:

Implemente uma função que atribua um novo item ao slot. Se o parâmetro `new_item` for `null`, o slot será limpo automaticamente:

```
func set_item(new_item: Item):
    if new_item:
        item = new_item.clone()
        item_icon.texture = item.texture # Define o ícone do item
        quantity_label.text = str(item.quantity) # Exibe a quantidade
        quantity_label.visible = item.quantity > 1 # Mostra a quantidade apenas se for maior
            que 1
    else:
        clear_item()
```

Inicialização na Função `_ready()`:

Para garantir que o `index_label` permaneça oculto, você pode tanto selecioná-lo na cena (clicando no ícone do olho) quanto escondê-lo via código:

```
func _ready() -> void:
    index_label.hide()
```



12.4.1 Implementando Drag and Drop

Para habilitar o **drag and drop** no slot, utilizaremos funções especiais do Godot:

- **_get_drag_data(position)**: Invocada quando o usuário inicia o arrasto, deve retornar os dados do item arrastado.
- **_can_drop_data(position, data)**: Chamada continuamente enquanto o item é arrastado, determinando se o drop é permitido.
- **_drop_data(position, data)**: Executada quando o item é solto; o parâmetro **data** contém os dados definidos anteriormente em **_get_drag_data()**.

Implementação da Função **_get_drag_data()**:

Esta função cria um **preview** do item enquanto ele é arrastado e retorna uma referência ao próprio slot:

```
func _get_drag_data(_position):
    if item:
        var preview = TextureRect.new()
        preview.texture = item_icon.texture           # Usa o ícone do item
        preview.custom_minimum_size = Vector2(32, 32) # Define o tamanho do preview
        preview.modulate.a = 0.8                     # Aplica transparência ao preview
        set_drag_preview(preview)

        dragged_item = item
        if is_in_group("inventory_slots"):
            clear_item()
        return self
    return null
```

Verificação para Drop:

A função **_can_drop_data()** garante que o objeto de destino seja um **ItemSlot**:

```
func _can_drop_data(_position, data):
    return data is ItemSlot
```



Implementação da Função `_drop_data()`:

Esta função adiciona o item arrastado ao slot de destino. Se o slot já estiver ocupado, os itens serão trocados:

```
func _drop_data(_position, data):
    var src_slot: ItemSlot = data
    var dst_slot: ItemSlot = self

    var transaction_success = false

    # Se o item está sendo arrastado dentro do inventário, realiza a TROCA
    if src_slot.is_in_group("inventory_slots") and dst_slot.is_in_group("inventory_slots"):
        transaction_success = true
        InventoryManager.swap_items(src_slot.slot_index, dst_slot.slot_index)

    # Caso a transação falhe, restaura o item no slot de origem
    if not transaction_success and src_slot.dragged_item:
        src_slot.set_item(src_slot.dragged_item)

    # Limpa a referência temporária
    src_slot.dragged_item = null
```

Tratamento de Falhas no Drop:

Para evitar a perda do item caso ele seja solto em um local inválido, utilize a função `_notification(what: int) -> void`. Se a notificação for `NOTIFICATION_DRAG_END` e o drop não for bem-sucedido, o item é restaurado ao slot original:

```
# Detecta o fim do arrasto
func _notification(what: int) -> void:
    if what == NOTIFICATION_DRAG_END and not get_viewport().gui_is_drag_successful():
        # Drop falhou! Restaura o item no slot original
        if dragged_item:
            set_item(dragged_item)
            dragged_item = null
```



12.5 Criando um Item

Nesta seção, vamos criar um **Item** a partir do script **item.gd**. Siga os passos abaixo:

1. **Crie a Pasta:** Em **res://resources**, crie uma pasta denominada **items**.
2. **Crie o Recurso:** Dentro da pasta **items**, crie um novo recurso. Ao ser solicitado, selecione a classe definida em **item.gd**.
3. **Nomeie o Arquivo:** Salve o recurso com o nome **enxada.tres**.
4. **Configure as Propriedades:** No menu lateral, em *Propriedades*, você verá todos os atributos marcados com **@export** na classe **Item**. Edite-os conforme indicado.

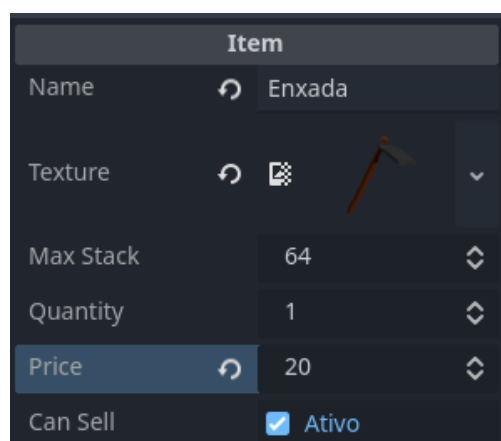


Figura 58: Propriedades do Item



12.6 Ajustando o Layout do Slot

Nesta etapa, atualizaremos o layout dos **Slots** para que a interface fique mais intuitiva.

1. Atualizando o Script `item_slot.gd`:

No script `item_slot.gd`, modifique a função `_ready()` para:

```
func _ready() -> void:
    index_label.hide()
    var temp_item: Item = load("res://resources/items/enxada.tres")
    set_item(temp_item)
```

Após executar a cena, você deverá visualizar um slot exibindo uma **enxada**. Teste as funcionalidades de arrastar e interagir com o item.

2. Atualizando o Script `main_menu.gd`:

Altere a função `_ready()` da cena `main_menu.gd` conforme o código abaixo:

```
func _ready() -> void:
    # index_label.hide()
    var temp_item: Item = load("res://resources/items/enxada.tres")
    temp_item.quantity = 2
    set_item(temp_item)
```

Com essa modificação, a cena exibirá o número de enxadas conforme ilustrado:



Figura 59: Quantidade de Enxadas



3. Posicionamento do Indicador de Quantidade:

Desejamos que o número que representa a quantidade de um item fique no **canto inferior direito**. Para isso:

- Edite o nó **ItemQuantity** e insira um valor temporário na propriedade **texto** para visualização durante a edição.
- Configure a predefinição de âncoras para **Inferior Direita**.
- Nas propriedades, ajuste:
 - **Horizontal Alignment** para **Right**;
 - **Vertical Alignment** para **Bottom**.
- Se necessário, altere o **Font Size** em **Control** → **Themes Overrides** → **Font Sizes**.

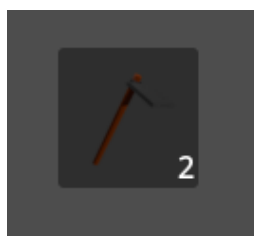


Figura 60: Layout Ajustado: Quantidade de Enxadas

4. Ajustando o Index:

O **Index** será exibido na **hotbar** para identificar o item selecionado e a tecla correspondente. Para posicioná-lo no **canto superior esquerdo**:

- Desoculte o **Index** e insira um valor temporário na propriedade **texto** para facilitar a visualização durante a edição.
- Defina o **Container Sizing** com:
 - **Alinhamento Horizontal:** Reduzir ao Início;
 - **Alinhamento Vertical:** Reduzir ao Início.
- Ajuste o **Font Size** para **12px**.



Figura 61: Ajuste do **Index** na Hotbar



5. Restaurando o Estado Original:

Após os testes e ajustes, remova os textos temporários das labels, oculte o **Index** e atualize a função `_ready()` para:

```
func _ready() -> void:  
    index_label.hide()
```



12.7 Ajustando Layout do Inventário

12.7.1 Ajustando o Layout

Antes de voltarmos para a cena `inventory.tscn`, realize os ajustes iniciais na cena `item_slot.tscn`. Configure o nó `ItemSlot` em **Container Sizing** para que tanto o **Alinhamento Horizontal** quanto o **Alinhamento Vertical** estejam definidos como *Expandir*. Lembre de configurar Slot Scene para referenciar a cena `item_slot.tscn`. Ao executar a cena `inventory.tscn`, o layout exibido será semelhante ao ilustrado na Figura 62.

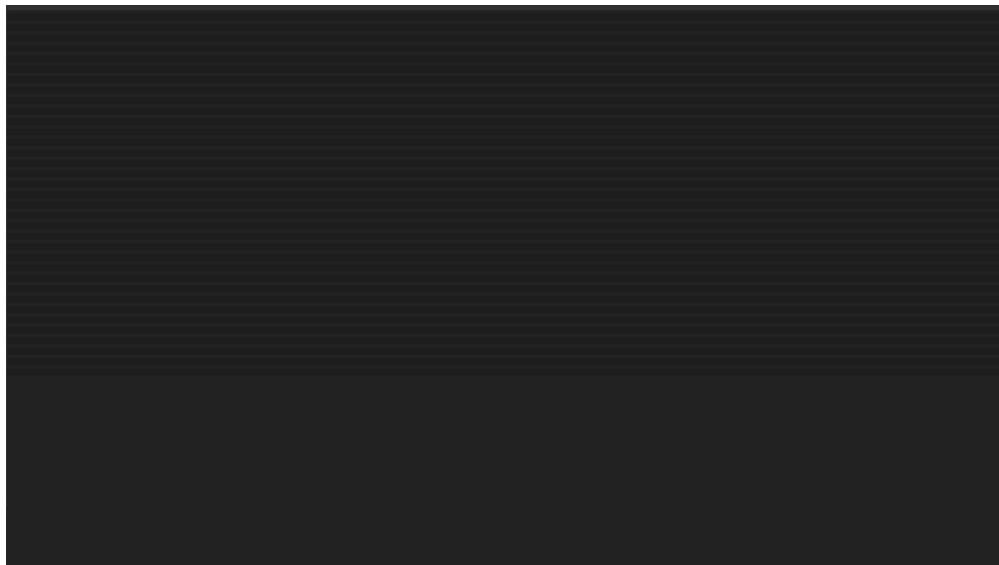


Figura 62: Layout Inicial do Inventário

Isso ocorre porque ainda não ajustamos a propriedade das colunas do **GridContainer**. Para resolver esse problema, edite o script `inventory.gd` e insira o seguinte código:

```
func _ready():
    inventory_grid.columns = InventoryManager.COLUMNS
    hotbar_grid.columns = InventoryManager.COLUMNS
    initialize_ui()
    InventoryManager.inventory_updated.connect(refresh_ui)
```



Além disso, selecione os nós **InventoryGrid** e **HotbarGrid** e ajuste o **Container Sizing** definindo o **Alinhamento Vertical** como *Expandir*. Ao executar a cena, o layout será semelhante ao apresentado na Figura 63.

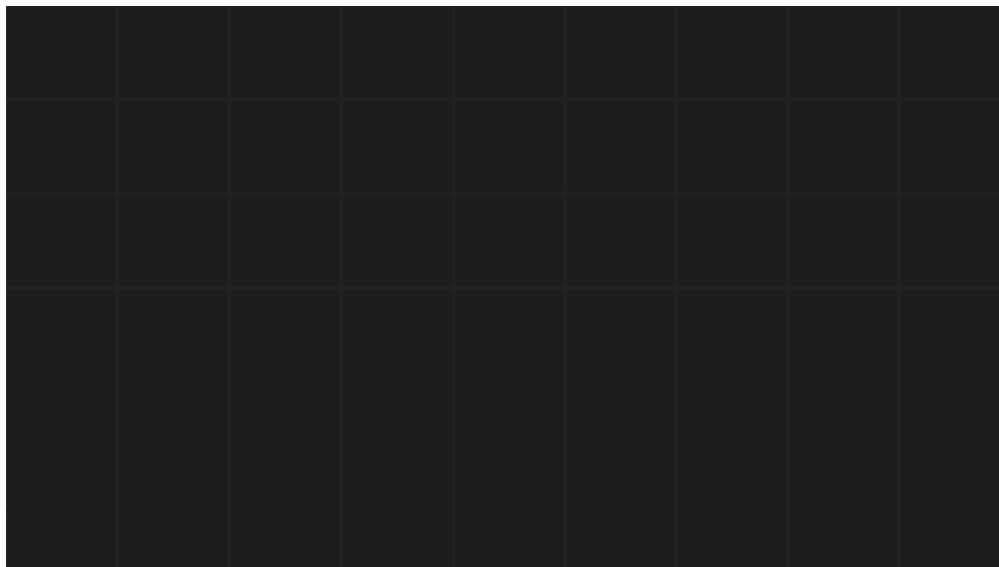


Figura 63: Layout do Inventário - Etapa 2

Se ainda não o fez, adicione as margens necessárias:

- No nó **Margin** (pai do **Panel**), configure **60** unidades em todas as direções.
- No nó **Margin** (pai do **Vbox**), configure **30** unidades em todas as direções.

Após essa configuração, o layout deverá ser semelhante ao da Figura 64.

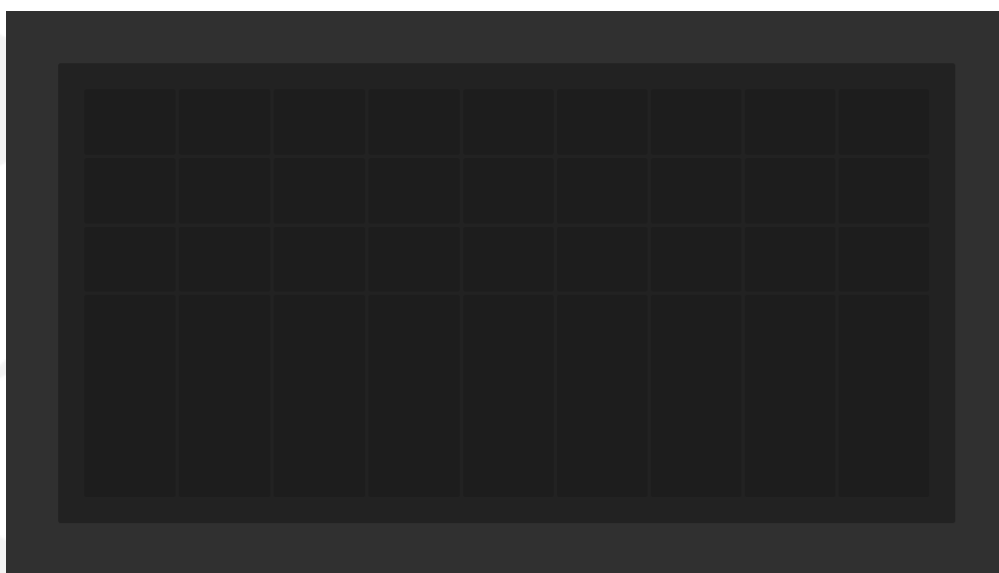


Figura 64: Layout do Inventário - Etapa 3



Para garantir que a última linha do inventário (a **hotbar**) possua tamanho uniforme em relação às demais, ajuste a propriedade **Stretch Ratio** do **InventoryGrid** em **Container Sizing** para **3**. Dessa forma, dos 4 “pedaços” disponíveis, 3 serão atribuídos ao **InventoryGrid** (3 linhas) e 1 à **HotbarGrid** (1 linha), mantendo a proporcionalidade. Adicionalmente, no nó **VBox** adicione um espaçamento (separation) de **20** para diferenciar visualmente os slots da hotbar dos demais slots do inventário. O resultado será semelhante ao da Figura 65.

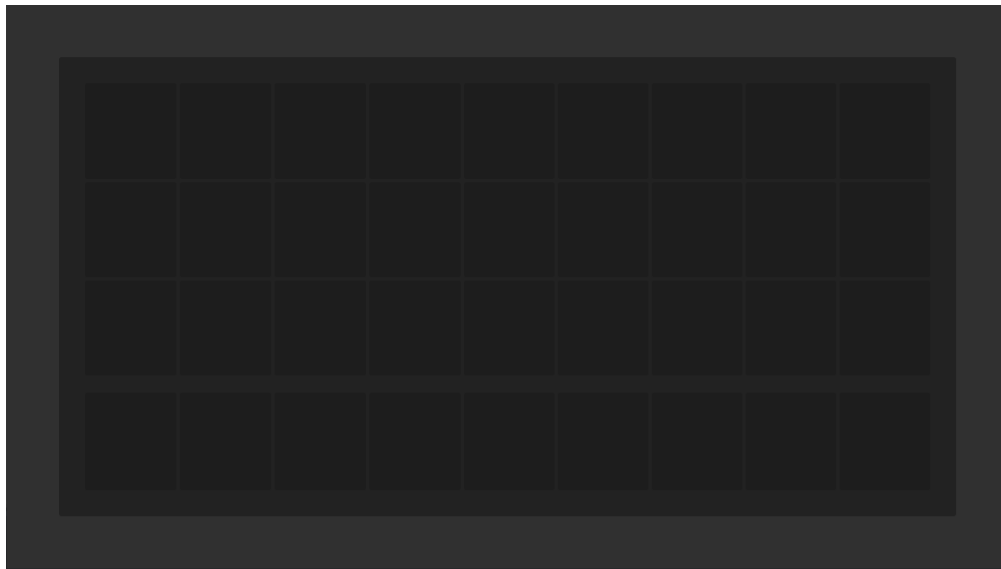


Figura 65: Layout do Inventário - Etapa 4

Para tornar o código mais flexível, podemos programaticamente ajustar o **Stretch Ratio** conforme o número de linhas, conforme mostrado abaixo:

```
func _ready():
    inventory_grid.columns = InventoryManager.COLUMNS
    hotbar_grid.columns = InventoryManager.COLUMNS
    inventory_grid.size_flags_stretch_ratio = InventoryManager.ROWS - 1
    hotbar_grid.size_flags_stretch_ratio = 1
    initialize_ui()
    InventoryManager.inventory_updated.connect(refresh_ui)
```



12.7.2 Criando Tema

Note que o espaçamento entre os slots pode estar muito reduzido. Para ajustar esse detalhe, adicione um **Tema** ao nó **Inventory**. No editor de temas, crie um novo tema do tipo **GridContainer**, habilite a opção **Mostrar Padrão** e, na aba de constantes (ícone representado por um **.c**), defina os parâmetros **h_separation** e **v_separation** para **10**. Assim, tanto o **InventoryGrid** quanto o **HotbarGrid** serão afetados, centralizando o controle do estilo do inventário. O layout atualizado será conforme ilustrado na Figura 66.

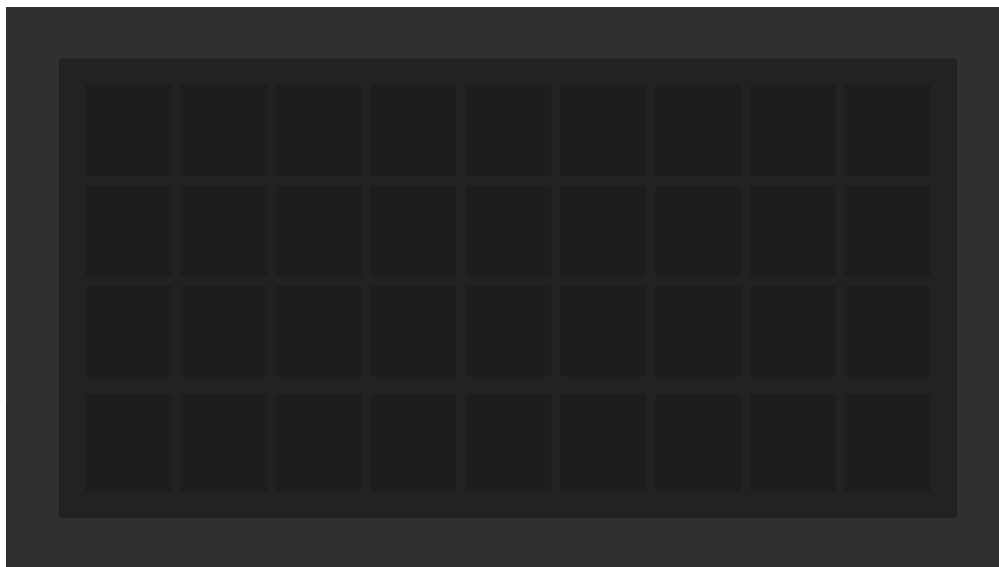


Figura 66: Layout do Inventário - Etapa 5

12.7.3 Testando o Inventário

Para testar o funcionamento do inventário, crie um novo item utilizando o mesmo procedimento que usamos para criar a *Enxada*. Desta vez, crie o item **Regador**:

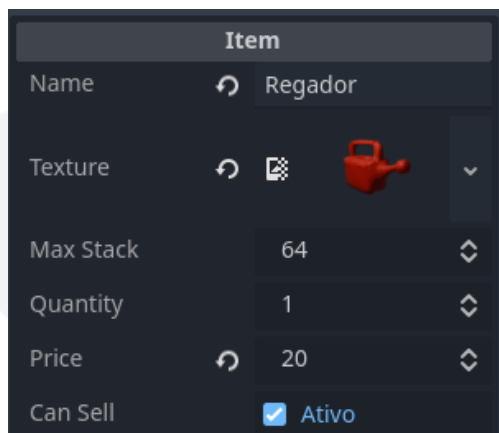


Figura 67: Regador



No script **inventory.gd**, adicione o código abaixo para testar os itens:

```
func _ready():
    inventory_grid.columns = InventoryManager.COLUMNS
    hotbar_grid.columns = InventoryManager.COLUMNS
    inventory_grid.size_flags_stretch_ratio = InventoryManager.ROWS - 1
    hotbar_grid.size_flags_stretch_ratio = 1
    initialize_ui()
    InventoryManager.inventory_updated.connect(refresh_ui)
    var enxada: Item = load("res://resources/items/enxada.tres")
    InventoryManager.add_item_to_inventory(enxada)
    var regador: Item = load("res://resources/items/regador.tres")
    InventoryManager.add_item_to_inventory(regador)
    # Adicionando uma segunda enxada para testar o empilhamento (stack)
    InventoryManager.add_item_to_inventory(enxada)
```

Ao executar a cena, o inventário será exibido conforme ilustrado na Figura 68.

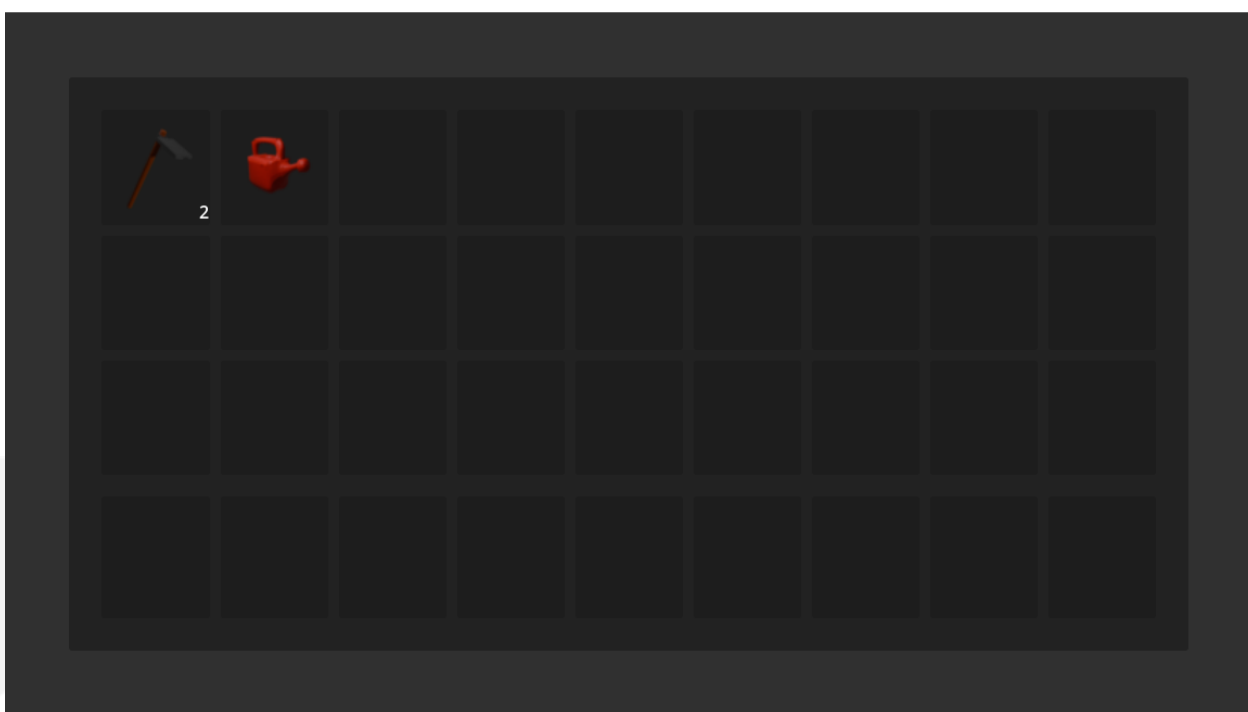


Figura 68: Teste dos Itens no Inventário



12.8 Integrando o Inventário ao Jogo

Com o sistema de inventário funcional implementado, é hora de integrá-lo à interface do jogo. Siga os passos descritos abaixo:

Instanciando a Cena de Inventário:

- Abra o arquivo `main_scene.tscn`.
- Instancie a cena `inventory.tscn` como filha do nó **UI**.
- Ocultar a cena: Configure a cena para iniciar oculta.

Referenciando a Cena no Script UI:

Edite o script associado ao nó **UI** para incluir as referências aos controles de **inventário** e **menu de pausa**:

```
@onready var inventory: Control = $Inventory
@onready var pause_menu: Control = $PauseMenu
```

Configurando o Controle no UIManager:

No arquivo `ui_manager.gd`, adicione a variável que indica se o inventário está aberto:

```
var is_inventory_open: bool = false
```

Em seguida, edite a função `initialize()` para garantir que tanto o inventário quanto o menu de pausa iniciem ocultos:

```
func initialize(ui_node: CanvasLayer):
    ui = ui_node
    Input.set_mouse_mode(Input.MOUSE_MODE_CAPTURED)
    ui.inventory.visible = false
    ui.pause_menu.visible = false
```

Alternando a Visibilidade do Inventário:

Implemente a função `toggle_inventory()` para alternar a visibilidade do inventário e ajustar o modo do mouse:

```
func toggle_inventory() -> void:
    is_inventory_open = !is_inventory_open
    ui.inventory.visible = is_inventory_open
    if is_inventory_open:
        Input.set_mouse_mode(Input.MOUSE_MODE_VISIBLE)
    else:
        Input.set_mouse_mode(Input.MOUSE_MODE_CAPTURED)
```



Atualizando o Script ui.gd:

Altere a função `_input()` para que, ao pressionar a tecla **ESC**, o inventário seja fechado antes de abrir o menu de pausa:

```
func _input(event):
    if event.is_action_pressed("ui_cancel"):
        if UIManager.is_inventory_open:
            UIManager.toggle_inventory()
        else:
            UIManager.toggle_pause()
```

Mapeando a Tecla I para Alternar o Inventário:

- **Configuração de Input:** Adicione um mapeamento para a tecla **I** com o nome `inventory_toggle`.
- **Script do Jogador:** No arquivo `player.gd`, atualize a função `_input()` para alternar o inventário ao pressionar **I**:

```
func _input(event):
    if event is InputEventMouseMotion and Input.get_mouse_mode() == Input.MOUSE_MODE_CAPTURED:
        rotation_horizontal -= event.relative.x * mouse_sensitivity.x
        rotation_vertical -= event.relative.y * mouse_sensitivity.y
        rotation_vertical = clamp(rotation_vertical, deg_to_rad(-90), deg_to_rad(90))

    if event.is_action_pressed("camera_switch"):
        cycle_camera_mode()

    if event.is_action_pressed("inventory_toggle"):
        UIManager.toggle_inventory()
```

Resultado Esperado:

- Ao rodar o jogo, o inventário deve ser aberto e fechado ao pressionar a tecla **I**.
- Se o inventário estiver aberto, pressionar **ESC** irá fechá-lo, e ao apertar **ESC** novamente, o menu de pausa será aberto.



13 Sistema de Dia e Noite

13.1 Introdução ao Sistema

Nos jogos que se passam em ambientes abertos, a representação da passagem do tempo é um dos responsáveis por tornar o mundo mais dinâmico e realista, aumentando a imersão do jogador. Em um jogo de fazenda, por exemplo, o jogador pode sentir a necessidade de planejar seu dia, sabendo que a noite se aproxima e que certas tarefas podem ser mais difíceis ou impossíveis de realizar. Então, foi elaborado um sistema de ciclo de dia e noite que tem como objetivo principal simular a transição entre diferentes períodos do dia, alterando a iluminação e as cores do céu com o passar do tempo.

Além disso, o tempo avançando dentro do jogo dá a impressão de que o mundo está vivo e em movimento. Sem essa mecânica, o ambiente pode parecer estático e repetitivo, o que pode reduzir o engajamento do jogador a longo prazo.

Dica

Novas mecânicas: poder sentir a passagem dos dias também pode servir como base para criar outras mecânicas de jogo, como eventos que ocorrem em determinados períodos. Por exemplo: lojas que fecham à noite, criaturas que surgem apenas de madrugada, etc.

Dica

Novas estratégias: alguns elementos do jogo podem se basear no ciclo de tempo para criar desafios ou oportunidades estratégicas. Por exemplo, determinadas culturas podem crescer melhor em horários específicos, certos animais podem sair apenas à noite, ou eventos especiais podem acontecer ao amanhecer ou anoitecer.

13.2 Elementos

O sistema de ciclo de dia e noite é composto por três elementos principais: **Sol** e **Lua**, **Céu** e **Tempo**. Cada um desses elementos se encontram detalhados nos tópicos seguintes. De forma resumida:

- **Sol (`DirectionalLight3D`):** Controla a iluminação diurna;
- **Lua (`DirectionalLight3D`):** Controla a iluminação noturna;
- **Céu (`SkyMaterial`):** Ajusta dinamicamente as cores do céu para simular o ciclo do dia;
- **Tempo (`cycle_duration`, `time_elapsed`):** Responsável pela iluminação noturna;

13.2.1 Sol e Lua

Na engine Godot, a **iluminação global** do ambiente é controlada por luzes direcionais (**`DirectionalLight3D`**). Esse tipo de luz simula uma fonte de luz distante e uniforme, como o Sol, iluminando todos os objetos do cenário na mesma direção.

No nosso sistema de dia e noite, utilizamos duas luzes direcionais:

- **Sol (`sun`):** Atua como a principal fonte de iluminação durante o dia;
- **Lua (`moon`):** Responsável pela iluminação noturna;



Essas luzes são rotacionadas ao longo do tempo para simular o movimento do sol e da lua no céu. Quando o sol começa a se pôr, sua intensidade diminui, enquanto a luz da lua começa a aumentar gradualmente, criando um efeito natural de transição entre dia e noite.

Com isso, torna-se possível representar o dia, conforme apresentado na Figura 69 e a noite, apresentada na Figura 70..

Observação

O uso de muitas luzes dinâmicas pode impactar a **performance do jogo**, especialmente em dispositivos menos potentes. Como temos apenas uma luz ativa por vez (Sol ou Lua), evitamos esse problema.

Observação

Se quiser adicionar sombras dinâmicas, é necessário ativá-las na configuração do **DirectionalLight3D**. Porém, sombras exigem processamento extra, então é importante testar o impacto no desempenho do jogo.

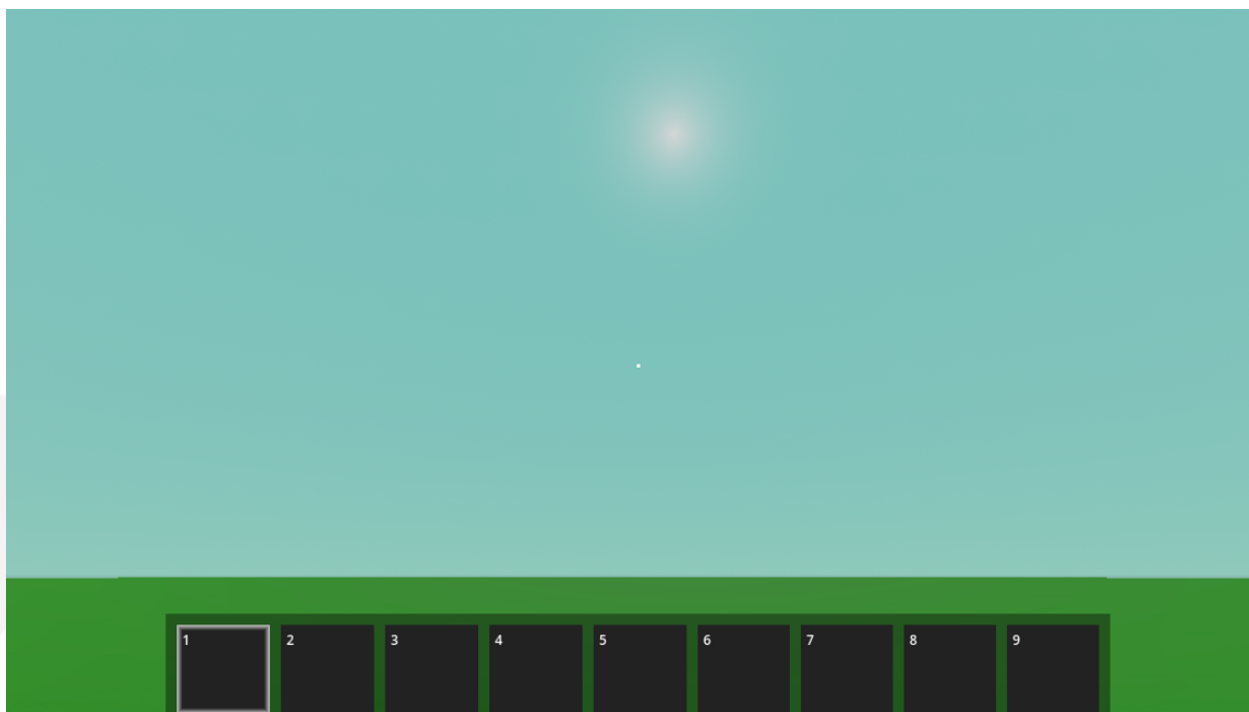


Figura 69: Representação do Dia no jogo.

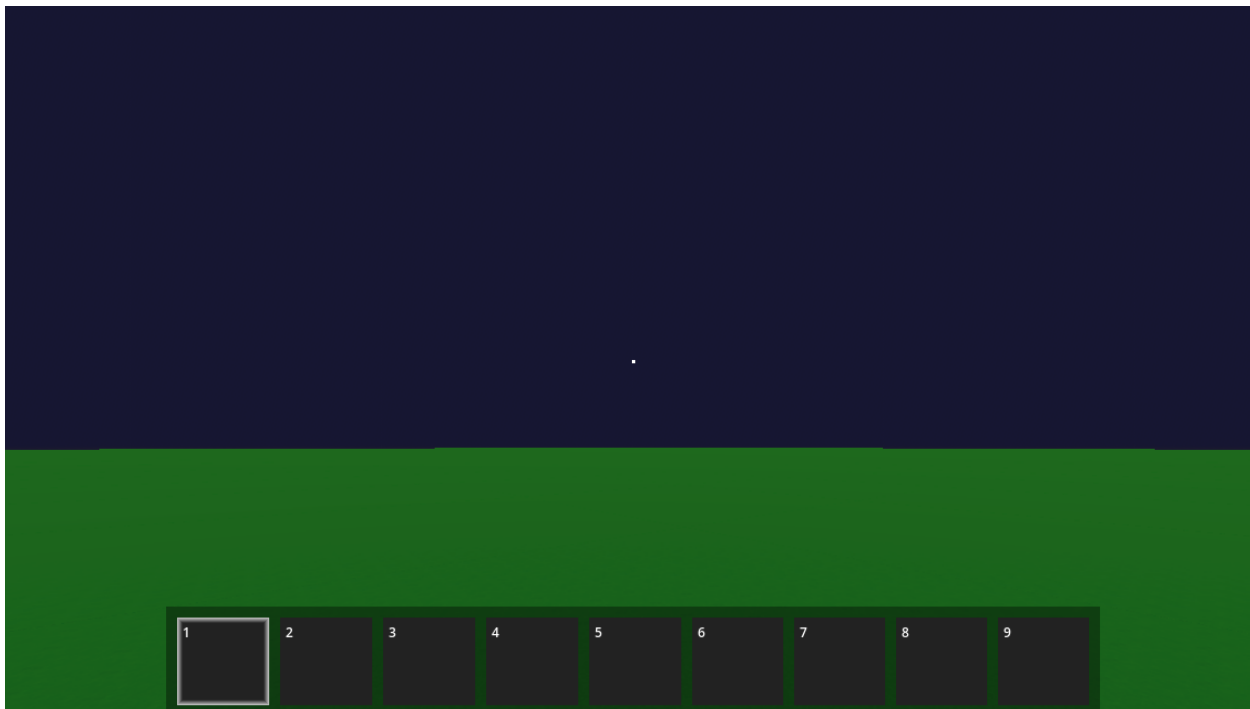


Figura 70: Representação da Noite no jogo.

13.2.2 Céu

Além da rotação do Sol e da Lua, outro aspecto importante para a ambientação do jogo é a mudança gradual das cores do céu.

O céu no Godot é controlado pelo **SkyMaterial**, que faz parte do **WorldEnvironment**. No nosso sistema, ajustamos as seguintes propriedades do material para criar transições suaves entre dia e noite:

- **sky_top_color**: Atua como a principal fonte de iluminação durante o dia;
- **sky_horizon_color**: Define a cor do horizonte, onde o céu encontra o solo. Essa cor muda bastante ao longo do dia, indo de tons azulados ao meio-dia para tons avermelhados no pôr do sol;
- **ground_horizon_color**: Define a cor do solo distante. Mesmo que o chão em si seja um objeto físico, a forma como ele se funde com o horizonte do céu ajuda a criar uma transição mais suave entre os elementos do cenário;

A transição entre cores acontece por meio da **função lerp()** (interpolação linear), que mistura as cores conforme o tempo avança. Isso cria um efeito de mudança gradual da iluminação e do céu.

O céu também pode ser configurado para usar uma textura personalizada em vez de um **SkyMaterial**. Se optar por uma textura, a transição de cores precisará ser feita de outra maneira.

Dica

Para jogos que precisam de um ciclo de tempo mais complexo, é possível adicionar efeitos como estrelas aparecendo à noite ou nuvens se movendo ao longo do dia.



13.2.3 Tempo

O último elemento fundamental do sistema de ciclo de dia e noite é o **controle do tempo**. O tempo dentro do jogo não segue o tempo real, mas sim uma escala definida pelo desenvolvedor.

Usamos duas variáveis principais para gerenciar o tempo:

- **cycle_duration**: Define quanto tempo (em segundos) dura um ciclo completo de dia e noite. Quanto menor esse valor, mais rápido o tempo passará no jogo;
- **time_elapsed**: Guarda o tempo decorrido dentro do ciclo atual. Essa variável é atualizada constantemente e usada para calcular o progresso do ciclo;

Com isso é possível definir a lógica do tempo e o seu funcionamento. A cada frame, adicionamos o valor de delta (que corresponde ao tempo desde o último frame) a **time_elapsed**. Quando **time_elapsed** ultrapassa **cycle_duration**, ele é resetado para 0, iniciando um novo ciclo. O progresso do ciclo (definido como **cycle_progress**) é calculado como uma fração (**time_elapsed** / **cycle_duration**), permitindo que ajustemos a iluminação e a rotação das luzes proporcionalmente.

Isso garante que o Sol e a Lua sempre sigam um movimento previsível e contínuo, mantendo a coerência da iluminação do jogo.

Observação

Se houver NPCs ou eventos baseados no horário do dia, eles devem estar sincronizados com o sistema de tempo para evitar inconsistências (por exemplo: lojas abrindo de manhã e fechando à noite).

Aviso

O valor de **cycle_duration** deve ser ajustado com cuidado. Se o ciclo for muito rápido, pode parecer artificial e confuso para o jogador. Se for muito lento, pode dar a sensação de um mundo parado.

13.3 Configuração do Script

Para que o sistema de ciclo de dia e noite funcione corretamente, precisamos definir algumas variáveis dentro do script **sun.gd**. Essas variáveis serão responsáveis por armazenar informações essenciais sobre o tempo, a iluminação e o céu.

No Godot, podemos utilizar variáveis exportadas (**@export**) para tornar certos parâmetros ajustáveis diretamente no Editor de Inspeção. Isso permite que possamos modificar facilmente valores como a duração do ciclo de tempo, as cores do céu e até mesmo ativar ou desativar o sistema de passagem do tempo. Além disso, utilizamos variáveis internas, que servem para armazenar o estado atual do sistema e controlar o funcionamento do ciclo de forma dinâmica.

As variáveis exportadas podem ser ajustadas diretamente no Editor do Godot, sem a necessidade de modificar o código manualmente. Isso torna o sistema mais flexível e facilita ajustes para equilibrar a jogabilidade.

Aqui estão as **variáveis exportadas** do script **sun.gd**:

1. **Variável para Controle do Tempo**: usada para definir se a passagem do tempo está ativada ou desativada. Caso *false*, o tempo para de avançar e o ciclo de dia e noite fica congelado no estado atual.



```
@export var enable_time_passage: bool
```

Se quisermos pausar a transição do tempo durante um menu ou cutscene, basta desativar essa opção no editor ou via código (**enable_time_passage = false**).

2. **Variável para Ambiente Global:** usada como referência ao ambiente global do jogo (**WorldEnvironment**). O **WorldEnvironment** é responsável por controlar efeitos como iluminação global, cores do céu e reflexos ambientais. É nele que alteramos dinamicamente as cores do céu durante a transição entre dia e noite.

```
@export var world_env: WorldEnvironment
```

O **WorldEnvironment** deve estar presente na cena e vinculado corretamente ao script `sun.gd`. Se não for definido, o código pode gerar erro ao tentar modificar as propriedades do céu.

3. **Variável para Duração do Ciclo:** usada para definir a duração total do ciclo de dia e noite (em segundos). Esse valor determina a velocidade com que o tempo passa no jogo.

```
@export var cycle_duration: float = 500
```

Se **cycle_duration = 500**, significa que um ciclo completo (do amanhecer até a próxima madrugada) leva 500 segundos no tempo real do jogo. Para ciclos mais rápidos, podemos diminuir esse valor, como **cycle_duration = 300**. Para ciclos mais longos e realistas, podemos aumentar, como **cycle_duration = 1200**, por exemplo, para 20 minutos.

4. **Variáveis para Cores do Céu:** variáveis para controlar as cores do céu durante diferentes períodos do dia. Essas cores são usadas na interpolação (função **lerp()**) para criar uma transição suave entre noite e dia..

```
@export var night_color: Color = Color(0.1, 0.1, 0.2)
@export var day_color: Color = Color(0.6, 0.96, 0.9)
@export var day_horizon_color: Color = Color(0.7, 1.0, 0.9)
```

A **night_color** representa a cor predominante no céu durante a noite (tons escuros de azul). Já a **day_color** é a cor predominante durante o dia (tons claros de azul). Por fim, a **day_horizon_color** representa a cor do horizonte ao longo do dia (pode ter um tom levemente amarelado para simular luz solar).

5. **Variáveis para Luzes do Sol e da Lua:** são referências às fontes de luz que simulam o Sol e a Lua no jogo. O script ajusta a rotação dessas luzes conforme o tempo avança, criando a ilusão do movimento do Sol e da Lua no céu.

```
@export var sun: DirectionalLight3D
@export var moon: DirectionalLight3D
```



É importante se certificar de que a luz do Sol e a luz da Lua estejam corretamente configuradas na cena e conectadas a essas variáveis. Se elas não estiverem atribuídas, o código pode não funcionar corretamente.

6. **Variável para Nó Âncora para Rotação:** usada para define o nó que será rotacionado para simular o movimento do Sol e da Lua. Esse nó serve como ponto central de rotação para as luzes, garantindo que ambas girem ao redor do mundo do jogo de maneira sincronizada.

```
@export var anchor: Node3D
```

O nó anchor geralmente é um **Node3D** posicionado no centro da cena. O Sol e a Lua devem ser filhos desse nó para que girem corretamente ao redor do mundo.

Dica

Podemos personalizar as cores **night_color**, **day_color** e **day_horizon_color** no editor para criar diferentes atmosferas. Por exemplo, para um estilo mais fantasioso, podemos adicionar tons roxos ou avermelhados ao céu da noite.

Além das variáveis exportadas, usamos uma **variável interna** para controlar o avanço do tempo no ciclo, ou seja, para termos um controle do tempo passado.

```
var time_elapsed: float = 0.0
```

Essa variável tem como finalidade armazenar o tempo decorrido dentro do ciclo atual. Ela é atualizada constantemente no **__process(delta)**, garantindo que o Sol e a Lua se movam suavemente.

O seu funcionamento se dá da seguinte forma: no início do jogo, **time_elapsed** começa em 0.0. A cada frame, o valor de delta (tempo decorrido desde o último frame) é adicionado a **time_elapsed**. Quando ele o valor de **cycle_duration**, ele é resetado para 0.0, iniciando um novo ciclo de dia e noite.

13.4 Funcionamento do Código

Agora que já configuramos as variáveis essenciais no script **sun.gd**, vamos entender como o código processa o tempo, movimenta o Sol e a Lua e altera dinamicamente a iluminação do céu.

O funcionamento do sistema se baseia na função **__process(delta)**, que é chamada automaticamente pelo Godot a cada frame. Essa função é responsável por avançar o tempo dentro do jogo e atualizar as mudanças visuais necessárias.

13.4.1 Processamento do Tempo

A passagem do tempo no jogo é controlada por um sistema que acumula o tempo decorrido e calcula o progresso dentro de um ciclo de dia e noite. Isso é feito dentro da função **__process(delta)**, que roda constantemente.

A lógica do tempo funciona da seguinte forma: A cada frame, a variável **time_elapsed** acumula o valor de delta, que representa o tempo decorrido desde o último frame. O código então calcula o progresso do ciclo dividindo **time_elapsed** pela **cycle_duration**. Se o progresso atingir 100%, ou seja, um ciclo



completo passou, **time_elapsed** é resetado para 0, iniciando um novo ciclo.

```
func _process(delta):
    if !enable_time_passage:
        return

    time_elapsed += delta
    var cycle_progress = (time_elapsed / cycle_duration)

    if cycle_progress >= 1:
        time_elapsed = 0
        cycle_progress = 0
```

Se **enable_time_passage** estiver desativado (**false**), o tempo não avança e o ciclo de dia e noite fica congelado.

time_elapsed acumula tempo conforme o jogo roda.

cycle_progress é um valor entre 0.0 e 1.0, representando o quanto do ciclo já passou. Quando **cycle_progress** atinge 1.0, ou seja, um ciclo completo se passou, o tempo é reiniciado para começar um novo ciclo.

Exemplo: Se **cycle_duration = 500** segundos, isso significa que:

- Após 250 segundos, **cycle_progress** será 0.5 (metade do ciclo, ou seja, meio-dia ou meia-noite).
- Após 500 segundos, **cycle_progress** será 1.0 e o ciclo reinicia.

Essa lógica garante que o sistema de dia e noite funcione continuamente sem interrupções!

13.4.2 Rotação do Sol e da Lua

Para criar a ilusão de que o Sol e a Lua estão se movendo pelo céu, o código rotaciona o nó **anchor**, que controla a posição das luzes sun e moon.

O nó **anchor** recebe uma rotação gradual baseada no **cycle_progress**. Isso faz com que o Sol e a Lua girem ao redor do mundo, simulando um nascer e pôr do sol realista.

```
anchor.rotation_degrees.x = -cycle_progress * 360 * 2
```

No código acima, **cycle_progress * 360 * 2** converte o progresso do ciclo em um ângulo de rotação.

Multiplicar por 360 garante que o Sol e a Lua completem uma volta completa no céu e multiplicar por 2 estende o movimento para incluir tanto o Sol quanto a Lua.

O sinal negativo (-) inverte a direção da rotação para garantir um movimento natural.

Isso afeta no jogo pois, no início do ciclo (**cycle_progress = 0.0**), o Sol começa no horizonte.

Conforme o tempo avança, ele sobe no céu até atingir o ponto máximo (meio-dia).

Depois, ele começa a descer e, quando some no horizonte, a Lua começa a subir.

Quando o ciclo completa, ou seja, **cycle_progress = 1.0**, o Sol nasce novamente e a Lua desaparece.



Esse sistema cria um ciclo de iluminação realista, dando uma sensação natural de passagem do tempo.

13.4.3 Iluminação

Outro aspecto fundamental do sistema de dia e noite é a transição suave das cores do céu. O código ajusta as cores gradualmente para criar um efeito dinâmico e natural.

O código usa a função **lerp()** para interpolar as cores entre os valores de noite e dia com base no **intensity**, que varia entre 0.0 (noite) e 1.0 (dia).

```
var intensity = clamp(sin(cycle_progress * 4 * PI), 0.0, 1.0)

if world_env:
    var sky_material = world_env.environment.sky.get_material()
    if sky_material:
        sky_material.sky_top_color = night_color.lerp(day_color, intensity)
        sky_material.sky_horizon_color = night_color.lerp(day_horizon_color, intensity)
        sky_material.ground_horizon_color = night_color.lerp(day_horizon_color, intensity)
```

No código acima, **sin(cycle_progress * 4 * PI)** calcula uma curva suave de transição entre noite e dia. Já o **clamp(..., 0.0, 1.0)** garante que o valor de **intensity** sempre fique entre 0.0 (mínimo) e 1.0 (máximo). As cores do céu são interpoladas (**lerp()**) entre as cores da noite e do dia, criando uma transição gradual.

Dessa forma, durante a noite, **intensity** está próximo de 0.0, então o céu fica escuro (**night_color**). Conforme o Sol nasce, **intensity** aumenta gradualmente, alterando a cor para **day_color**. No meio-dia, **intensity** atinge 1.0, fazendo o céu ficar completamente claro. Conforme o Sol se põe, **intensity** diminui e o céu escurece novamente.

Isso cria um efeito de transição suave, evitando mudanças bruscas na iluminação e garantindo um visual imersivo e realista.

13.4.4 Configurando no Godot

13.4.5 Configurando no Godot

No editor da Godot, siga os seguintes passos para configurar corretamente o sistema de ciclo de dia e noite:

- Criar o ambiente do céu Adicione um nó do tipo **WorldEnvironment** à cena principal. No **Inspector**, vá até a propriedade **Environment** e crie um novo **SkyMaterial**. Configure as cores do céu para refletirem os diferentes momentos do dia, ajustando os valores de cor e iluminação conforme necessário.
- Criar a estrutura de rotação do Sol e da Lua Adicione um nó do tipo **Node3D** e nomeie-o como **CicloSolar**. Esse nó servirá como um pivô para rotacionar o Sol e a Lua. Dentro do nó **CicloSolar**, adicione dois nós do tipo **DirectionalLight3D** e nomeie-os como **Sol** e **Lua**. Ajuste a rotação inicial do Sol para que ele comece no horizonte, e faça o mesmo com a Lua na posição oposta.
- Configurar os parâmetros das luzes Selecione o nó **Sol** e, no **Inspector**, ajuste a intensidade da luz para um valor adequado para o dia. Ative sombras dinâmicas caso deseje realismo adicional. Selecione o nó **Lua** e configure uma intensidade mais fraca e um tom azulado para simular a luz noturna.
- Adicionar o script de controle Adicione o script desenvolvido na cena e coloque os valores das variáveis necessárias.



- Testar e ajustar os valores Execute a cena e observe se a transição entre dia e noite está fluida. Ajuste a duração do ciclo, a intensidade da luz e as cores do céu até alcançar o efeito desejado.



14 Mecânicas de Cultivo - Planta

O sistema de plantação seguirá as seguintes etapas: primeiro, o jogador precisará **preparar o terreno** para o plantio. Após isso, será necessário **fertilizar o solo**, que então poderá receber a semente. Além disso, será possível **regar a planta** para acelerar seu crescimento. As mecânicas de **regar e fertilizar** não serão obrigatórias, mas permitirão que as plantas cresçam mais rapidamente.

14.1 Configurando as Entradas de Controle

Para implementar as mecânicas de cultivo, primeiro precisamos configurar as entradas do sistema. Para isso, siga os passos abaixo:

1. Acesse o menu **Projeto** (Figura 91).

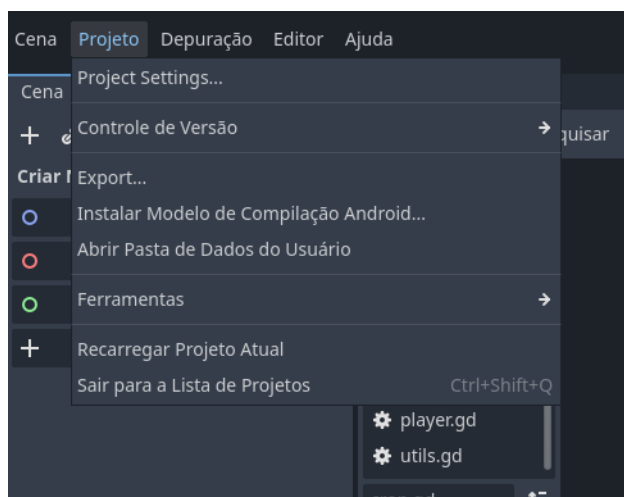


Figura 71: Acessando as configurações do projeto

2. Dentro do menu, clique em **Configurações do Projeto** (*Project Settings*).
3. Navegue até a aba **Mapa de Entrada** (*Input Map*), conforme mostrado na Figura 72.

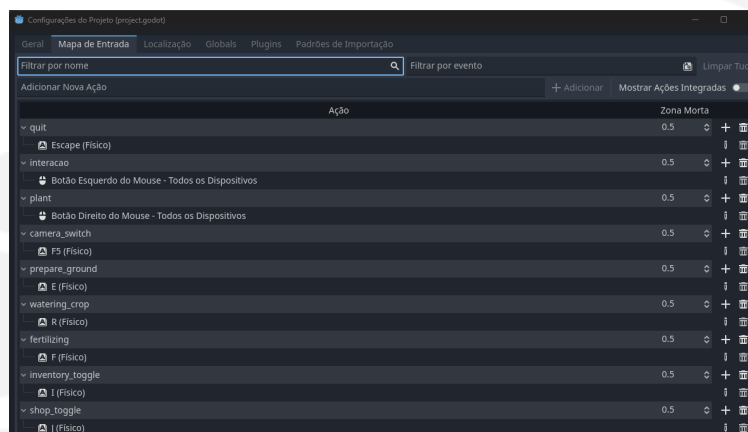


Figura 72: Tela do Mapa de Entrada

4. Adicione os seguintes comandos ao Mapa de Entrada:



- **interacao** - Botão esquerdo do mouse
- **plant** - Botão direito do mouse
- **fertilizing** - Tecla F
- **watering_crop** - Tecla W
- **prepare_ground** - Tecla E

Após essa configuração, o Mapa de Entrada estará pronto para ser utilizado nas mecânicas de cultivo.

14.2 Cena do Milho

Para iniciar a implementação do sistema de cultivo, precisamos primeiro de uma planta. Começaremos criando uma cena chamada **corn.tscn**. Essa cena deve ser do tipo **Nó 3D**. Para isso, siga os passos mostrados na Figura 73.

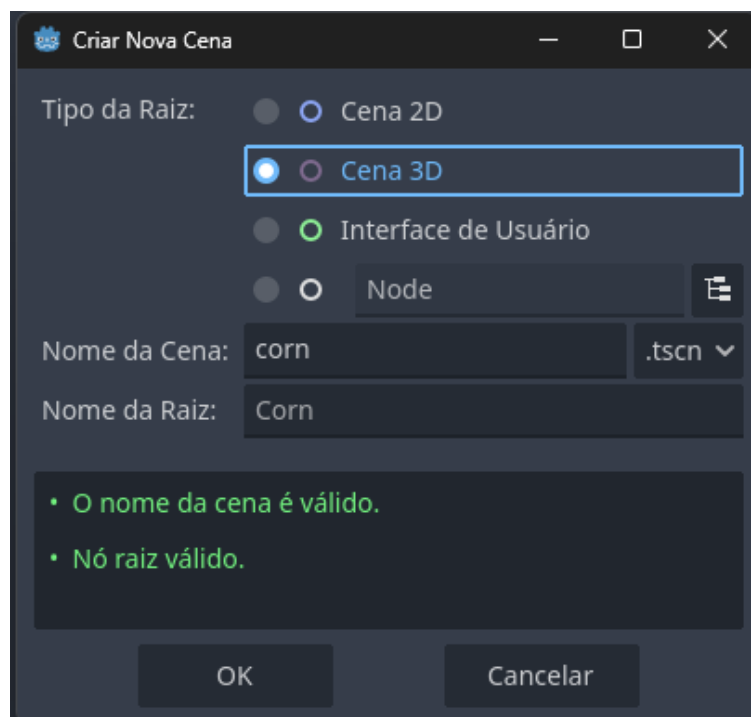


Figura 73: Criação da cena **corn.tscn**

Após criar a cena, siga os passos abaixo:

1. Delete o **Nó Raiz** padrão.
2. Clique em **Outro Nó** e crie um **StaticBody3D**.
3. Renomeie o nó criado para **Corn**.

14.2.1 O que é um StaticBody3D?

Na **Godot**, um **StaticBody3D** é um tipo de nó usado para objetos que não se movem, mas que precisam interagir fisicamente com outros elementos do jogo. Ele pode detectar colisões, mas não é afetado pela física, sendo ideal para representar elementos como solo, obstáculos e, no nosso caso, a planta do milho.



14.3 Adicionando os Componentes da Planta

Agora que temos a base do milho configurada, adicionaremos os elementos visuais e as colisões para representar seu crescimento. Siga os passos abaixo:

1. Adicione três **CollisionShape3D** como filhos do nó **Corn**.
2. Renomeie os nós adicionados para:
 - **Small** (representa o milho pequeno)
 - **Medium** (representa o milho médio)
 - **Large** (representa o milho grande)

A Figura 74 mostra a estrutura hierárquica esperada da cena.

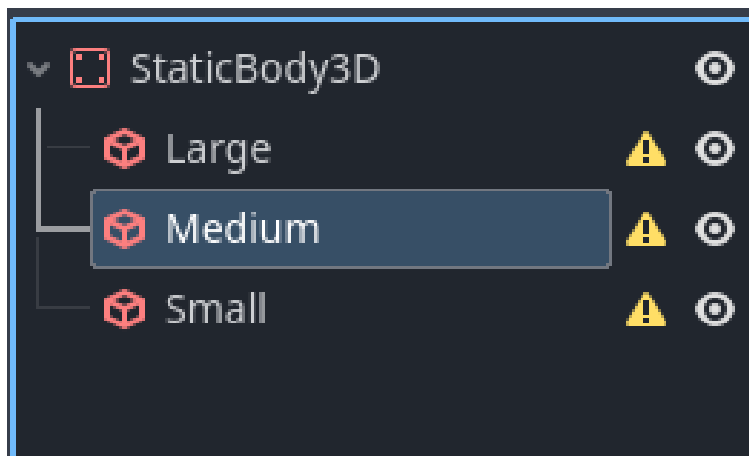


Figura 74: Hierarquia da cena do milho com os nós criados

Agora precisamos adicionar os modelos 3D para representar cada estágio de crescimento do milho:

1. Acesse a pasta **assets/modelos**.
2. Localize os arquivos **milho-baixo**, **milho-medio** e **milho-alto**.
3. Arraste cada um dos modelos para seus respectivos nós:
 - **milho-baixo** → Nó **Small**
 - **milho-medio** → Nó **Medium**
 - **milho-alto** → Nó **Large**

O resultado final deve se parecer com a Figura 75.

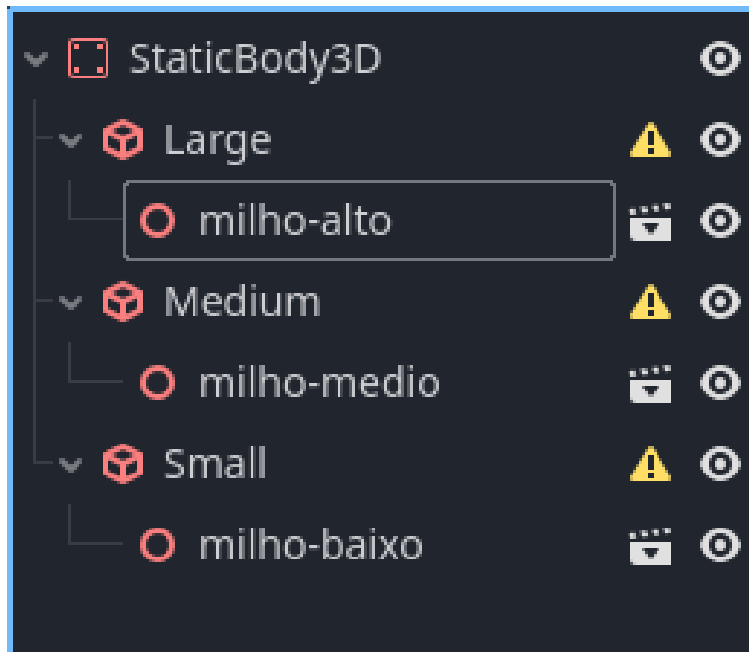
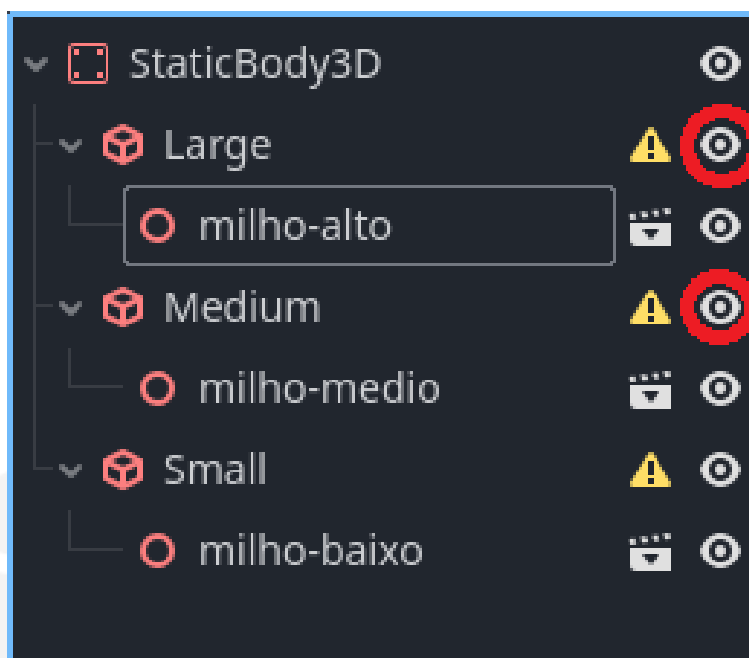


Figura 75: Modelos 3D do milho adicionados à cena

Agora devemos desabilitar a visibilidade dos nós **Large** e **Medium**, para que apenas o milho pequeno seja visível inicialmente. Siga o exemplo da Figura 76.

Figura 76: Desativando a visibilidade dos nós **Large** e **Medium**

14.4 Ajustando as Colisões

Agora precisamos configurar as colisões para que cada estágio da planta tenha um **CollisionShape3D** adequado.



1. Acesse os nós **Small**, **Medium** e **Large**.
2. Adicione um **CollisionShape3D** em cada um deles.
3. Defina o formato da colisão como uma **Cápsula**, conforme a Figura 77.

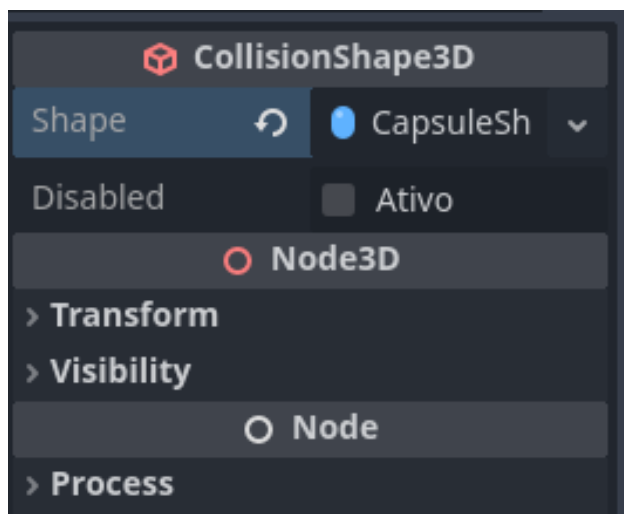


Figura 77: Selecionando o formato de colisão "Cápsula"

Agora vamos ajustar os colisores para que eles se alinhem corretamente ao modelo 3D:

14.4.1 Ajuste do Colisor do Small

1. Selecione o nó **Small**.
2. Reduza o tamanho do colisor até que ele se ajuste ao modelo, como mostrado na Figura 78.

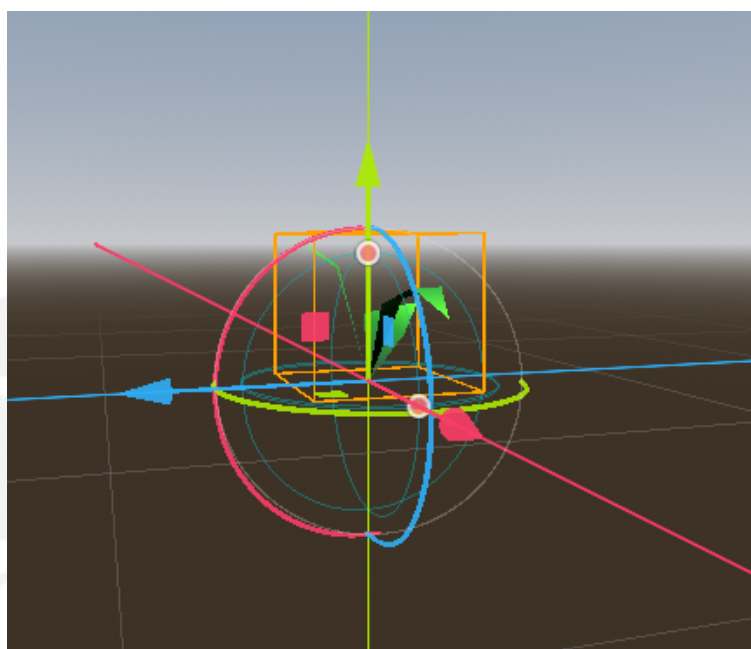


Figura 78: Ajuste do colisor do milho pequeno (**Small**)



14.4.2 Ajuste do Colisor do Medium

1. Altere a visibilidade para que apenas o Nó **Medium** seja visto.
2. Selecione o nó **Medium**.
3. Primeiro, levante o colisor (Figura 79).
4. Depois, selecione o modelo 3D **milho-medio** e abaixe-o (Figura 80).
5. Afine o colisor até obter o seguinte resultado (Figura 81).

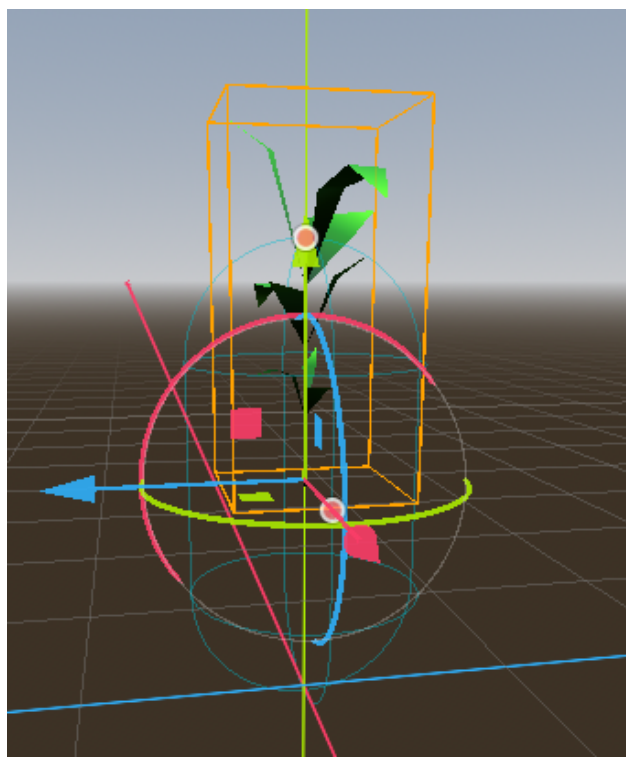
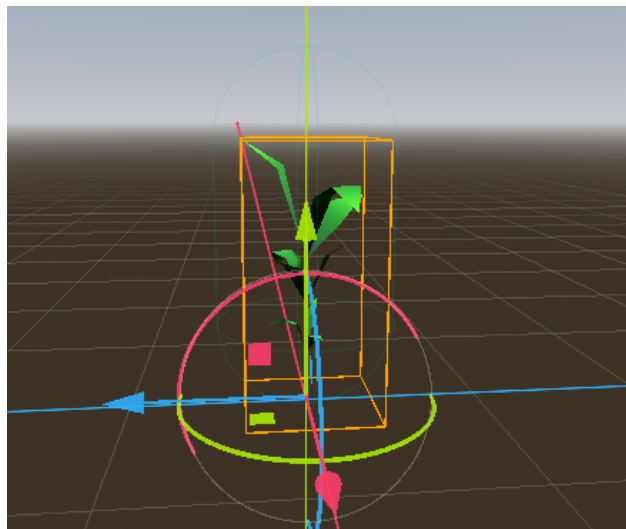
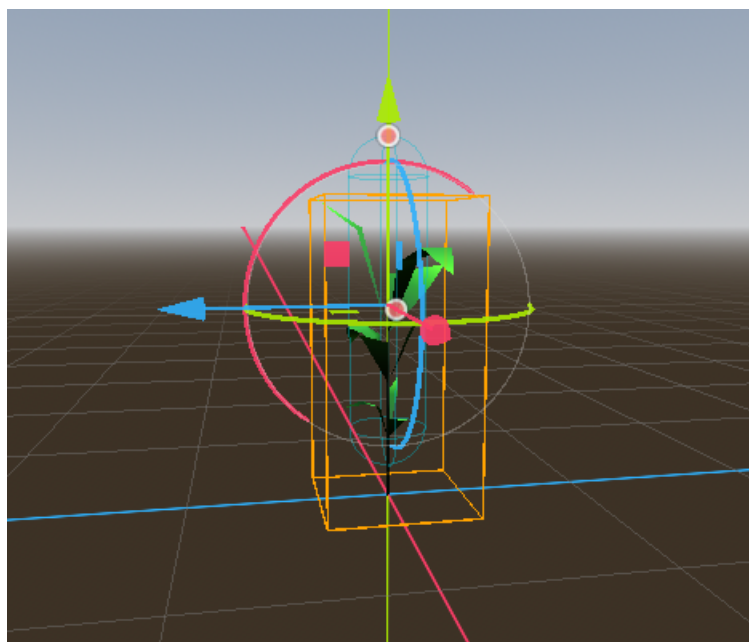
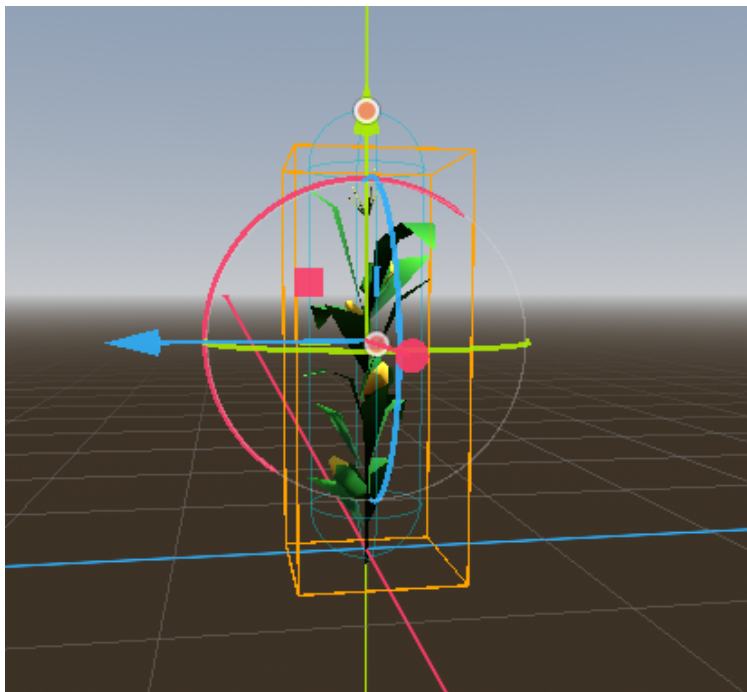


Figura 79: Levantando o colisor do **Medium**

Figura 80: Abaixando o modelo 3D do **Medium**Figura 81: Ajuste final do colisor do **Medium**

14.4.3 Ajuste do Colisor do Large

Agora repetimos o mesmo processo para o **Large**, ajustando o colisor e posicionando corretamente o modelo 3D. O resultado esperado pode ser visto na Figura 82.

Figura 82: Ajuste final do colisor do **Large**

Com isso, temos nossa cena de milho totalmente configurada e pronta para ser utilizada no sistema de cultivo do jogo.

14.4.4 Scripts de Plantas

Antes de iniciar o script específico do milho, precisamos criar um **script geral** para que seja mais simples adicionar novos tipos de plantas no futuro. Para isso, siga os passos abaixo:

1. Crie um script chamado **CropTypes.gd**.
2. Defina seu tipo como **Node**.
3. Insira o seguinte código:

```
extends Node
class_name CropTypes

# ENUM PARA TODOS OS TIPOS DE PLANTAS
enum CROP_TYPE {CORN, TOMATO, OUTRO = -1}
```

Esse código cria a classe **CropTypes** e define um **enum** chamado **CROP_TYPE**, que lista todos os tipos possíveis de plantas. Neste momento, estamos utilizando apenas o **milho** (**CORN**) e o **tomate** (**TOMATO**), mas outras plantas podem ser adicionadas no futuro.

14.5 Criando o Script do Milho

Agora que temos a classe base, podemos criar o script específico do milho. Para isso:

1. Crie um script e conecte-o ao **Nó Raiz** da cena **corn.tscn**.



2. Nomeie esse script como **crop.gd**.
3. Começaremos a programação do milho a partir desse script.

14.5.1 Definindo as Variáveis da Planta

Antes de iniciar a implementação do crescimento da planta, precisamos definir quais variáveis serão essenciais para o seu funcionamento. Cada planta deverá armazenar as seguintes informações:

- **Tipo da planta:** Pode ser Milho (**CORN**) ou Tomate (**TOMATO**).
- **Quantidade colhida:** Número de itens adicionados ao inventário ao colher a planta.
- **Tempo de crescimento:** Quantidade de tempo necessária para que a planta atinja o estágio final.
- **Estágio de crescimento atual:** Valor que indica quanto tempo falta para a planta crescer completamente.
- **Estado de rega:** Indica se a planta foi regada (ajuda a acelerar o crescimento).
- **Estado de colheita:** Determina se a planta já pode ser colhida.

A implementação inicial do script **Crop.gd** ficará da seguinte forma:

```
extends StaticBody3D

@export var crop_type: CropTypes.CROP_TYPE # Tipo de planta a ser usada
@export var harvest_amount = 1 # Quantidade que será coletada

# Referências aos modelos 3D da planta
@onready var large = $Large
@onready var medium = $Medium
@onready var small = $Small

# Crescimento da planta: valor atual e tempo total necessário
var growth_value = 0.0
var growth_time = 30

# Estado da planta
var watered = false # Indica se foi regada
var harvestable = false # Indica se pode ser colhida
```

Cada estágio da planta é representado por um modelo 3D diferente, e precisaremos ajustar a visibilidade conforme o crescimento ocorre.

14.5.2 Funções Básicas da Planta

Agora, adicionaremos as funções básicas do nosso sistema de cultivo. Primeiramente, criamos métodos para obter o tipo da planta e a quantidade colhida ao final do crescimento:

```
func get_harvest_amount():
    return harvest_amount # Retorna a quantidade adicionada ao inventário

func get_crop_type():
    return crop_type # Retorna o tipo da planta
```



A seguir, implementamos a interação com a planta. Se ela estiver pronta para ser colhida, será removida da cena:

```
func interact():
    if harvestable: # Destroi a instância após a colheita
        queue_free()
```

14.5.3 Atualizando a Interface do Jogador

Para que o jogador saiba quando pode interagir com a planta, criamos um método que atualiza a interface e exibe uma mensagem com as ações disponíveis:

```
func update_prompt_text(): # Atualiza a interface do jogo
    if harvestable:
        return "Click MOUSE 2 to Harvest"
    else:
        if !watered:
            return "Press R to Water"
        return ""
```

14.5.4 Implementando o Crescimento da Planta

Agora implementamos a lógica de crescimento da planta. O código a seguir faz com que a variável **growth_value** aumente gradualmente até atingir 1.0, momento em que a planta estará pronta para ser colhida. Durante esse processo, a visibilidade dos modelos 3D é ajustada conforme a planta cresce.

```
# Processa o crescimento da planta
func _process(delta):
    if growth_value < 1.0: # growth_value representa a porcentagem de crescimento
        growth_value += delta / growth_time

    # Ajustar visibilidade com base no crescimento
    small.visible = growth_value < 0.5
    medium.visible = growth_value >= 0.5 and growth_value < 1.0
    large.visible = growth_value >= 1.0

    # Ajustar colisores para corresponder ao estágio de crescimento
    small.set_physics_process(growth_value < 0.5)
    medium.set_physics_process(growth_value >= 0.5 and growth_value < 1.0)
    large.set_physics_process(growth_value >= 1.0)

    # Determinar se a planta esta pronta para colheita
    harvestable = growth_value >= 1.0
```

Esse código faz o seguinte:

- Incrementa o valor **growth_value** continuamente até alcançar 1.0.
- Modifica a visibilidade dos modelos 3D conforme o estágio de crescimento:
 - Pequeno (**Small**) visível quando **growth_value** < 0.5.
 - Médio (**Medium**) visível entre 0.5 e 1.0.



- Grande (**Large**) visível quando **growth_value** = 1.0.
- Ajusta os colisores de acordo com o tamanho da planta.
- Define a variável **harvestable** como **true** quando a planta atinge o crescimento máximo.

14.5.5 Código Completo do Crop.gd

O código completo do script **Crop.gd** agora é o seguinte:



```
extends StaticBody3D

@export var crop_type: CropTypes.CROP_TYPE
@export var harvest_amount = 1

@onready var large = $Large
@onready var medium = $Medium
@onready var small = $Small

var growth_value = 0.0
var growth_time = 30

var watered = false
var harvestable = false

func get_harvest_amount():
    return harvest_amount

func get_crop_type():
    return crop_type

func interact():
    if harvestable:
        queue_free()

func update_prompt_text():
    if harvestable:
        return "Click MOUSE 2 to Harvest"
    else:
        if !watered:
            return "Press R to Water"
        return ""

func _process(delta):
    if growth_value < 1.0:
        growth_value += delta / growth_time

    small.visible = growth_value < 0.5
    medium.visible = growth_value >= 0.5 and growth_value < 1.0
    large.visible = growth_value >= 1.0

    small.set_physics_process(growth_value < 0.5)
    medium.set_physics_process(growth_value >= 0.5 and growth_value < 1.0)
    large.set_physics_process(growth_value >= 1.0)

    harvestable = growth_value >= 1.0
```

Após finalizado o script selecione o Nó raiz do milho, e vá ao inspetor, aonde esta escrito Crop Type selecione Corn.

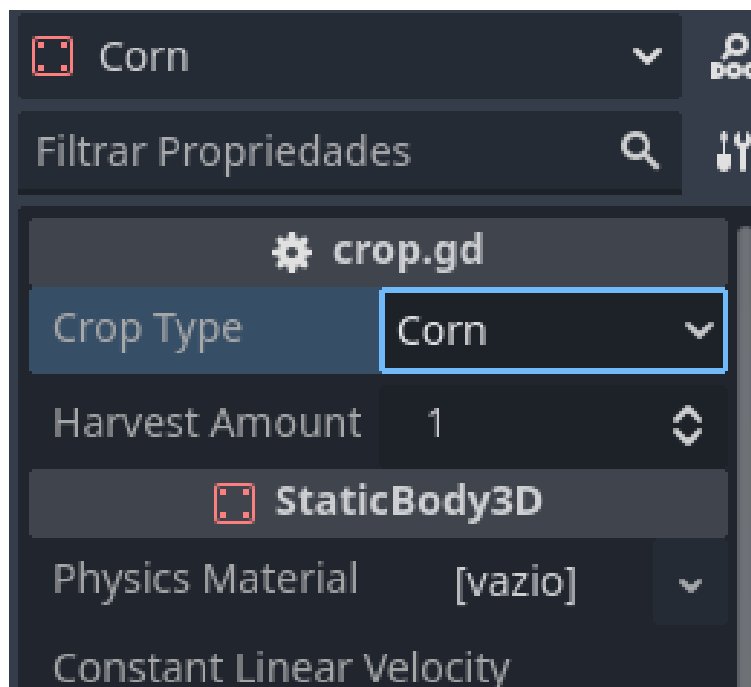


Figura 83: Setando tipo do milho

Esse script define a base do sistema de crescimento das plantas. No próximo passo, implementaremos mecânicas adicionais.



15 Mecânicas de Cultivo - Plantio

15.1 Preparando o Terreno

Para iniciar a mecânica de plantio, primeiro precisamos criar o solo onde as plantas poderão ser cultivadas. Siga os passos abaixo para configurar o terreno:

1. Crie uma nova cena e defina o nó raiz como **StaticBody3D**.
2. Nomeie a cena como **plantation.tscn**.
3. Adicione dois nós filhos:
 - **MeshInstance3D** - Para representar a aparência do solo.
 - **CollisionShape3D** - Para permitir interação física.

O resultado esperado pode ser visto na Figura 84.

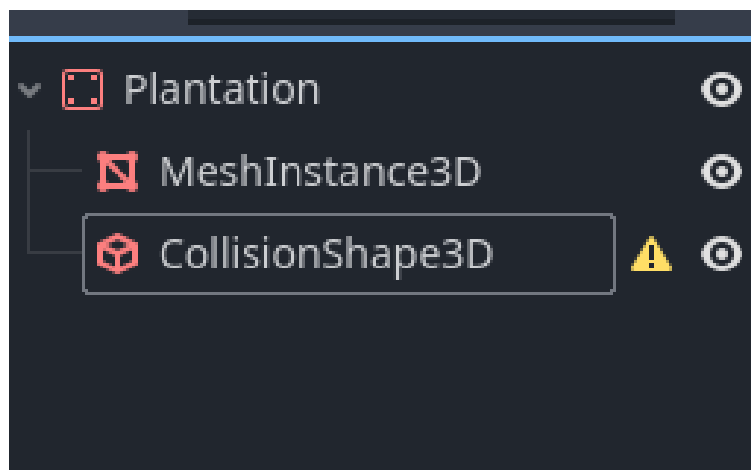


Figura 84: Estrutura da cena **plantation.tscn**

Agora, precisamos definir a forma do solo:

1. Selecione o **CollisionShape3D** e o **MeshInstance3D**.
2. No campo **Shape**, escolha a opção **Cube**.

Isso criará um cubo branco, como mostrado na Figura 85.

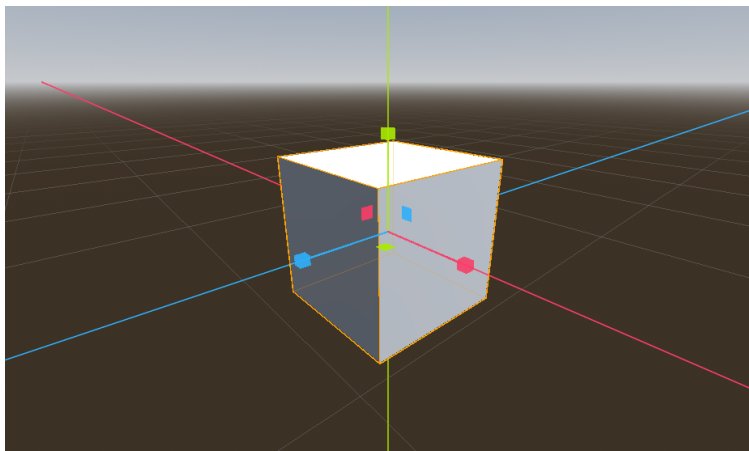


Figura 85: Terreno inicial representado por um cubo

15.1.1 Aplicando Cor ao Solo

Para tornar o solo visualmente mais adequado, aplicaremos uma cor ao **MeshInstance3D**. Siga os passos abaixo:

1. Acesse a aba **Geometry** (Figura 86).

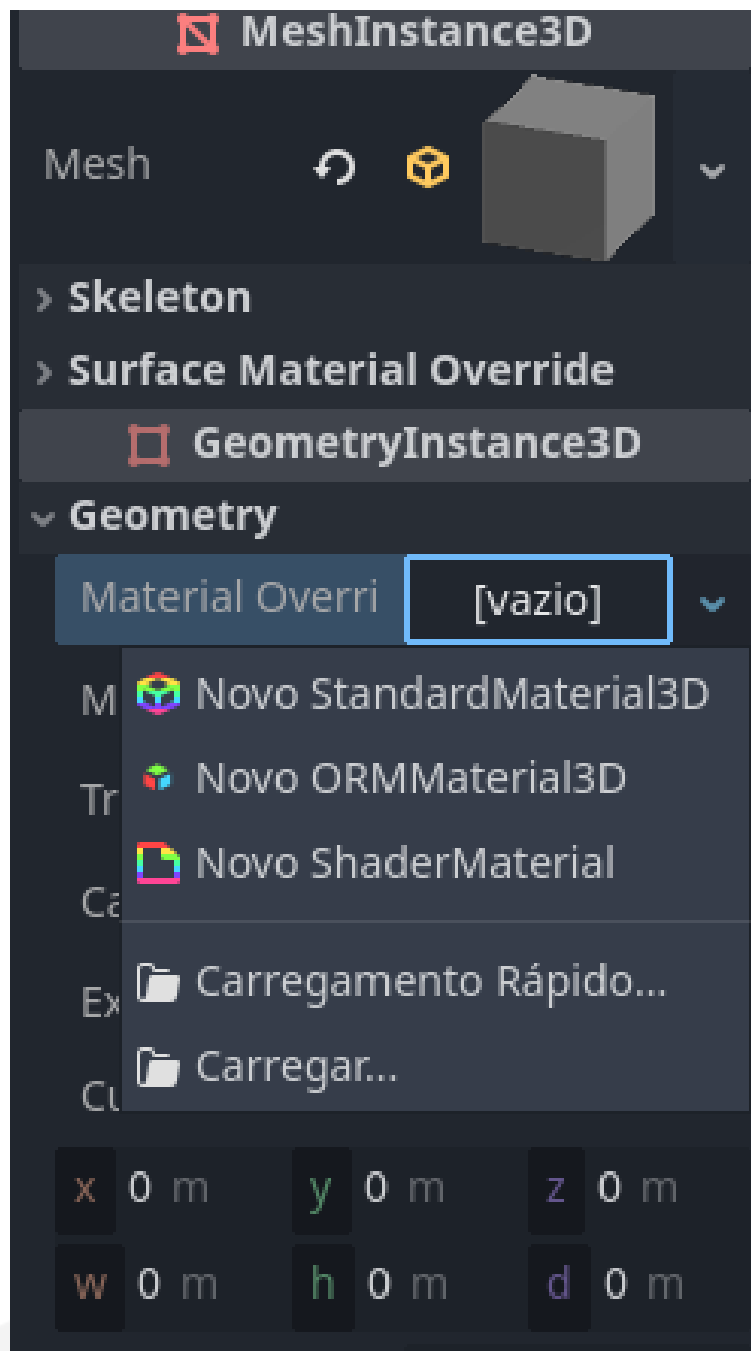


Figura 86: Aba Geometry aberta

2. Clique em **Novo StandardMaterial3D**.
3. Vá até a opção **Albedo**.
4. Escolha a cor desejada (Figura 87).

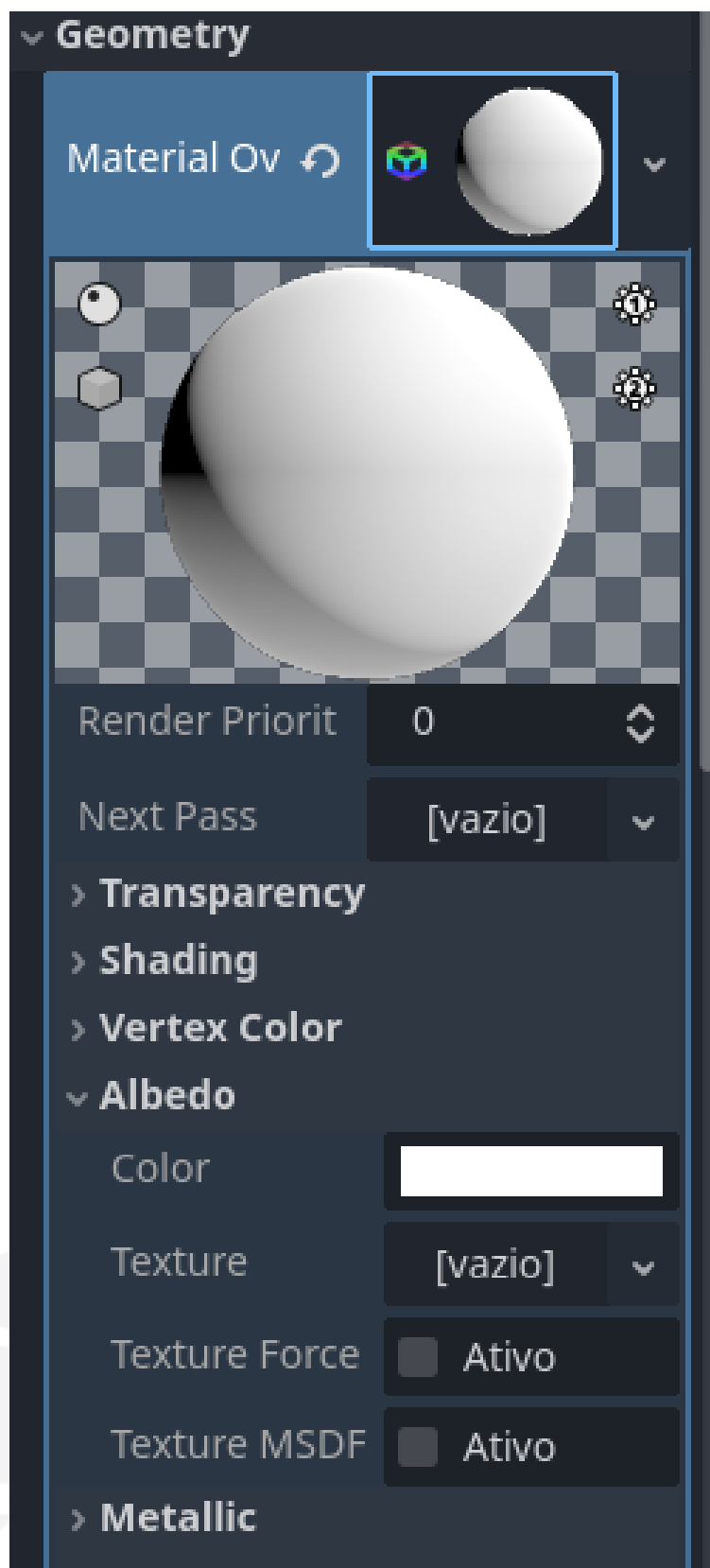


Figura 87: Selecionando a cor para o solo



15.1.2 Ajustando o Formato do Terreno

Agora, devemos ajustar o formato do solo para que fique mais semelhante a uma área de plantio. Para isso:

1. Selecione o nó raiz (**StaticBody3D**).
2. Achate o cubo para que fique com um formato mais adequado para representar o solo.

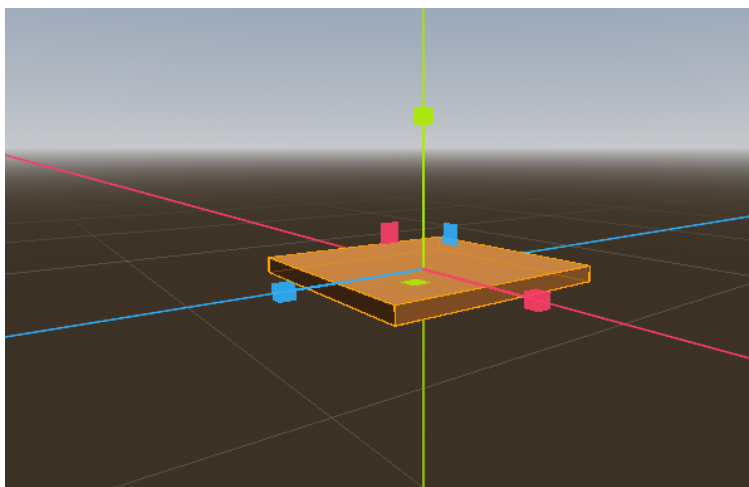


Figura 88: Plantation finalizado

15.1.3 Configuração de Colisão

Para permitir que o terreno seja detectado corretamente durante a mecânica de plantio, precisamos configurar as camadas de colisão:

1. No nó **StaticBody3D**, vá até a aba **Collision**.
2. Defina a **Collision Layer** e a **Collision Mask** como **3**.

O resultado esperado pode ser visto na Figura 89.

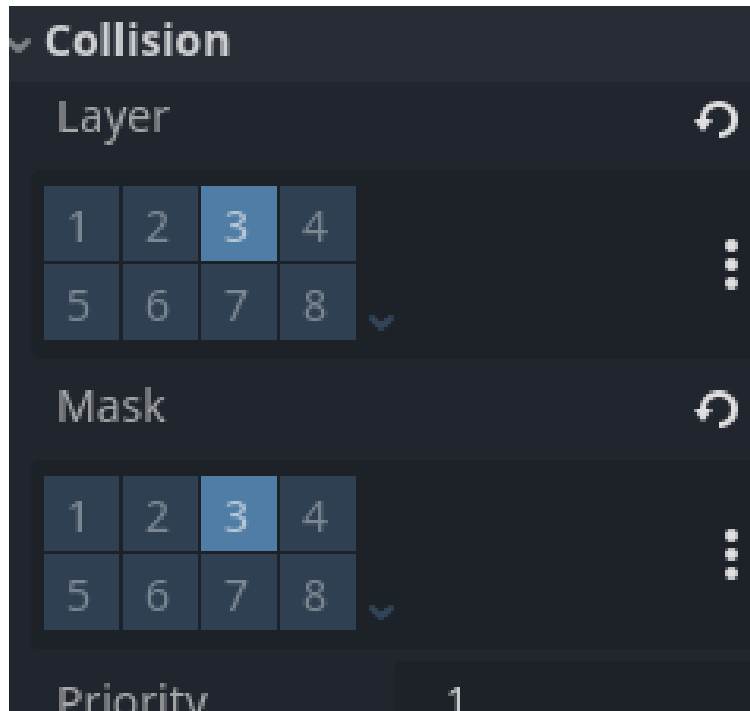


Figura 89: Configuração de colisão do terreno de plantio

Após isso, adicione um Script ao Nó raiz de Plantation

```
extends StaticBody3D

var occupied = false # VE SE O SOLO JA TEM UMA PLANTA (1 PLATA POR SOLO)
var crop: Node = null # REFERENCIA A PLANTA DESSE SOLO
var fertilized = false # CHECA SE JA FERTILIZOU NESTE DIA

func update_prompt_text(): # INTERFACE DO JOGO
    if !fertilized:
        return "Press F to Fertilize "
    else:
        if !occupied:
            return "Click MOUSE 1 to Plant"
        return ""
```

Com isso, nosso terreno de plantio está pronto! No próximo passo, implementaremos o sistema de interação do jogador com o solo.

15.2 Player Interacter

Agora iremos preparar o **player** para interagir com o nosso milho e o sistema de plantio. Para isso, siga os passos abaixo:

1. Vá até a cena do **player**.
2. Adicione um **Node3D** como filho do nó raiz.
3. Renomeie esse nó para **Interacter**.



O resultado esperado pode ser visto na Figura 90.

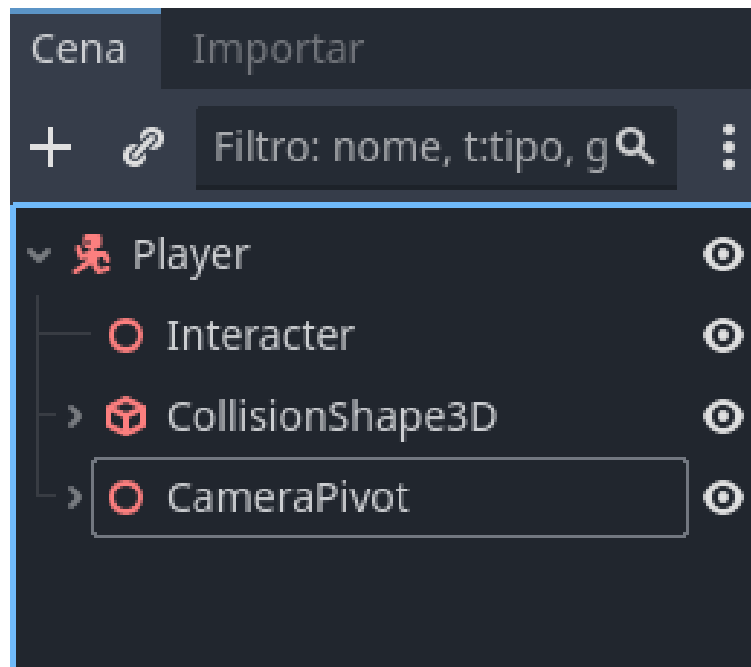


Figura 90: Hierarquia do player com o nó **Interacter**

Agora, precisamos criar um script para o nó **Interacter**. Esse nó será responsável por todas as interações do jogador com o ambiente, incluindo **plantar**, **regar** e **colher** as plantas.

Para isso, utilizaremos uma técnica chamada **raycasting**. Essa técnica emite um **raio invisível** a partir do centro da câmera do jogador, detectando objetos dentro de um determinado alcance. Caso o raio encontre um objeto interagível, ele executará uma função correspondente.

Crie um novo script para o **Interacter** e adicione o seguinte código:

```
extends Node3D

var distance = 100.0

signal update_prompt_text
signal harvested_crop

@export var corn_scene : PackedScene
@export var plantation_scene : PackedScene
```

Esse é o início do código do **Interacter**, e ele contém:

- **distance**: Define a distância máxima que o raycast pode alcançar (inicialmente alta para facilitar o debug).
- **update_prompt_text**: Um sinal que será emitido para alterar o texto da interface do jogador, informando se a planta está pronta para ser colhida.
- **harvested_crop**: Um sinal que será emitido quando uma planta for colhida.
- **corn_scene**: Referência à cena do milho, permitindo sua instância no mundo.
- **plantation_scene**: Referência à cena do solo, permitindo sua interação.



15.2.1 Implementando a Função de Raycast

Agora, vamos criar a principal função do **Interacter**: o raycast.

```
func raycast(layer: int):  
    var space_state = get_world_3d().get_direct_space_state()  
    var cam = get_node("../CameraPivot/FirstPerson") # Caminho para a câmera  
    var mouse_pos = get_viewport().get_mouse_position()  
  
    # Calcula a origem e a direção do raio com base na posição do mouse  
    var origin = cam.project_ray_origin(mouse_pos)  
    var end = origin + cam.project_ray_normal(mouse_pos) * distance  
  
    # Configurando os parâmetros do raycast  
    var query = PhysicsRayQueryParameters3D.new()  
    query.from = origin  
    query.to = end  
    query.collision_mask = layer # Camada de colisão  
    query.collide_with_areas = true # Permitir colisão com áreas  
  
    # Executando o raycast  
    var result = space_state.intersect_ray(query)  
    return result
```

Como Funciona o Raycast?

O **raycasting** é uma técnica que projeta um raio invisível a partir de um ponto na tela (neste caso, do centro da câmera do jogador). Ele permite detectar objetos dentro de uma determinada distância e retornar informações sobre o que foi atingido. No código acima:

- Obtém o estado do espaço 3D com `get_world_3d().get_direct_space_state()`.
- Recupera a referência da **câmera** do jogador.
- Captura a posição do mouse na tela.
- Calcula a **origem** e a **direção** do raio.
- Configura um objeto **PhysicsRayQueryParameters3D** para definir as regras da colisão.
- Executa o raycast e retorna o objeto detectado (caso haja um).

Configurando as Camadas de Colisão

Para que o raycast funcione corretamente, precisamos garantir que ele detecte apenas os objetos desejados. Para isso, configuraremos as camadas de colisão:

1. Acesse o menu **Projeto** (Figura 91).

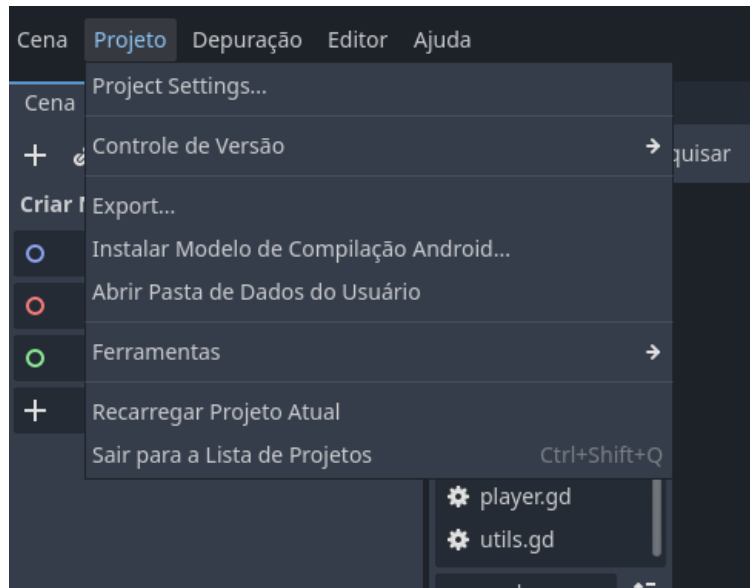


Figura 91: Acessando o menu de configurações do projeto

2. Em seguida, clique em **Configurações do Projeto** (*Project Settings*).
3. Vá até a aba **Geral** (*General*), conforme mostrado na Figura 92.

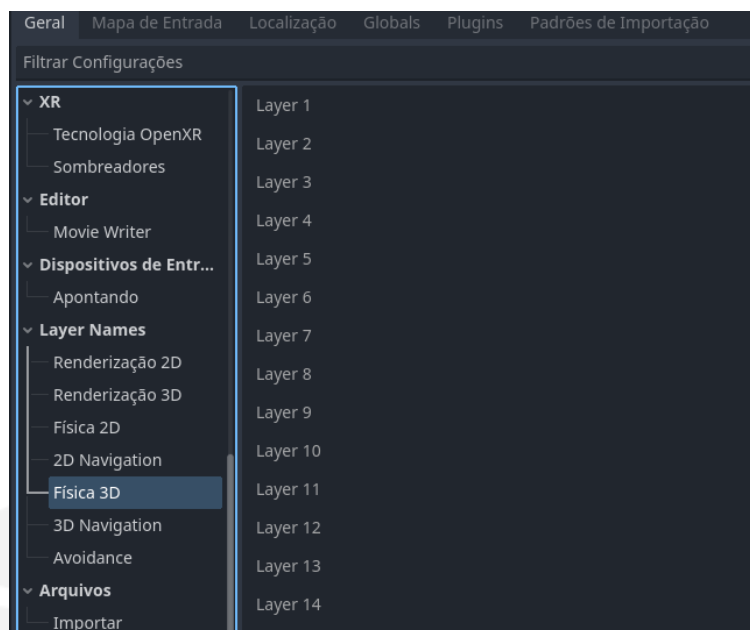


Figura 92: Aba Geral do Project Settings

4. Role a barra da esquerda até o final e acesse **Physics 3D**.
5. Em **Physics 3D**, organize as camadas conforme a Figura 93.
6. Layer 1 = ground
7. Layer 2 = crop



8. Layer 3 = plantation
9. Layer 4 = player

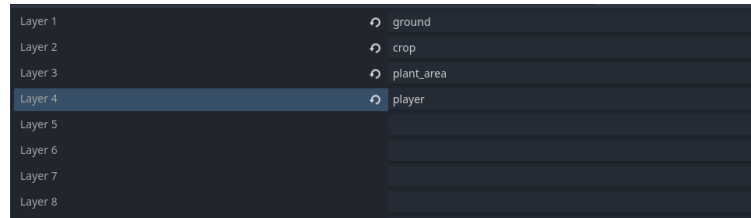


Figura 93: Configuração das camadas de colisão

Com essa configuração, garantimos que o raycast detectará corretamente os objetos interativos no ambiente do jogo.

15.2.2 Adicionando Plantation à Crop

Agora que possuímos a cena **Plantation**, podemos adicionar uma referência a ela dentro da **Crop**. Essa referência será útil para permitir a interação entre a planta e o solo em que foi plantada.

```
extends StaticBody3D

@export var crop_type: CropTypes.CROP_TYPE # Tipo de planta a ser usada
@export var harvest_amount = 1 # Quantidade que será coletada

# Referências aos modelos 3D da planta
@onready var large = $Large
@onready var medium = $Medium
@onready var small = $Small

# Crescimento da planta: valor atual e tempo total necessário
var growth_value = 0.0
var growth_time = 30

var watered = false # Indica se foi regada
var harvestable = false # Indica se pode ser colhida
var plantation: Node = null # Referência ao solo onde foi plantada
```

15.2.3 Implementando a Função de Preparar o Solo

Agora vamos criar uma função que prepara o solo para o plantio. Isso será feito utilizando um **raycast**, que verificará se o jogador está mirando em uma área válida de solo.



```
func prepare_ground():
    var result = raycast(1) # Camada 1 corresponde ao solo geral
    if result:
        if result.collider.get_collision_layer() == 1:
            print("Raio colidiu com: ", result.collider.get_collision_layer())
            var plantation_instance = plantation_scene.instantiate()
            get_tree().get_root().add_child(plantation_instance)
            plantation_instance.global_transform.origin = result.position
```

Essa função utiliza um **raycast** para detectar um terreno e criar um espaço de plantio. O funcionamento é o seguinte:

1. Chama a função **raycast(1)**, onde **1** representa a **camada de colisão do solo geral**.
2. Se um objeto for detectado (**result** não for nulo):
 - Verifica se o objeto atingido está na camada **1** (solo geral).
 - Exibe no console a confirmação da colisão.
 - Instancia um novo **Plantation** (terreno de plantio).
 - Adiciona o terreno de plantio à cena na posição detectada.

15.2.4 Implementando a Função de Plantar

Agora vamos criar a função que permitirá ao jogador plantar uma semente em um terreno preparado.



```
func plant(type : CropTypes.CROP_TYPE):  
    var result = raycast(4) # Camada de colisão 3 -> Representada como 4 em binario (100b)  
    if result:  
        if result.collider.get_collision_layer() == 4: # Verifica se o objeto é um solo  
            valido para plantio  
            var soil = result.collider  
            if !soil.occupied: # Verifica se o terreno já está ocupado  
                var crop_instance  
                if type == CropTypes.CROP_TYPE.CORN:  
                    crop_instance = corn_scene.instantiate()  
                    print("CORN")  
  
                elif type == CropTypes.CROP_TYPE.TOMATO:  
                    crop_instance = tomato_scene.instantiate()  
                    print("TOMATO")  
  
                crop_instance.plantation = soil  
                get_tree().get_root().add_child(crop_instance)  
  
                # Posiciona a planta no centro do terreno  
                var center_position = soil.global_transform.origin  
                crop_instance.global_transform.origin = center_position  
  
                # Rotação aleatória para variar a aparência  
                crop_instance.rotate_y(randf_range(0.0, TAU))  
  
                # Pequena variação no tamanho para maior naturalidade  
                crop_instance.global_scale(Vector3.ONE * randf_range(0.9,1.1))  
  
                # Marca o terreno como ocupado  
                soil.occupied = true  
                soil.crop = crop_instance  
  
                # Caso o solo esteja fertilizado, reduz o tempo de crescimento  
                if soil.fertilized:  
                    crop_instance.growth_time *= 0.9 # Diminui o tempo de crescimento em 10%
```

Essa função permite ao jogador plantar uma semente em um terreno específico, garantindo que apenas terrenos desocupados possam receber uma nova planta.

1. O **raycast** é chamado na camada **3** (correspondente ao terreno de plantio).
2. Se um terreno for detectado:
 - Verifica se ele está na camada **3** (permitida para plantio).
 - Verifica se ele já está ocupado por outra planta.
 - Se estiver livre:
 - Instancia a planta correta (Milho ou Tomate).
 - Define o solo onde ela foi plantada.
 - Posiciona a planta no centro do terreno.
 - Adiciona uma rotação e uma pequena variação no tamanho para evitar repetição visual.
 - Marca o terreno como ocupado.
 - Caso o solo tenha sido fertilizado, reduz o tempo de crescimento em 10%.



Nota: O raycast está testando o valor **4** porque o sistema de camadas do **Godot** representa as camadas de forma binária.

- A camada **1** ativada é representada como **001** (1 em decimal). - A camada **1 e 2** ativadas juntas são representadas como **011** (3 em decimal). - A camada **3** ativada sozinha é representada como **100**, que convertida para decimal resulta em **4**.

Portanto, quando o raycast verifica a camada **4**, ele está na verdade detectando objetos que pertencem exclusivamente à camada **3**.

Essa implementação garante que o sistema de plantio seja dinâmico, permitindo ao jogador criar novos terrenos e plantar sementes de maneira organizada. No próximo passo, implementaremos a mecânica de rega e colheita!

15.2.5 Regar e Fertilizar

Agora iremos implementar as funções de **regar** e **fertilizar**. Essas funções são relativamente simples e servem para **acelerar o crescimento da planta**, alterando os valores booleanos **watered** (rega) e **fertilized** (fertilização).

A principal diferença entre ambas é que:

- Para **regar**, verificamos uma **colisão do raycast com a planta**.
- Para **fertilizar**, verificamos uma **colisão do raycast com o solo preparado**.

Abaixo está o código correspondente:

```
func watering_crop():
    var result = raycast(2)
    if result:
        if result.collider.get_collision_layer() == 2:
            var crop = result.collider
            if !crop.watered:
                print("Regado")
                crop.growth_value += 0.1 # Acelera 10% o crescimento
                crop.watered = true

func fertilizing():
    var result = raycast(4) # 4 representa a camada do solo preparado
    if result:
        if result.collider.get_collision_layer() == 4:
            var soil = result.collider
            if !soil.fertilized:
                print("Solo fertilizado")
                soil.fertilized = true
```

1. Função **watering_crop()** Esta função permite que o jogador **regue uma planta** para acelerar seu crescimento.

1. Chama o **raycast(2)**, onde **2** representa a **camada de colisão das plantas**.
2. Se um objeto for detectado:
 - Verifica se pertence à camada correta (**2**).
 - Obtém a referência da planta atingida.
 - Se a planta ainda não tiver sido regada:



- Exibe no console a confirmação da ação.
- Aumenta o **growth_value** da planta em **10%**.
- Define **watered = true**, impedindo que a planta seja regada novamente.

2. Função **fertilizing()** Esta função permite que o jogador **fertilize o solo**, preparando-o para um crescimento mais eficiente das plantas.

1. Chama o **raycast(4)**, onde **4** representa a **camada de colisão do solo preparado**.
2. Se um objeto for detectado:
 - Verifica se pertence à camada correta (**4**).
 - Obtém a referência do solo atingido.
 - Se o solo ainda não tiver sido fertilizado:
 - Exibe no console a confirmação da ação.
 - Define **fertilized = true**, permitindo que futuras plantas cresçam mais rápido.

Com essa implementação, o jogador pode regar e fertilizar suas plantações, garantindo um crescimento mais rápido das culturas.

15.2.6 Função de Colher

Agora finalizaremos as funções do **Interacter**, adicionando a função de colheita. Essa função verificará a **colisão do raycast com a camada de plantas (crops)** e, caso haja uma planta colhível, ela será removida do jogo.

Além disso, a função:

- Emite um sinal informando que a planta foi colhida.
- Remove a planta da cena.
- Libera o solo para futuras plantações.
- Remove qualquer fertilizante aplicado ao solo.

Abaixo está o código correspondente:

```
func interact():
    var result = raycast(2) # Camada de colisão 2 -> Representada como 010b (2 em decimal)
    if result:
        if result.collider.get_collision_layer() == 2:
            var crop = result.collider
            if crop.harvestable:
                emit_signal("harvested_crop", crop.get_crop_type(), crop.get_harvest_amount())
            crop.interact()

            crop.plantation.occupied = false # Desocupa o solo
            crop.plantation.fertilized = false # Remove fertilizante do solo
            crop.plantation.crop = null # Remove referência da planta no terreno
```

Esta função é responsável por detectar quando o jogador tenta colher uma planta e garantir que o terreno seja corretamente atualizado.

1. Chama o **raycast(2)**, onde **2** representa a **camada de colisão das plantas**.



2. Se um objeto for detectado:

- Verifica se pertence à camada correta (2).
- Obtém a referência da planta atingida.
- Se a planta estiver pronta para colheita (**harvestable = true**):
 - Emite um sinal **"harvested_crop"** contendo o tipo da planta e a quantidade colhida.
 - Chama a função **interact()** da própria planta, que a remove da cena.
 - Atualiza o terreno:
 - * Define **occupied = false**, permitindo o plantio de uma nova semente.
 - * Define **fertilized = false**, removendo qualquer fertilizante aplicado.
 - * Remove a referência da planta no terreno, evitando referências inválidas.

Essa implementação garante que o sistema de colheita funcione corretamente, permitindo ao jogador interagir de forma dinâmica com suas plantações.

Nota: O booleano de regar deve ser resetado baseado no sistema de dia e noite.

15.2.7 Função process

A função process deve ler os inputs do teclado e checar qual ação está sendo feita, a partir disso ela chama a função em questão.

```
if Input.is_action_just_pressed("interacao"):
    interact()
if Input.is_action_just_pressed("plant"):
    plant(CropTypes.CROP_TYPE.CORN)
if Input.is_action_just_pressed("prepare_ground"):
    prepare_ground()
if Input.is_action_just_pressed("watering_crop"):
    watering_crop()
if Input.is_action_just_pressed("fertilizing"):
    fertilizing_soil()

var crop_ray = raycast(2)
var soil_ray = raycast(4)
var ground_ray = raycast(1)
elif crop_ray :
    emit_signal("update_prompt_text", crop_ray.collider.update_prompt_text())
elif soil_ray:
    emit_signal("update_prompt_text", soil_ray.collider.update_prompt_text())
elif ground_ray :
    emit_signal("update_prompt_text", ground_ray.collider.update_prompt_text())
else:
    emit_signal("update_prompt_text", "")
```

Após finalizado o script selecione o Nó do Interacter, e vá ao inspector, onde está escrito Corn Scene arraste a cena Corn e onde está escrito Plantation Scene arraste a cena Plantation.

Com isso já teremos um sistema de plantação funcional, basta apenas programar a UI de update_prompt_text.

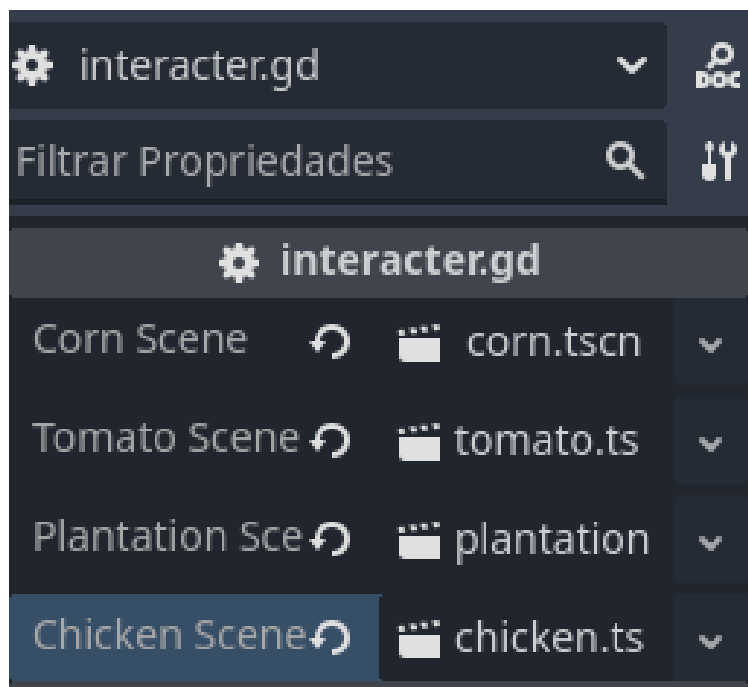


Figura 94: Setando imports do interacter

16 Criando HUD

Neste capítulo, vamos criar a **HUD** (*Heads-Up Display*), que mostrará ao jogador quais interações estão disponíveis no jogo. Essa HUD utilizará o **signal update_prompt** fornecido pelo **Interacter** para exibir informações relevantes.

16.1 Criando a Estrutura da HUD

Para começar, siga os passos abaixo:

1. Crie uma nova cena chamada **hud.tscn**.
2. Defina o nó raiz como um **Control**.
3. Adicione os seguintes nós como filhos do nó raiz:
 - Um nó do tipo **Label**, renomeado para **Prompt**.
 - Um nó do tipo **ColorRect**, renomeado para **CrossHair**.

O resultado esperado pode ser visto na Figura 95.

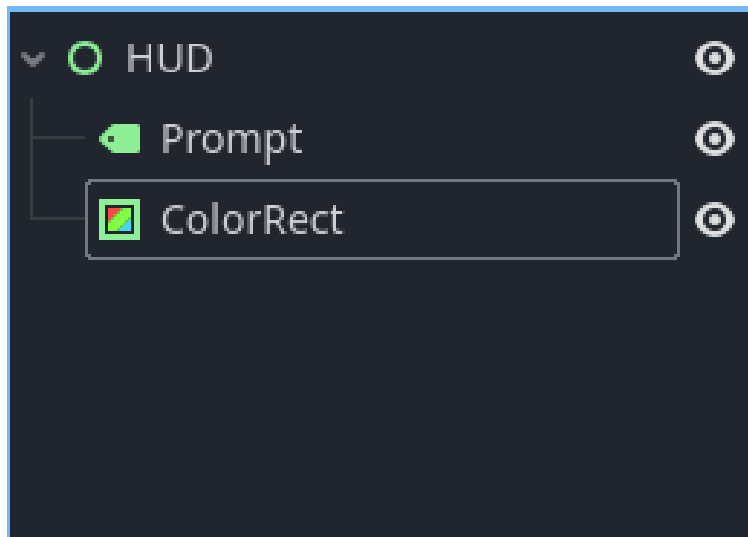


Figura 95: Estrutura inicial da HUD com os nós criados

16.2 Configurando a HUD

Agora, vamos ajustar a posição e o tamanho dos elementos da HUD:

1. Entre no **modo de edição 2D** (Figura 96).

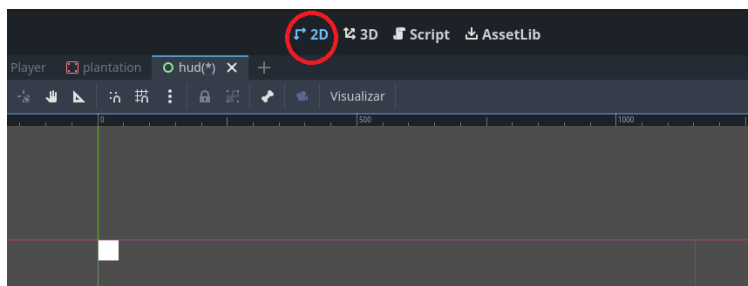


Figura 96: Modo de edição 2D da Godot

2. Selecione o nó raiz da HUD e **atualize a âncora** para centralizá-lo (Figura 97).

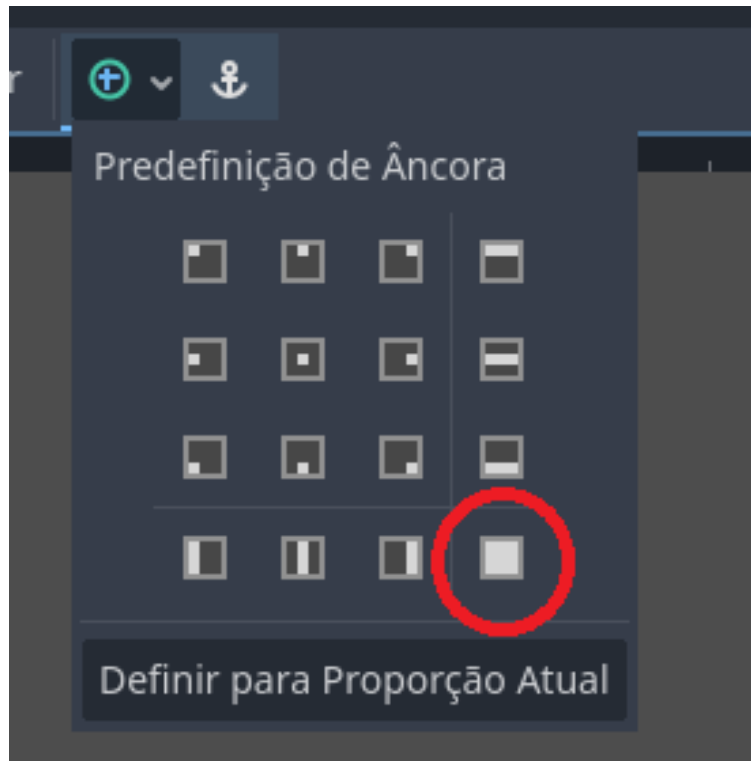


Figura 97: Centralizando a âncora da HUD

3. Centralize o nó **CrossHair** (**ColorRect**) e diminua seu tamanho.
4. Centralize o **Prompt** (**Label**) e mova-o levemente para a esquerda.
O resultado final pode ser visto na Figura 98.

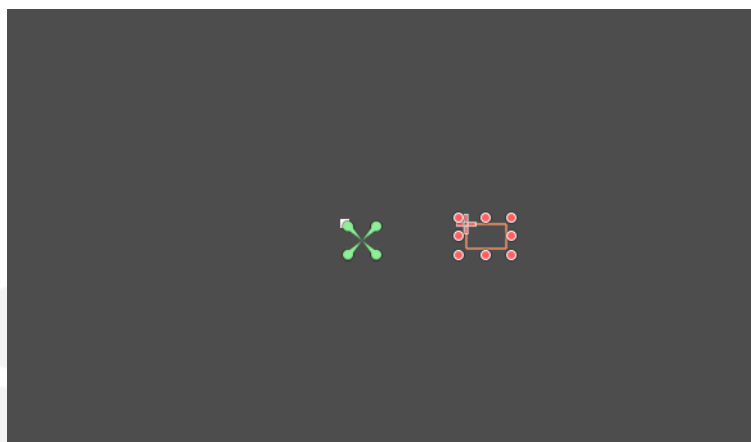


Figura 98: Resultado final da HUD posicionada corretamente

16.3 Criando o Script da HUD

Agora, criaremos um script para o nó raiz da HUD e adicionaremos o seguinte código:



```
extends Control

@onready var prompt: Label = $Prompt
```

Esse script permite que a HUD acesse o **Prompt**, que será atualizado dinamicamente de acordo com as interações do jogador.

16.4 Conectando a HUD ao Interacter

Para que a HUD exiba corretamente as mensagens de interação, precisamos conectá-la ao **Interacter**. Para isso, adicione o seguinte código ao **player.gd**:

```
@onready var interacter = $Interacter

func _ready():
    await get_tree().process_frame # Aguarda um frame para garantir que a HUD foi carregada

    update_camera_mode()
    interacter.connect("update_prompt_text", UIManager.update_prompt_text)
```

- **@onready var interacter = \$Interacter** Obtém a referência do **Interacter**, responsável por detectar interações.
- **await get_tree().process_frame** Aguarda um quadro antes de continuar a execução, garantindo que a HUD já tenha sido carregada na cena.
- **interacter.connect("update_prompt_text", UIManager.update_prompt_text)** Conecta o sinal **update_prompt_text** do **Interacter** à função de atualização da HUD.

Com isso, finalizamos a estrutura básica da HUD!



17 Animando Ferramentas

Neste capítulo, vamos criar uma animação geral para as ferramentas e finalizar as funções de plantio. Essa animação simulará o movimento da mão do jogador ao segurar e utilizar ferramentas.

17.1 Criando a Estrutura da Animação

Para começar, precisamos adicionar um novo nó ao jogador, que será responsável por armazenar as animações. Siga os passos abaixo:

1. Na cena do **player**, crie um nó filho do tipo **Node3D** e nomeie-o como **Hand**.
2. Como filho de **Hand**, adicione um **MeshInstance3D** e atribua um formato de **cápsula**. Esse objeto servirá como um guia para as animações.
3. Adicione os seguintes nós como filhos de **Hand**:
 - **AnimationPlayer** - Responsável por armazenar as animações.
 - **AnimationTree** - Gerenciará a transição entre animações.
4. Posicione o nó **Hand** de forma que simule a mão do personagem (Figura 99).

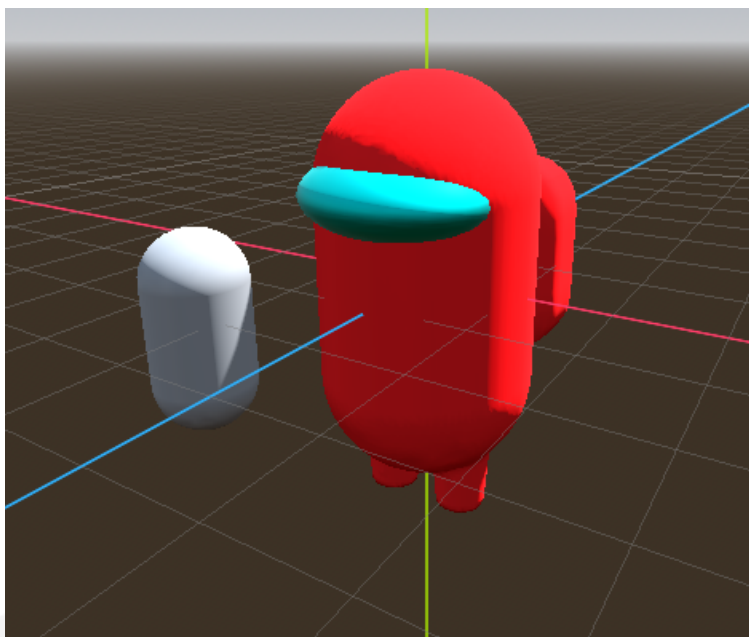


Figura 99: Posicionando o nó **Hand** para simular a mão do personagem

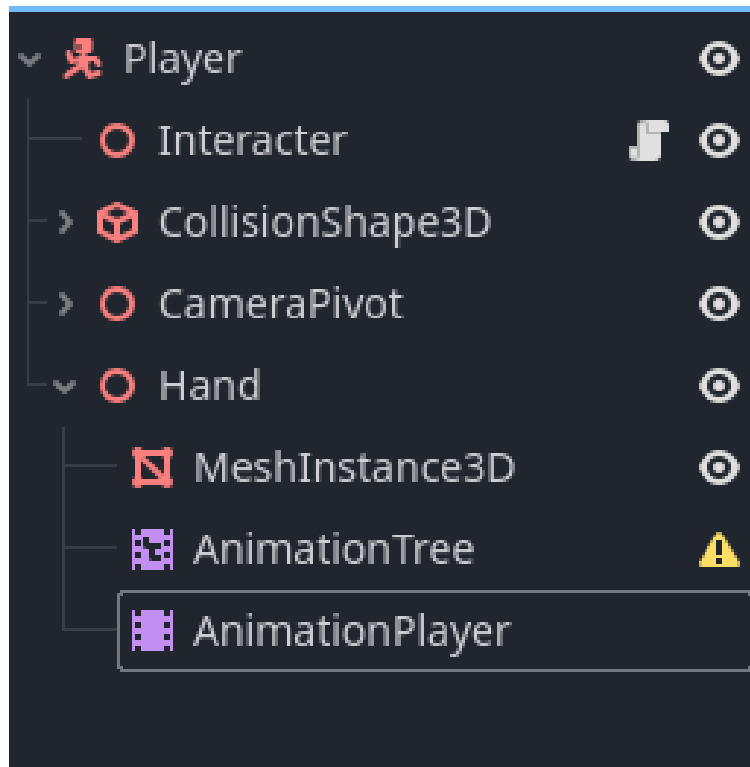


Figura 100: Hierarquia final dos nós dentro de **Hand**

17.2 Criando a Animação da Ferramenta

Agora, vamos criar a animação que representará o movimento do jogador ao segurar uma ferramenta.

1. Selecione o **AnimationPlayer** dentro do nó **Hand**.
2. Crie uma nova animação chamada **item_anim**.
3. Clique em **Adicionar Faixa** e selecione **Faixa de Rotação 3D**.
4. Referencie o nó **Hand** para que ele seja animado.
5. Adicione um **KeyFrame** no início da animação.
6. No tempo 0.5s da animação, rotacione o nó **Hand** **27.5 graus no eixo X**.
7. Adicione um **KeyFrame** em **0.5 segundos**.
8. No final da animação retorne o nó para o estado inicial.
9. Adicione um **KeyFrame** no final.
10. Ative o modo **Repetir Animação** (Figura 101).

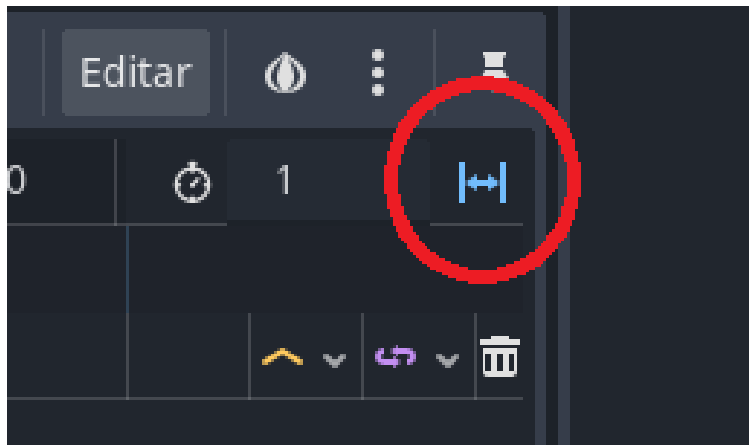


Figura 101: Ativando o modo de repetição da animação

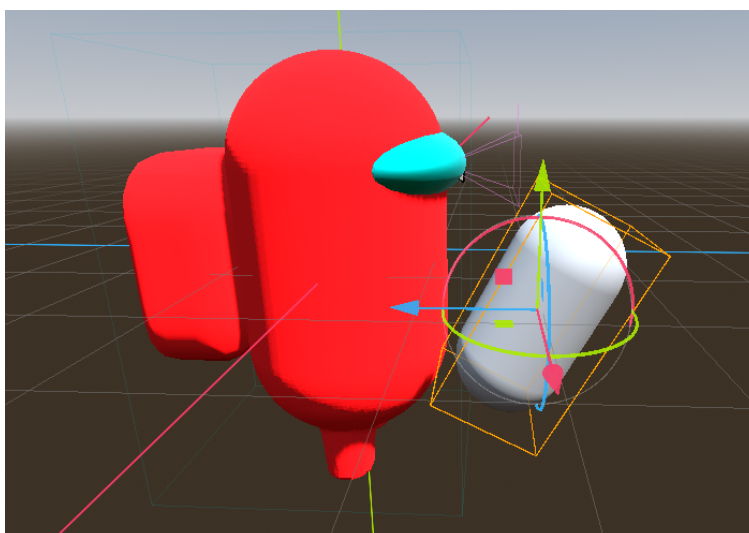


Figura 102: Resultado da animação da ferramenta no ambiente 3D

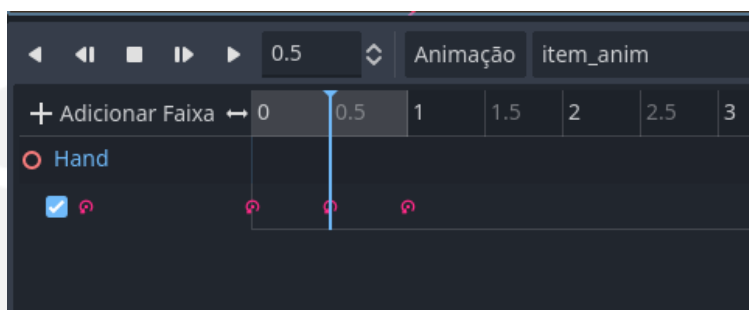


Figura 103: Resultado final da animação



17.3 Criando Animações Adicionais

Agora que criamos a animação básica de movimentação da ferramenta, adicionaremos mais duas animações:

- **Idle:** Para quando o jogador está parado.
- **Interagir:** Para representar qualquer ação de interação.

17.3.1 Animação idle

A animação **idle** será usada quando o personagem **não estiver se movendo ou interagindo**. Para criá-la, siga os passos abaixo:

1. No **AnimationPlayer**, crie uma nova animação chamada **idle**.
 2. Siga o mesmo processo usado na animação **item_anim**.
 3. **Não adicione nenhum KeyFrame**, pois queremos que a ferramenta permaneça parada.
- O resultado esperado pode ser visto na Figura 104.

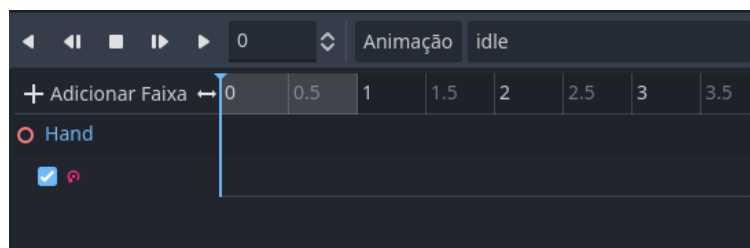


Figura 104: Resultado da animação **idle**

17.3.2 Animação interagir

A animação **interagir** será usada como uma **animação padrão para todas as interações**. Essa animação representa um movimento da ferramenta ao ser utilizada.

Para criá-la, siga os passos abaixo:

1. No **AnimationPlayer**, crie uma nova animação chamada **interagir**.
2. Siga o mesmo processo usado na animação **item_anim**.
3. Adicione **apenas dois KeyFrames**:
 - O primeiro KeyFrame deve representar a ferramenta na posição inicial.
 - O segundo KeyFrame (no final da animação) deve rotacionar o nó **Hand** **70 graus no eixo X**.
4. **Não ative** a repetição.

O resultado esperado pode ser visto na Figura 105.

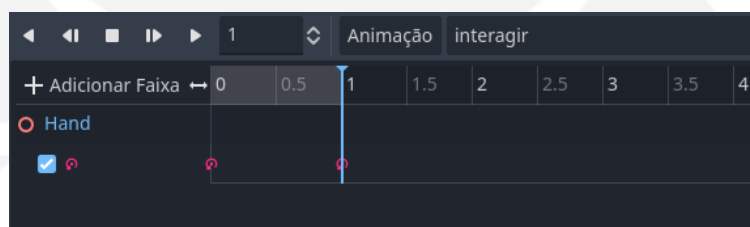


Figura 105: Resultado da animação **interagir**



Com isso, finalizamos as animações básicas das ferramentas.

17.4 Configuração do AnimationTree

Agora precisamos configurar o **AnimationTree** do nó **Hand**, garantindo que ele gerencie corretamente as animações da ferramenta.

17.4.1 Configuração do AnimationTree no Inspetor

Siga os passos abaixo para configurar o nó **AnimationTree**:

1. No Inspetor, vá para a opção **Tree Root** no **AnimationTree**.
2. Selecione a opção **NodeStateMachine** (Figura 106).

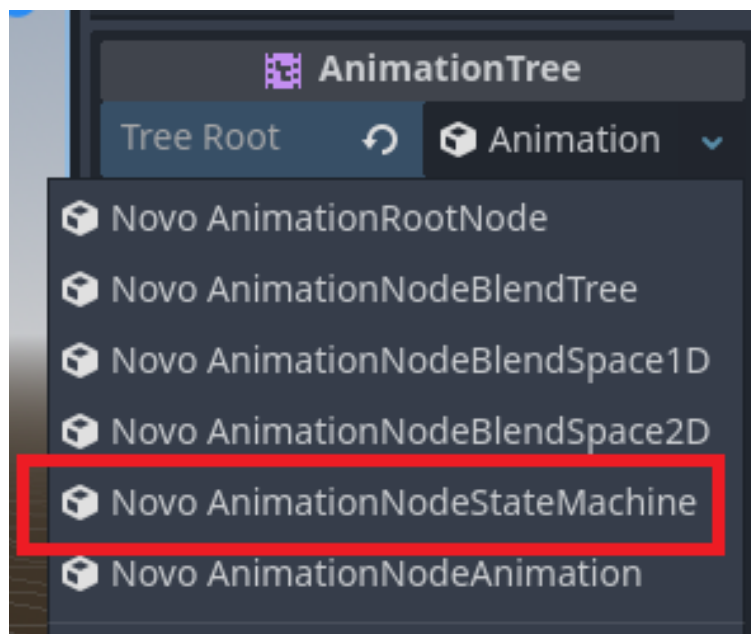


Figura 106: Selecionando o modo **NodeStateMachine**

3. Em **Anim Player**, selecione o **AnimationPlayer** do nó **Hand**.
4. Entre no editor do **AnimationTree**, onde já existirão dois nós: **Start** e **End** (Figura 107).



Figura 107: Nós iniciais do **AnimationTree**



17.4.2 Adicionando Estados de Animação

Agora, adicionaremos as animações ao **AnimationTree**:

1. Clique com o botão direito em um espaço vazio no editor do **AnimationTree** (Figura 108).
2. Adicione os três estados de animação criados anteriormente:
 - **Idle** (estado parado)
 - **item_anim** (animação de movimentação da ferramenta)
 - **interagir** (animação de interação)
3. Organize a posição dos estados para facilitar a visualização (Figura 109).

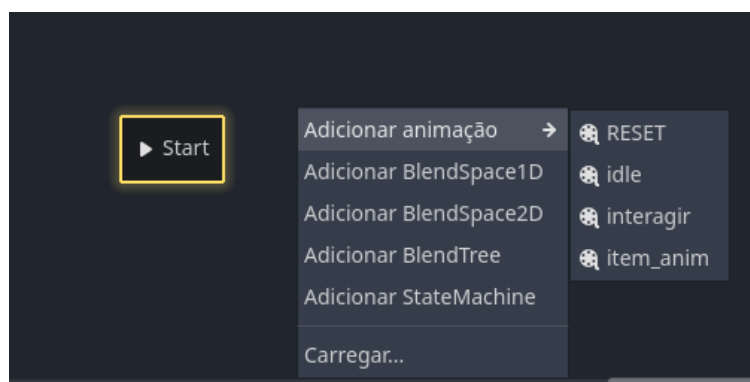


Figura 108: Adicionando estados de animação

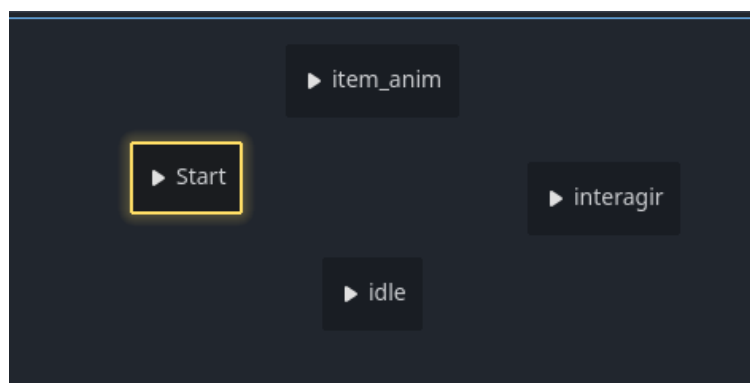


Figura 109: Organizando os estados no **AnimationTree**

17.4.3 Criando Transições Entre os Estados

Agora, precisamos conectar as animações criando as transições entre os estados.

1. Selecione a ferramenta de criação de transições (Figura 110).

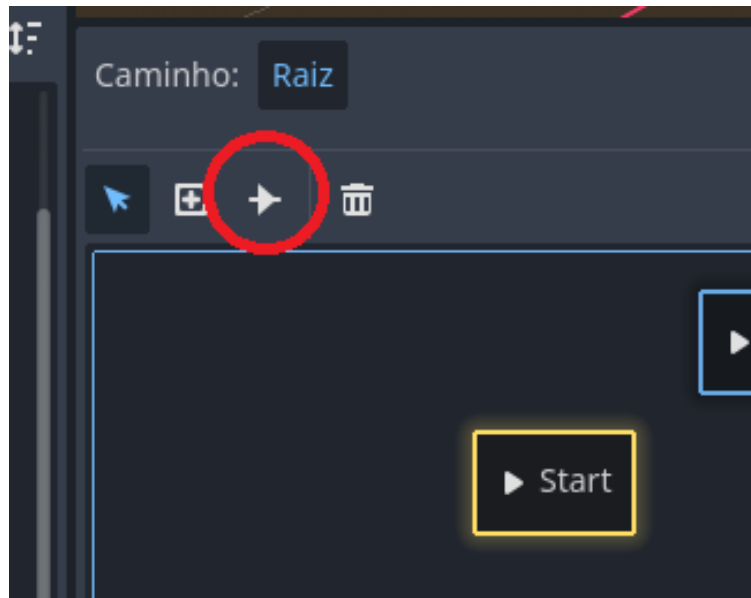
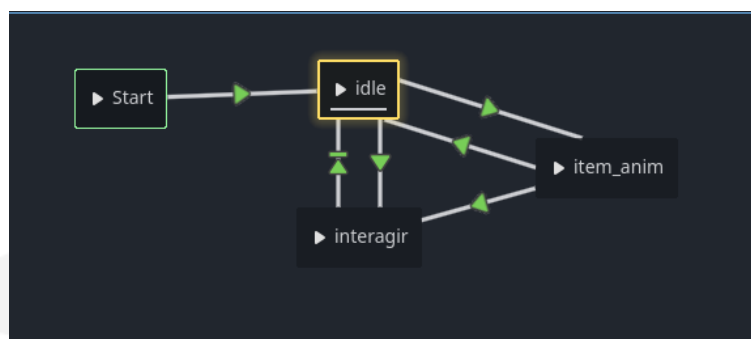


Figura 110: Ferramenta de criação de transições

2. Crie as seguintes transições entre os estados:

- **Start** → **Idle**
- **Idle** → **item_anim**
- **Idle** → **interagir**
- **item_anim** → **Idle**
- **item_anim** → **interagir**
- **interagir** → **Idle**

Figura 111: Transições criadas no **AnimationTree**

17.4.4 Adicionando Condições às Transições

Agora, precisamos adicionar condições para as transições entre os estados. Para isso, siga as configurações abaixo:



- Idle → item_anim - **Mode:** Auto - **Switch Mode:** Immediate - **Condition:** walking (Figura 112)

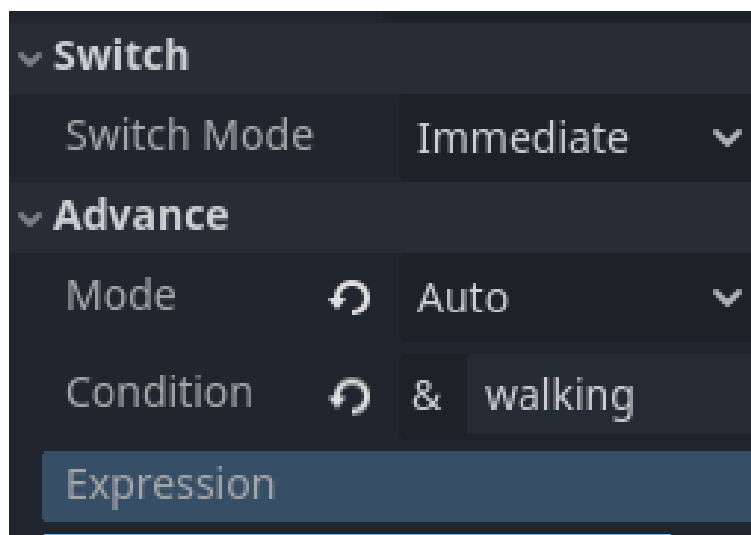


Figura 112: Configuração da transição Idle → item_anim

- Idle → interagir - **Mode:** Auto - **Switch Mode:** Immediate - **Condition:** interagindo
- item_anim → Idle - **Mode:** Auto - **Switch Mode:** Immediate - **Condition:** idle
- item_anim → interagir - **Mode:** Auto - **Switch Mode:** Immediate - **Condition:** interagindo
- interagir → Idle - **Mode:** Auto - **Switch Mode:** At End - **Condition:** Nenhuma condição



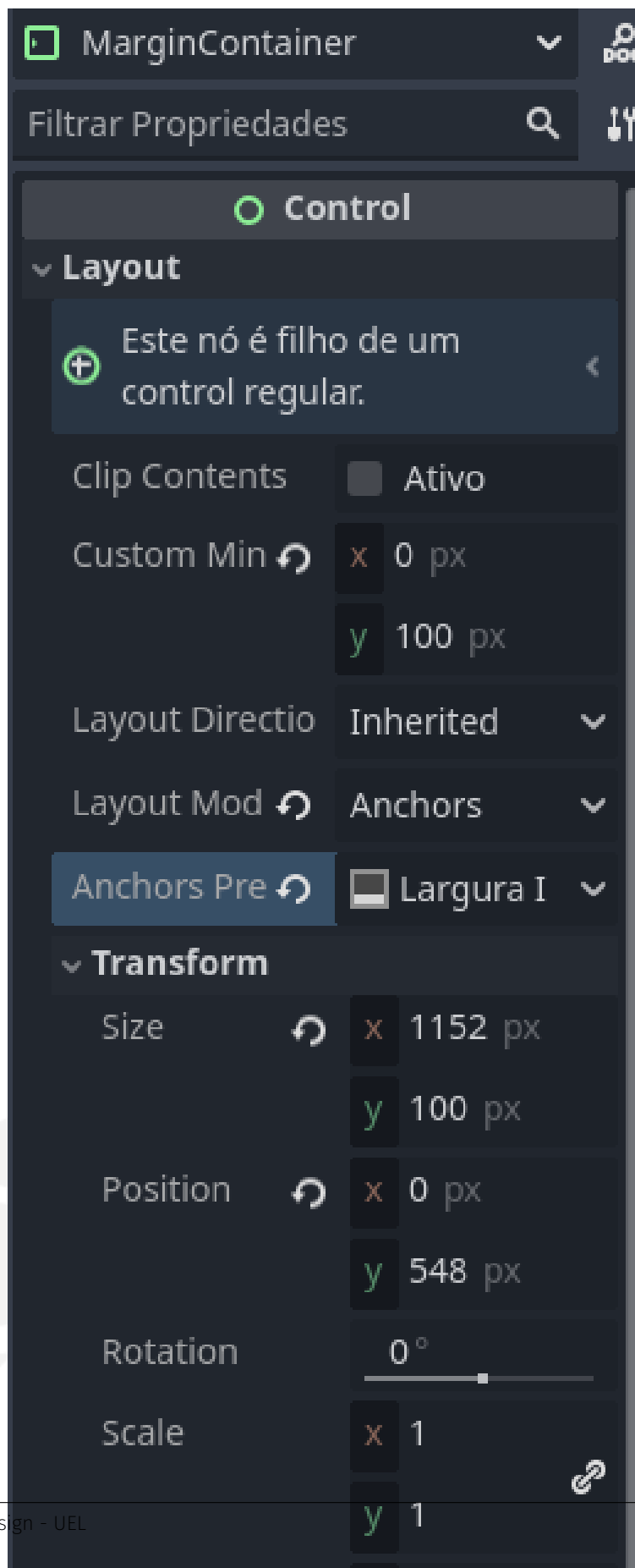
18 Hotbar

Agora iremos implementar a **Hotbar**, que servirá como um inventário rápido para o jogador. Para isso, precisaremos modificar a cena **HUD**.

18.1 Estruturando a Hotbar

Siga os passos abaixo para adicionar os elementos necessários à interface:

1. Dentro da cena **HUD**, crie um nó **MarginContainer** como filho do **HUD**.
2. No **Inspetor**, vá até a aba **Margin** e altere as configurações de layout para os seguintes valores (Figura 113).





3. Como filho do **MarginContainer**, adicione um **ColorRect** e renomeie-o para **Overlay**.
4. No **Inspetor**, configure a cor do **Overlay** de acordo com a imagem (Figura 114).

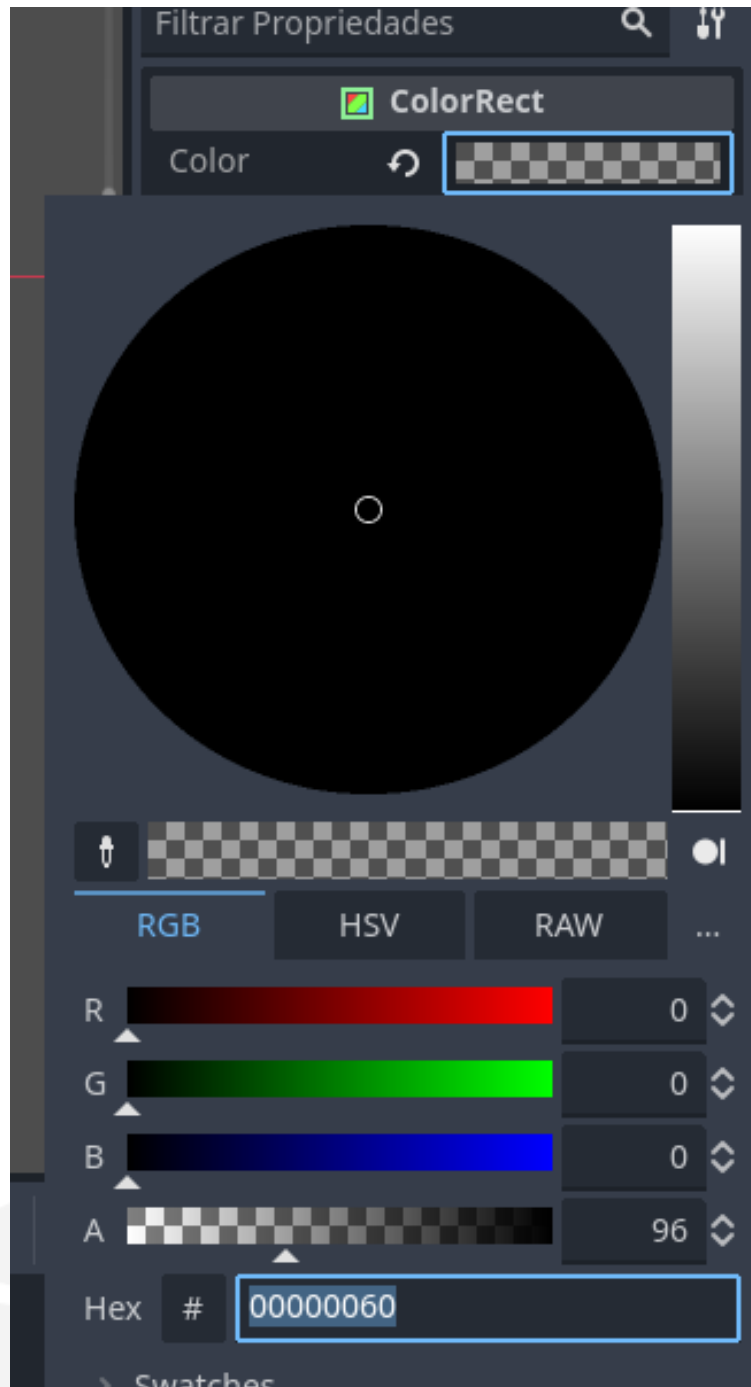


Figura 114: Configuração de cor para o Overlay

5. Adicione um novo **MarginContainer** como filho do **MarginContainer** criado anteriormente.
6. Como filho desse novo **MarginContainer**, adicione um **GridContainer** e renomeie-o para **Hotbar**.
7. No **Inspetor** do **Hotbar**, altere o número de colunas para **9**, conforme mostrado na Figura 115.



Figura 115: Configuração do número de colunas da Hotbar

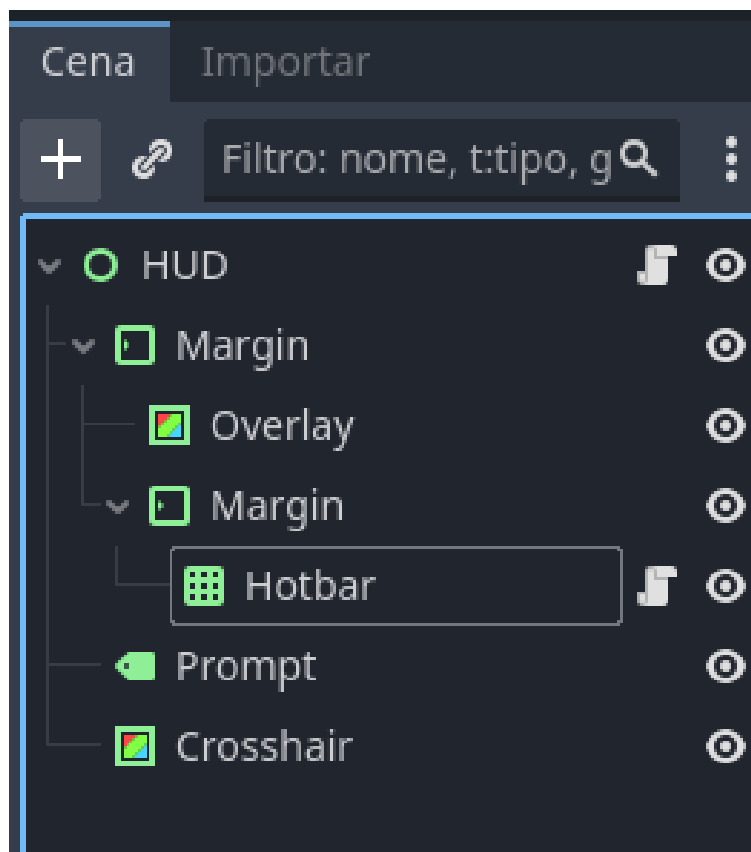


Figura 116: Estrutura final do Hud

18.2 Criando um Resource para a Hotbar

Antes de implementarmos o script da **Hotbar**, precisamos criar um **Resource** que será utilizado para destacar visualmente o item selecionado.

18.2.1 Criando o Resource de Estilização

Siga os passos abaixo para criar o recurso:

1. Navegue até a pasta **resources/styleboxes**.
2. Crie um novo **Recurso**.
3. Ao criar o recurso, selecione o tipo **StyleBoxFlat**.
4. Nomeie o arquivo como **selected_hotbar_slot.tres**.
5. No **Inspetor**, configure as propriedades conforme a Figura 118.

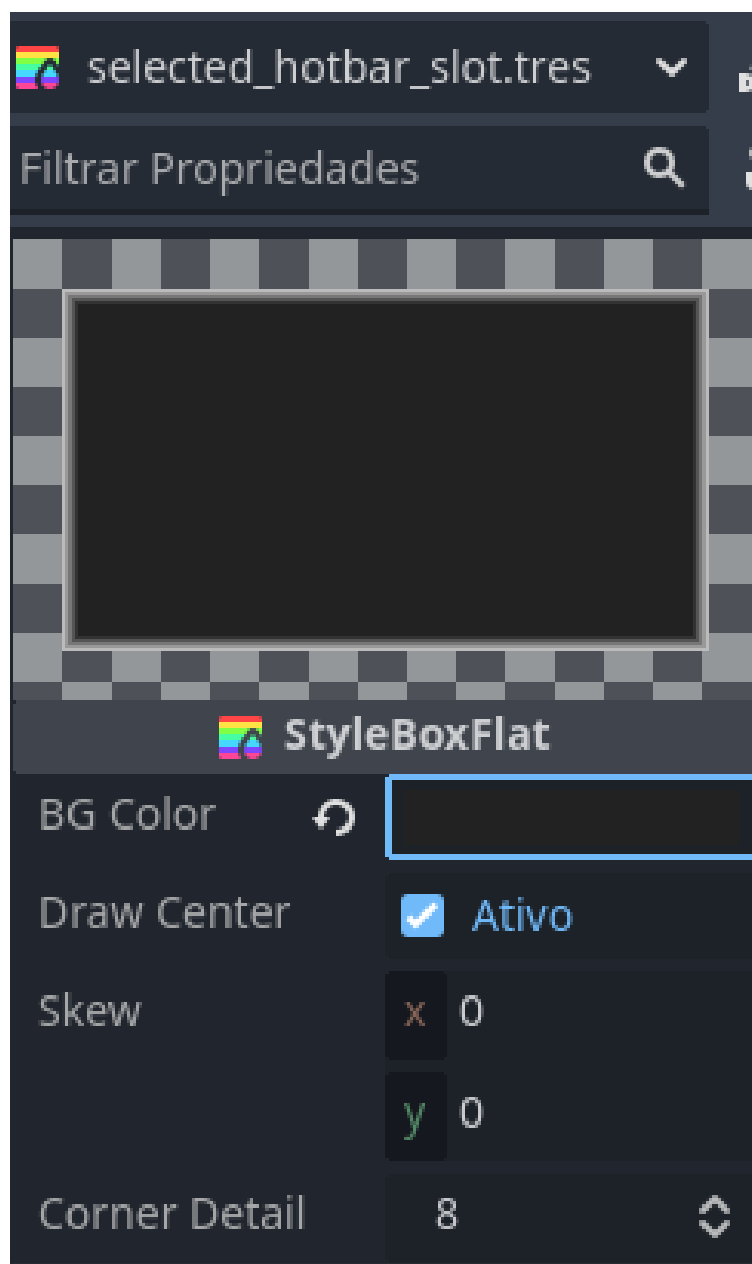


Figura 117: Configurações do recurso `selected_hotbar_slot.tres`

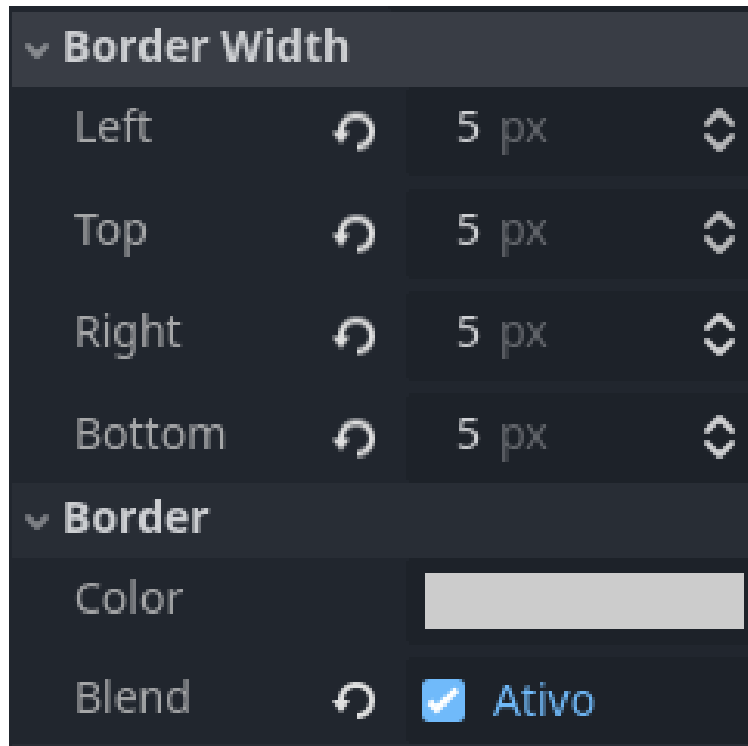


Figura 118: Configurações do recurso `selected_hotbar_slot.tres`

Cor utilizada em BG Color = #222222

18.3 Implementando o Script da Hotbar

Agora que criamos a estrutura da **Hotbar**, vamos adicionar a lógica para gerenciar os slots de itens. Para isso, criaremos um script no nó **Hotbar** e implementaremos as seguintes funções.

```
extends GridContainer

@export var slot_scene: PackedScene
var slots: Array[ItemSlot] = []
var selected_index: int = 0 # índice do slot atualmente selecionado
```

Variáveis do Script - **slot_scene**: Uma referência à cena base dos slots, permitindo instanciar novos slots na Hotbar. - **slots**: Um array que armazenará todos os slots criados. - **selected_index**: Índice do slot atualmente selecionado.

```
func _ready():
    setup_slots()
    InventoryManager.inventory_updated.connect(update_hotbar)
    select_slot(selected_index)
```

Função **_ready()** - **setup_slots()**: Inicializa os slots da Hotbar. - **Conecta o sinal do inventário**: Sempre que o inventário for atualizado, a Hotbar será atualizada automaticamente. - **Seleciona um slot inicial**: Define o primeiro slot como selecionado ao iniciar o jogo.



```
func setup_slots():
    for i in range(InventoryManager.HOTBAR_SIZE):
        var slot : ItemSlot = slot_scene.instantiate()
        add_child(slot)
        slots.append(slot)
        slot.set_item(InventoryManager.get_hotbar_item(i))
        slot.index_label.show()
        slot.index_label.text = str(i+1)
```

Função **setup_slots()** Essa função cria os slots da Hotbar e os configura corretamente.

1. Itera sobre o tamanho da Hotbar (**HOTBAR_SIZE**). 2. Instancia um novo **ItemSlot** usando a cena de slots. 3. Adiciona o slot à Hotbar. 4. Insere o slot no array **slots** para gerenciamento. 5. Define o item que será exibido no slot. 6. Exibe um número no slot indicando sua posição na Hotbar.

```
func update_hotbar():
    for i in range(InventoryManager.HOTBAR_SIZE):
        slots[i].set_item(InventoryManager.get_hotbar_item(i))
```

****Função update_hotbar()**** Essa função atualiza a Hotbar sempre que o inventário do jogador muda.

1. Percorre todos os slots da Hotbar. 2. Atualiza cada slot com os novos itens do inventário.

```
func _input(event):
    for i in range(9):
        if event.is_action_pressed("hotbar_%d" % (i + 1)):
            select_slot(i)
```

Função **_input(event)** Essa função permite ao jogador selecionar um slot da Hotbar pressionando as teclas numéricas.

1. Percorre os números de 1 a 9. 2. Se o jogador pressionar um dos atalhos (**hotbar_%d**), o slot correspondente é selecionado.

```
signal slot_selected(index)

func select_slot(index: int):
    if index >= 0 and index < InventoryManager.HOTBAR_SIZE:
        clear_selected_style()
        selected_index = index
        set_selected_style(slots[selected_index])
        slot_selected.emit(index) # Emite o sinal para atualizar a mao
```

Função **select_slot(index)** Essa função permite alternar entre os slots da Hotbar.

1. Verifica se o índice do slot está dentro dos limites válidos. 2. Remove o estilo visual do slot anteriormente selecionado (**clear_selected_style()**). 3. Atualiza o índice do slot selecionado. 4. Aplica o estilo visual ao novo slot selecionado (**set_selected_style()**). 5. Emite um sinal para atualizar a mão do jogador com o novo item selecionado.

```
func clear_selected_style():
    for slot in slots:
        slot.remove_theme_stylebox_override("normal")
```



Função **clear_selected_style()** Essa função remove o estilo do slot anteriormente selecionado.

1. Percorre todos os slots. 2. Remove a estilização do slot ativo.

```
func set_selected_style(slot):  
    var selected_style: StyleBox = preload("res://resources/styleboxes/selected_hotbar_slot.tres")  
    slot.add_theme_stylebox_override("normal", selected_style)
```

Função **set_selected_style(slot)** Essa função estiliza o slot atualmente selecionado.

1. Pré-carrega o estilo visual do slot selecionado (**selected_hotbar_slot.tres**). 2. Aplica o estilo ao slot selecionado.

Com isso, finalizamos a implementação da Hotbar.



19 Finalizando Mecânicas de Cultivo

Para finalizar as mecânicas de cultivo, precisamos garantir que as animações sejam ativadas no momento correto. Para isso, faremos algumas modificações no arquivo **player.gd** e no **Interacter**.

19.1 Sincronizando o AnimationTree do Hand com o Player

No arquivo **player.gd**, encontramos a função **set_walk()**, que atualmente apenas define a condição **walking** do **AnimationTree** do Player. Precisamos garantir que ela também modifique a condição do **AnimationTree** do nó **Hand**.

1. Primeiro, importe o **AnimationTree** do nó **Hand** usando o seguinte código:

```
@onready var hand_anim_tree : AnimationTree = $Hand/AnimationTree
@onready var hand_anim_state: AnimationNodeStateMachinePlayback = hand_anim_tree.get("parameters/playback")
```

2. Agora, modifique a função **set_walk()** para que ela também altere a animação do **Hand**:

```
func set_walk(value: bool):
    animation_tree["parameters/conditions/walking"] = value
    animation_tree["parameters/conditions/idle"] = not value
    hand_anim_tree["parameters/conditions/walking"] = value
    hand_anim_tree["parameters/conditions/idle"] = not value
```

Essa modificação permite que a animação **item_anim** seja iniciada quando o jogador se movimentar.

19.2 Ativando a Animação de Interação

Além da movimentação, precisamos garantir que a animação de interação seja ativada corretamente ao executar ações como plantar, regar ou colher.

1. Primeiro, importe o **AnimationTree** do nó **Hand** no arquivo do **Interacter**:

```
@onready var hand_anim_tree: AnimationTree = get_node("../Hand/AnimationTree")
```

2. Agora, crie a função **set_item_interagir()** para ativar a animação de interação:

```
func set_item_interagir(value: bool):
    hand_anim_tree["parameters/conditions/interagindo"] = value
```

3. Por fim, precisamos garantir que essa função seja chamada sempre que houver uma interação. Para isso, modificaremos a função **_process()** no **Interacter**:



```
func _process(_delta):
    if Input.is_action_just_pressed("interacao"):
        interact()
        set_item_interagir(true)
    if Input.is_action_just_pressed("plant"):
        plant(CropTypes.CROP_TYPE.CORN)
        set_item_interagir(true)
    if Input.is_action_just_pressed("prepare_ground"):
        prepare_ground()
        set_item_interagir(true)
    if Input.is_action_just_pressed("watering_crop"):
        watering_crop()
        set_item_interagir(true)
    if Input.is_action_just_pressed("fertilizing"):
        fertilizing_soil()
        set_item_interagir(true)

    var crop_ray = raycast(2)
    var soil_ray = raycast(4)
    var ground_ray = raycast(1)
    elif crop_ray:
        emit_signal("update_prompt_text", crop_ray.collider.update_prompt_text())
    elif soil_ray:
        emit_signal("update_prompt_text", soil_ray.collider.update_prompt_text())
    elif ground_ray and item_atual:
        emit_signal("update_prompt_text", ground_ray.collider.update_prompt_text())
    else:
        emit_signal("update_prompt_text", "")

    await get_tree().create_timer(0.1).timeout
    set_item_interagir(false)
```

No final da função, utilizamos um **timer** para desligar a animação de interação após um curto período de tempo.

19.3 Sincronizando Item da Hotbar com Animação

Agora precisamos garantir que o item selecionado na **Hotbar** apareça corretamente na mão do jogador. Para isso, vamos criar uma cena específica para representar a ferramenta usada.

19.3.1 Criando a Cena da Enxada

Primeiro, criamos uma nova cena para permitir a instanciação da enxada quando ela estiver equipada.

1. Navegue até a pasta **scenes/itens**.
2. Crie uma nova cena chamada **enxada.tscn**.
3. Defina o nó raiz como um **Node3D** e renomeie-o para **Enxada**.
4. Arraste o modelo 3D **enxada.glb** da pasta **assets/models** para a cena **enxada.tscn**.
5. Ajuste o posicionamento da **enxada.glb** para que fique corretamente alinhada na cena (Figura 119).

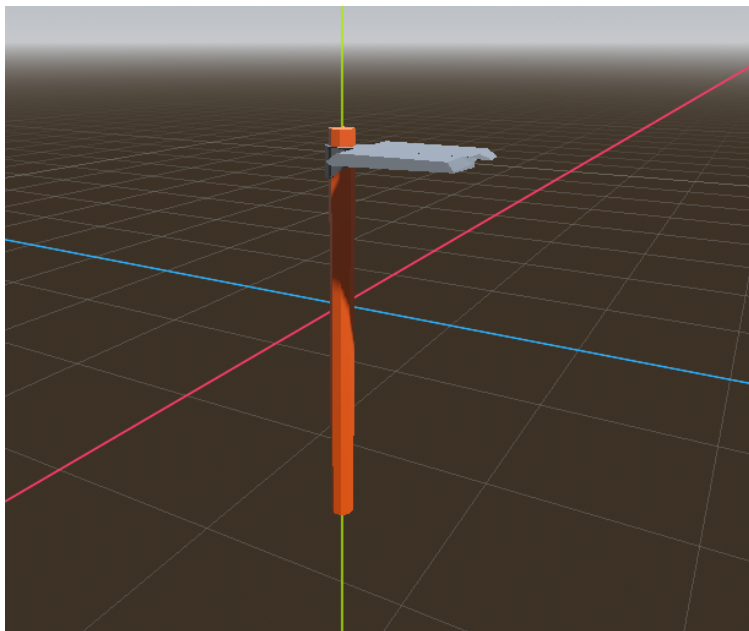
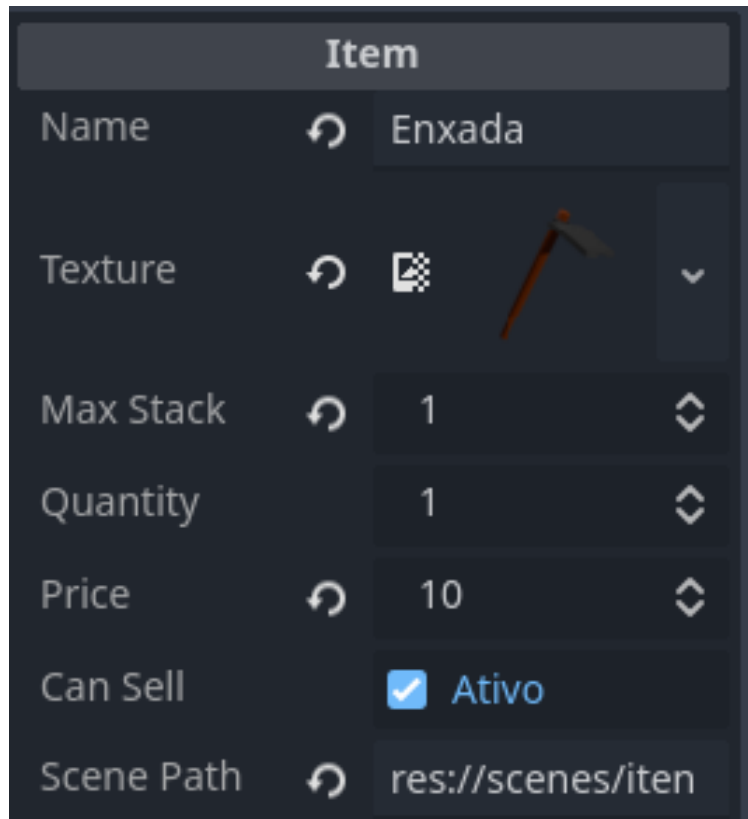


Figura 119: Posicionamento ajustado da enxada na cena

19.3.2 Criando o Recurso da Enxada

Agora, precisamos criar um **Resource** para representar a enxada dentro do jogo.

1. Navegue até a pasta **recursos/items**.
2. Crie um novo recurso do tipo **item.gd**.
3. Nomeie o recurso como **enxada.tres**.
4. No **Inspetor**, configure as seguintes propriedades (Figura 120):
 - **Texture**: Selecione a imagem da enxada localizada em **assets/icons**.
 - **Scene Path**: Defina o caminho da cena da enxada: **res://scenes/itens/enxada.tscn**.

Figura 120: Configuração do recurso `enxada.tres`

19.4 Sincronizando a Hotbar com o Sistema de Ferramentas

Agora precisaremos modificar o script do nó **HUD** para permitir que a **Hotbar** seja acessada corretamente pelo sistema de ferramentas.

19.4.1 Modificando o Script do HUD

No script do nó **HUD**, adicionamos uma função para retornar a Hotbar:

```
extends Control

@onready var prompt: Label = $Prompt
@onready var hotbar: Node = $Margin/Margin/Hotbar

func get_hotbar():
    return hotbar
```

Essa função permitirá que outros nós do jogo acessem a Hotbar e manipulem os itens selecionados.

19.4.2 Criando o Script para o Nó Hand

Agora, criaremos um script para o nó **Hand**. Esse script será responsável por:

- Capturar o item selecionado na Hotbar.



- Carregar e exibir esse item na mão do jogador.
- Remover o item anterior sempre que um novo for equipado.

Código do script do nó **Hand**:

```
extends Node3D

var hud_scene = preload("res://scenes/ui/hud.tscn") # Carrega a cena
@onready var hud: Node = null
@onready var hotbar: Node = null

func _ready():
    var hud_instance = hud_scene.instantiate() # Instancia a HUD
    get_tree().get_root().add_child(hud_instance) # Adiciona na árvore de nós

    await get_tree().process_frame # Espera um frame para garantir que a HUD foi carregada
    hud = get_node_or_null("/root/Main/UI/HUD") # Pegando o HUD corretamente

    if hud:
        hotbar = hud.get_node_or_null("Margin/Margin/Hotbar") # Pegando o Hotbar dentro da HUD
        if hotbar:
            hotbar.connect("slot_selected", update_hand_item) # Conectando ao evento de seleção
            print("Hotbar conectado com sucesso!")
        else:
            print("ERRO: Hotbar não encontrado dentro da HUD.")
    else:
        print("ERRO: HUD não encontrado na árvore de nós.")
```

Essa parte do código:

- Instancia a HUD na árvore de nós do jogo.
- Aguarda um quadro (**process_frame**) para garantir que a HUD foi carregada corretamente.
- Obtém a referência da HUD e da Hotbar.
- Conecta o evento **slot_selected** da Hotbar à função **update_hand_item()**, garantindo que a mão do jogador exiba o item correto ao mudar de slot.

19.4.3 Atualizando o Item na Mão do Jogador

Agora, implementamos a função **update_hand_item()** que atualizará a ferramenta na mão do jogador com base no item selecionado na Hotbar:



```
func update_hand_item(slot_index: int):
    # Remove qualquer item antigo na mao, exceto os nós de animacao
    for child in get_children():
        if not (child is AnimationPlayer or child is AnimationTree):
            child.queue_free()

    print("Slot selecionado:", slot_index)

    # Obtém o item do slot selecionado
    var item = InventoryManager.get_hotbar_item(slot_index)
    print("Item recebido do InventoryManager:", item)

    if item == null:
        print("Nenhum item encontrado no slot!")
        return # Evita tentar carregar um item nulo

    # Se chegou aqui, significa que o item existe, mas pode não ter uma cena associada
    if item.scene_path == "":
        print("Erro: O item não tem um caminho de cena válido!")
        print(item.scene_path)
        return

    # Tenta carregar a cena do item
    var item_scene = load(item.scene_path)
    if item_scene:
        var item_instance = item_scene.instantiate()
        add_child(item_instance)
        print("Item instanciado na mão: " + item.scene_path)
    else:
        print("Erro: Não foi possível carregar a cena do item!")
```

Função `update_hand_item()`

- Remove qualquer item antigo na mão do jogador para evitar sobreposições.
- Obtém o item correspondente ao slot selecionado na Hotbar. corretamente.
- Verifica se o item é nulo ou não possui um caminho de cena válido, evitando erros..
- Carrega e instancia o item na mão do jogador caso seja um item válido.

19.5 Finalizando o Interacter

Agora que o sistema de **Hotbar** está funcional, precisamos integrá-lo ao **Interacter**, garantindo que apenas os itens corretos possam executar determinadas ações no jogo.

19.5.1 Atualizando a Função `_process`

Para integrar a Hotbar ao sistema de interação, modifique a função `_process` do **Interacter** da seguinte forma:



```
func _process(_delta):
    var hotbar = get_node_or_null("/root/Main/UI/HUD/Margin/Margin/Hotbar") # Obtendo a
        hotbar
    var item_atual

    if hotbar:
        var selected_item = InventoryManager.get_hotbar_item(hotbar.selected_index) #
            Pegando o item no slot selecionado
        if selected_item:
            item_atual = selected_item.name

    if Input.is_action_just_pressed("interacao") and item_atual == "Foice":
        interact()
        set_item_interagir(true)
    if Input.is_action_just_pressed("plant"):
        plant(CropTypes.CROP_TYPE.TOMATO)
        set_item_interagir(true)
    if Input.is_action_just_pressed("prepare_ground") and item_atual == "Enxada":
        prepare_ground()
        set_item_interagir(true)
    if Input.is_action_just_pressed("watering_crop") and item_atual == "Regador":
        watering_crop()
        set_item_interagir(true)
    if Input.is_action_just_pressed("fertilizing"):
        fertilizing_soil()
        set_item_interagir(true)
    if Input.is_action_just_pressed("spawn_chicken"):
        spawn_chicken()
        set_item_interagir(true)
    if Input.is_action_just_pressed("collect_egg"):
        collect_egg()
        set_item_interagir(true)

    var crop_ray = raycast(2)
    var soil_ray = raycast(4)
    var ground_ray = raycast(1)
    var chicken_ray = raycast(8)

    if chicken_ray:
        emit_signal("update_prompt_text", chicken_ray.collider.update_prompt_text())
    elif crop_ray:
        emit_signal("update_prompt_text", crop_ray.collider.update_prompt_text())
    elif soil_ray:
        emit_signal("update_prompt_text", soil_ray.collider.update_prompt_text())
    elif ground_ray and item_atual == "Enxada":
        emit_signal("update_prompt_text", ground_ray.collider.update_prompt_text())
    else:
        emit_signal("update_prompt_text", "")

    await get_tree().create_timer(0.1).timeout
    set_item_interagir(false)
```



19.5.2 Explicação das Modificações

- **Verificação da Hotbar:** Obtém a referência da Hotbar e verifica qual item está selecionado no momento.
- **Ações específicas por ferramenta:**
 - **Foice:** Permite a interação com objetos colhíveis.
 - **Enxada:** Permite preparar o solo.
 - **Regador:** Permite regar as plantas.
 - **Itens Livres:** Algumas interações, como fertilizar o solo e plantar, não exigem um item específico.
- **Sistema de Raycast:** Verifica a colisão com diferentes objetos e atualiza o texto da interface para exibir instruções ao jogador.
- **Temporizador:** Desativa a animação de interação após um curto período de tempo.

19.5.3 Conclusão

Com essa implementação, agora apenas os itens apropriados podem executar certas ações no jogo. Isso adiciona um nível de realismo e complexidade à mecânica de cultivo e interação.



20 Implementando a Loja

Neste capítulo, vamos explorar a implementação da loja do jogo, explicando os scripts responsáveis pelo funcionamento e como configurá-los no editor da Godot.

20.1 Visão Geral da Loja

A loja permite que o jogador compre e venda itens usando dinheiro do jogo. Ela é composta por um inventário de itens disponíveis para compra e um sistema de gerenciamento de dinheiro.

Os principais scripts envolvidos são:

- **shop.gd** - Gerencia a interface da loja e a interação com o jogador.
- **shop_items.gd** - Define os itens que podem ser vendidos na loja.
- **shop_manager.gd** - Gerencia a lógica da loja, incluindo dinheiro e transações.

20.2 Explicando o Código

20.2.1 Script shop.gd

O script **shop.gd** controla a interface da loja e exibe os itens disponíveis para compra e venda.

```
extends Control

@export var slot_scene: PackedScene
@onready var player_inventory_grid: GridContainer = $Margin/Panel/Margin/VBox/
    PlayerInventoryGrid
@onready var shop_inventory_grid: GridContainer = $Margin/Panel/Margin/VBox/ShopGrid
@onready var money_label: Label = $Margin/Panel/Margin/VBox/HBox/Money

def _ready():
    initialize_ui()
    InventoryManager.inventory_updated.connect(refresh_ui)
    ShopManager.shop_updated.connect(refresh_ui)
    ShopManager.money_updated.connect(refresh_money)
```

Aqui, os elementos da interface são referenciados e configurados. Os sinais de atualização são conectados para manter a interface sincronizada com o estado da loja.

A função **initialize_ui()** cria os slots do inventário e da loja:



```
def initialize_ui():
    clear_children(player_inventory_grid)
    clear_children(shop_inventory_grid)

    for i in range(InventoryManager.INVENTORY_SIZE + InventoryManager.HOTBAR_SIZE):
        var slot = slot_scene.instantiate()
        slot.slot_index = i
        slot.add_to_group("inventory_slots")
        player_inventory_grid.add_child(slot)

    for i in range(ShopManager.SHOP_SIZE):
        var slot = slot_scene.instantiate()
        slot.slot_index = i
        slot.add_to_group("shop_slots")
        shop_inventory_grid.add_child(slot)

    refresh_ui()
```

Ela garante que os slots do inventário e da loja sejam criados corretamente.

20.2.2 Script shop_items.gd

Este script define um recurso contendo a lista de itens da loja:

```
class_name ShopItems
extends Resource

@export var items: Array[Item] = []
```

Os itens são carregados no jogo a partir de um arquivo de recursos.

20.2.3 Script shop_manager.gd

Este script gerencia o funcionamento da loja.

Definição dos sinais e variáveis principais:

```
extends Node

signal money_updated(new_money)
signal shop_updated

const ROWS := 3
const COLUMNS := 9
const SHOP_SIZE := ROWS * COLUMNS

var shop_items: Array[Utils.ItemRow] = []
var player_money: int = 100
```

Os sinais são usados para notificar mudanças na loja e no dinheiro do jogador. A função **load_items_to_shop()** carrega os itens no inventário da loja:



```
def load_items_to_shop():  
    var items: Array[Item] = load("res://resources/shop/shop_items.tres").items  
    var item_index := 0  
    for row in range(ROWS):  
        for col in range(COLUMNS):  
            if item_index < items.size():  
                shop_items[row].slots[col] = items[item_index]  
                item_index += 1  
            else:  
                break  
    shop_updated.emit()
```

Esta função distribui os itens do recurso **shop_items.tres** na grade da loja.
A função **buy_item()** permite que o jogador compre um item:

```
def buy_item(item: Item) -> bool:  
    if remove_money(item.price):  
        return true  
    return false
```

Ela verifica se o jogador tem dinheiro suficiente antes de concluir a compra.
A função **sell_item()** permite vender um item:

```
def sell_item(item: Item) -> bool:  
    if item.can_sell:  
        add_money(ceil(item.price * item.quantity * 0.8))  
        return true  
    return false
```

Ela adiciona dinheiro ao jogador baseado no valor do item vendido.

20.3 Configuração no Editor

Para configurar a loja no editor:

1. Crie um novo **Control** e adicione um painel com duas grades (extttGridContainer) para o inventário do jogador e o inventário da loja.
2. Adicione um rótulo (extttLabel) para exibir o dinheiro do jogador.
3. Crie um novo recurso do tipo **ShopItems** e preencha-o com itens.
4. Adicione o script **shop.gd** ao painel principal.
5. Configure os caminhos dos nós no script para referenciar os elementos criados.
6. Execute o jogo e teste a interação com a loja.

Com essa implementação, o jogador pode comprar e vender itens na loja dentro do jogo.



21 Criando a Galinha

Neste capítulo, vamos criar a galinha do nosso jogo e configurar seu comportamento. A galinha se move aleatoriamente e, após um determinado tempo, pode botar ovos que o jogador pode coletar.

21.1 Criando a Cena da Galinha

Para começar, crie uma nova cena com um nó raiz do tipo **CharacterBody3D**. Renomeie este nó para **Galinha**. Essa escolha permite que a galinha se mova e interaja com o ambiente de maneira adequada dentro do sistema de física da Godot.

Em seguida, adicione os seguintes nós como filhos:

- **MeshInstance3D** – Representa a malha 3D da galinha (o modelo visual).
- **CollisionShape3D** – Define a forma de colisão para a galinha.
- **Timer** – Para controlar o tempo de postura dos ovos (opcional, mas pode ser útil).

Certifique-se de configurar a propriedade **Shape** do **CollisionShape3D** para um formato adequado, como uma **CapsuleShape3D**, para evitar problemas de colisão.

21.2 Criando o Script da Galinha

Agora, vamos adicionar um script para controlar o comportamento da galinha. Selecione o nó **Galinha**, clique em **Adicionar Script** e crie um novo script chamado **galinha.gd**.

21.2.1 Definição do script e variáveis

O primeiro passo é definir o script e as variáveis que serão usadas:

```
extends CharacterBody3D

var speed = 3.0
var move_duration = 2.0
var gravity = -9.8
@export var egg_interval = 5

var move_direction = Vector3.ZERO
var move_timer = 0.0
var egg_timer = 0.0
var egg_ready
```

Aqui, estamos definindo: - **speed**: Velocidade de movimento da galinha. - **move_duration**: Tempo que a galinha se move antes de escolher uma nova direção. - **gravity**: Gravidade aplicada à galinha. - **egg_interval**: Tempo necessário para a galinha botar um ovo. - **move_direction**: Direção atual do movimento. - **move_timer** e **egg_timer**: Temporizadores que controlam o movimento e a postura de ovos. - **egg_ready**: Indica se um ovo está pronto para ser coletado.

21.2.2 Inicialização da galinha

Agora, definimos a função **_ready()**, que será chamada quando a galinha for instanciada na cena:



```
func _ready():
    add_to_group("chickens")
    randomize()
    pick_random_direction()
    egg_timer = egg_interval
    egg_ready = false
```

Aqui: - Adicionamos a galinha a um grupo chamado **"chickens"**, útil para futuras interações no jogo. - Chamamos **randomize()** para garantir direções de movimento aleatórias. - Escolhemos uma direção inicial com **pick_random_direction()**. - Definimos o temporizador de ovos para o valor inicial. - Inicializamos **egg_ready** como falso.

21.2.3 Contagem regressiva para botar ovos

A função **_process(delta)** reduz o tempo necessário para que um ovo seja posto:

```
func _process(delta):
    if egg_timer <= 0:
        egg_ready = true
    else:
        egg_timer -= delta
        egg_ready = false
```

Ela reduz **egg_timer** a cada quadro e marca o ovo como pronto quando o tempo chega a zero.

21.2.4 Movimento da galinha

A função **_physics_process(delta)** controla o movimento da galinha:

```
func _physics_process(delta):
    move_timer -= delta

    if move_timer <= 0:
        pick_random_direction()

    velocity.x = move_direction.x * speed
    velocity.z = move_direction.z * speed

    if move_direction != Vector3.ZERO:
        look_at(global_position + move_direction, Vector3.UP)

    velocity.y += gravity * delta
    move_and_slide()
```

Aqui: - A cada quadro, reduzimos **move_timer**. Quando ele chega a zero, escolhemos uma nova direção. - Ajustamos a velocidade da galinha de acordo com sua direção. - Fazemos a galinha "olhar" para onde está indo. - Aplicamos a gravidade e movimentamos a galinha usando **move_and_slide()**.

21.2.5 Escolha aleatória de direção

A função **pick_random_direction()** determina para onde a galinha se moverá:



```
func pick_random_direction():
    move_timer = randf_range(1.0, move_duration)
    if move_timer < 1.5:
        move_direction = Vector3.ZERO
    else:
        move_direction = Vector3(
            randf_range(-1, 1),
            0,
            randf_range(-1, 1)
        ).normalized()
```

Se o tempo de movimento for muito curto, a galinha fica parada. Caso contrário, ela escolhe uma nova direção aleatória.

21.2.6 Interação do jogador com a galinha

A função **interact()** permite que o jogador colete o ovo quando estiver pronto, a função será chamada pelo interacter implementado no capítulo da colheita:

```
func interact():
    if egg_ready:
        var inventory_manager = get_node("/root/InventoryManager")
        var egg_resource = load("res://resources/items/ovo.tres")
        var egg_item = egg_resource.duplicate()
        inventory_manager.add_item_to_inventory(egg_item)
        egg_ready = false
        egg_timer = egg_interval
```

Quando a função é chamada: - Se um ovo estiver pronto, ele é adicionado ao inventário do jogador. - O temporizador de ovos é reiniciado.

Depois disso, é necessário alterar o interacter, semelhante ao que foi feito no capítulo da colheita:

```
if Input.is_action_just_pressed("spawn_chicken"):
    spawn_chicken()
if Input.is_action_just_pressed("collect_egg"):
    collect_egg()

var chicken_ray = raycast(8)
if chicken_ray:
    emit_signal("update_prompt_text", chicken_ray.collider.update_prompt_text())
```

21.2.7 Exibição do tempo para o próximo ovo

A função **update_prompt_text()** informa quando um ovo estará disponível:

```
func update_prompt_text() -> String:
    if egg_ready:
        return "Pressione Mouse1 para coletar o ovo"
    else:
        return "%.1f" % egg_timer + " até o próximo ovo."
```



Essa função retorna um texto indicando se há um ovo pronto ou quanto tempo falta para o próximo.

21.3 Configuração no Editor

Antes de programarmos a lógica da galinha, precisamos configurar sua cena no editor da Godot. Siga os passos abaixo para garantir que tudo esteja corretamente ajustado.

21.3.1 Criando a Cena da Galinha

1. No editor da Godot, clique em **Criar Nova Cena**.
2. Adicione um nó raiz do tipo **CharacterBody3D** e renomeie-o para **Galinha**.
3. Salve essa cena como **galinha.tscn** para facilitar sua reutilização.

21.3.2 Importando o Modelo da Galinha

Se você possui um modelo 3D da galinha, siga estes passos:

1. Vá até a aba **Filesystem** e localize o modelo 3D da galinha.
2. Clique com o botão direito no arquivo e selecione **Reimportar** caso precise ajustar algo.
3. Arraste o modelo para dentro da cena da galinha e certifique-se de que ele está posicionado corretamente.

Se estiver usando um modelo simples como um **MeshInstance3D**, adicione esse nó e escolha uma malha adequada.

21.3.3 Adicionando o Script na Cena

Agora, precisamos adicionar um script que controlará o comportamento da galinha:

1. Selecione o nó **Galinha**.
2. Clique em **Adicionar Script** e crie um novo arquivo chamado **galinha.gd**.
3. Certifique-se de que o script está anexado corretamente ao nó.

21.3.4 Configurando o CollisionShape3D

Para garantir que a galinha colida corretamente com o ambiente, precisamos adicionar uma forma de colisão:

1. Selecione o nó **Galinha** e clique com o botão direito.
2. Escolha **Adicionar Nó Filho** e selecione **CollisionShape3D**.
3. No inspetor, clique na propriedade **Shape** e escolha um **CapsuleShape3D** ou **SphereShape3D**, dependendo do formato do modelo.
4. Ajuste o tamanho do **CollisionShape3D** para que cubra o modelo da galinha sem ultrapassá-lo.



21.3.5 Configurando a Camada de Colisão

Para que a interação da galinha com o jogador funcione corretamente, precisamos configurar suas camadas de colisão:

1. Com o nó **Galinha** selecionado, vá até o inspetor e localize a opção **Collision Layer**.
2. Certifique-se de que a galinha está em uma camada específica (por exemplo, a camada 2).
3. Marque a mesma camada em **Mask** para garantir que a galinha possa detectar interações.

Após configurar tudo corretamente, sua cena da galinha estará pronta para ser utilizada no jogo!

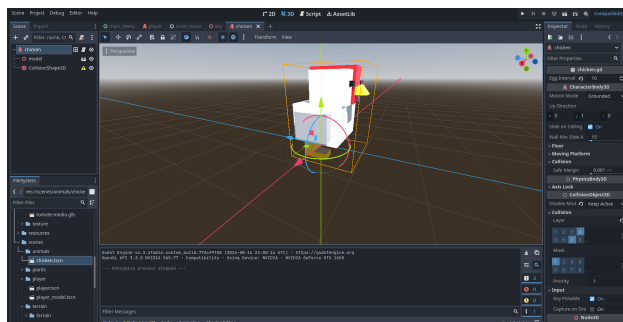


Figura 121: Galinha no Editor